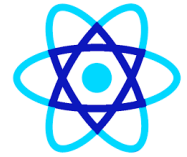


React js

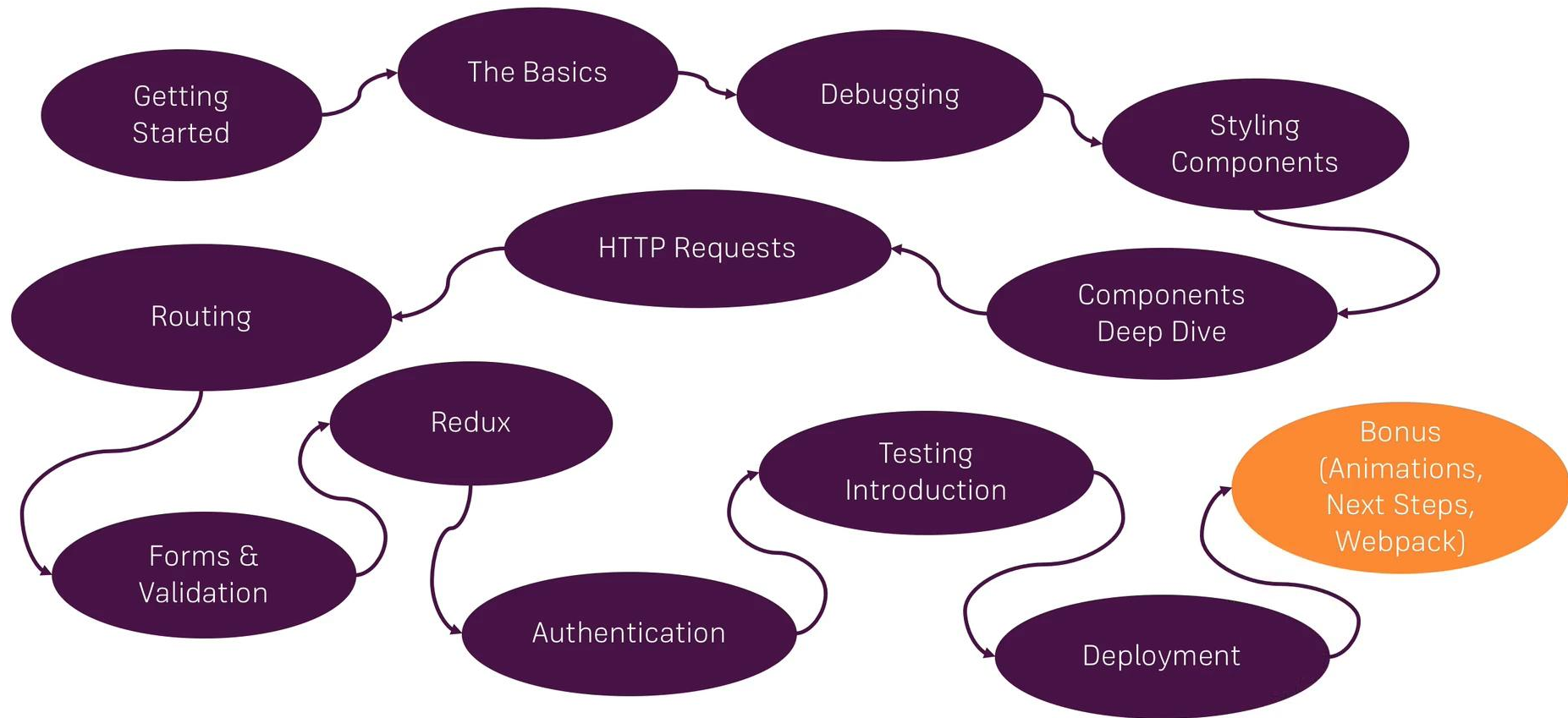
Parameswari E

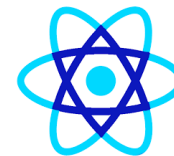
High performance. Delivered.

consulting | technology | outsourcing



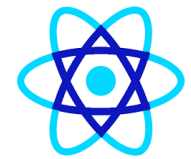
Course Outline



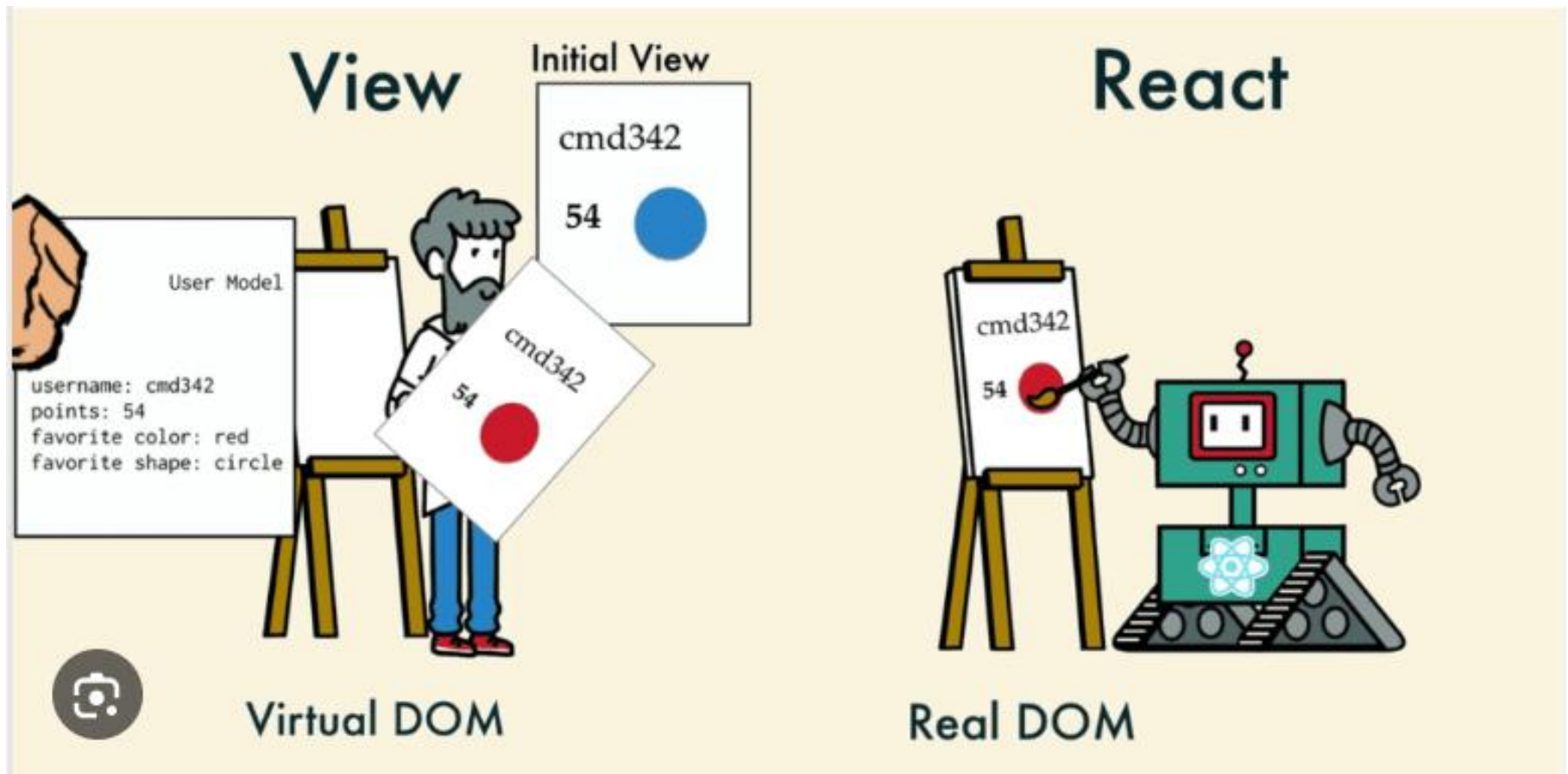


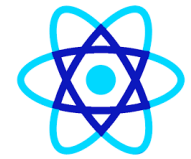
Getting Started

What? Why? How?



What is React



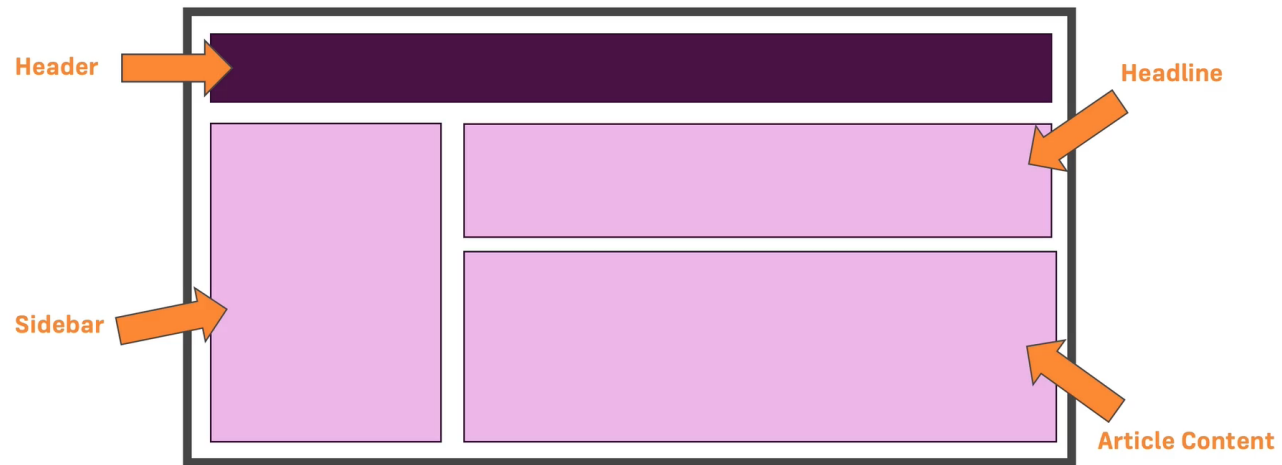


What is React

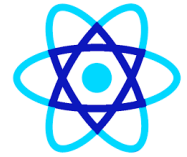
"A JavaScript Library for building User Interfaces"



Components



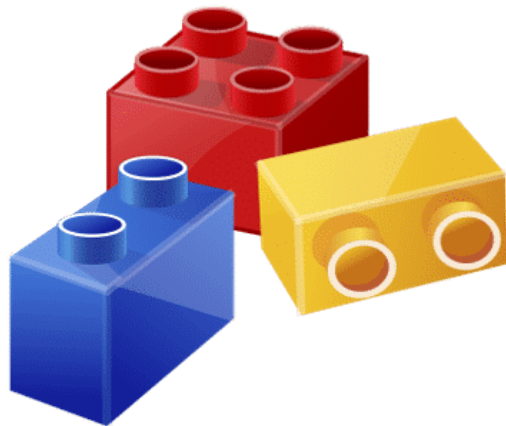
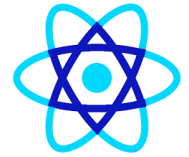
What Is React?



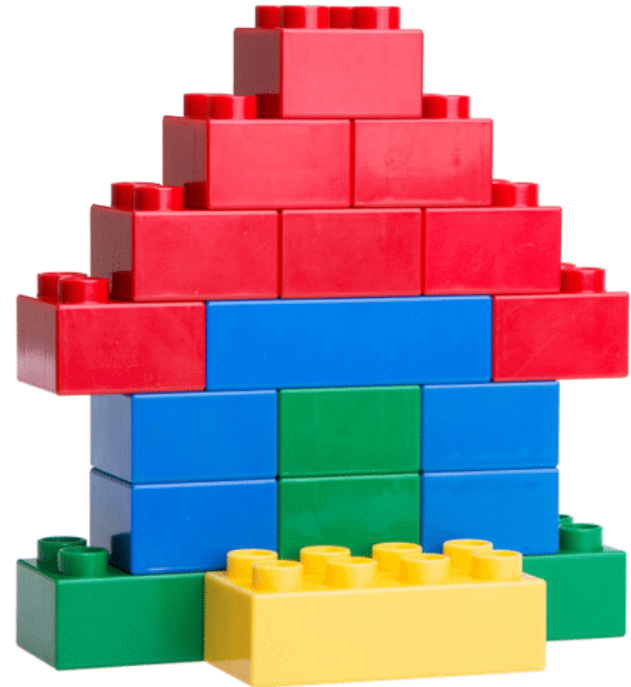
- **React** is an **open-source JavaScript library** (by Meta/Facebook) used to build **fast, interactive user interfaces**, especially **Single Page Applications (SPAs)**.

It lets you build UIs using **components**—small, reusable pieces that manage their own state and render efficiently.

What Is React?



Building Blocks



Final Product



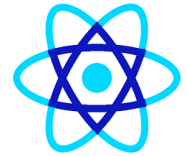
What is React

- ReactJS was developed by *Jordan Walke*, a software engineer working at Facebook.
- Facebook implemented ReactJS in 2011 in its *newsfeed* section, but it was released to the public in May 2013.
- After the implementation of ReactJS, Facebook's UI underwent drastic improvement.
- This resulted in satisfied users and a sudden boost in its popularity.

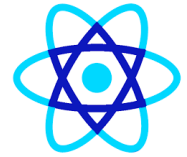


Why React is so popular

- **1) Component-based architecture**
 - Build UIs as reusable components (buttons, forms, dashboards).
 - Easier maintenance and scalability for large apps.
- **2) Fast performance (Virtual DOM)**
 - React updates a lightweight **Virtual DOM** first, then efficiently updates only what changed in the real DOM.
 - Results in smoother, faster UIs.

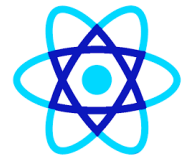


- **3) Declarative & predictable**
 - You describe *what* the UI should look like for a given state; React handles *how* to update it.
 - Less error-prone, easier to reason about.
- **4) Huge ecosystem & community**
 - Rich libraries: routing, state management, charts, forms, UI kits.
 - Massive community support, tutorials, and long-term stability.
- **5) Works everywhere**
 - **Web** (React)
 - **Mobile** (React Native)
 - **Desktop** (Electron + React)
 - **Server-side rendering** (Next.js) for SEO and performance.



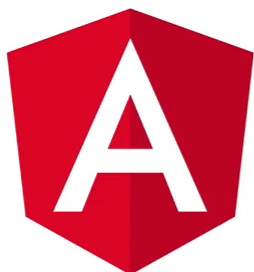
- **6) Backed by Meta & industry adoption**
 - Used by companies like Facebook, Instagram, Netflix, Airbnb, Uber.
 - Strong job demand and long-term viability.
- **7) Easy integration**
 - Can be added to an existing project incrementally.
 - Plays well with REST APIs, GraphQL, and modern tooling.

Java script Frameworks

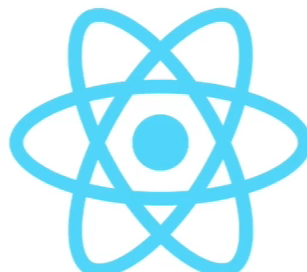




React Alternatives



Angular

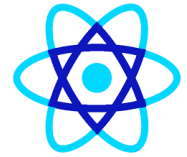


React

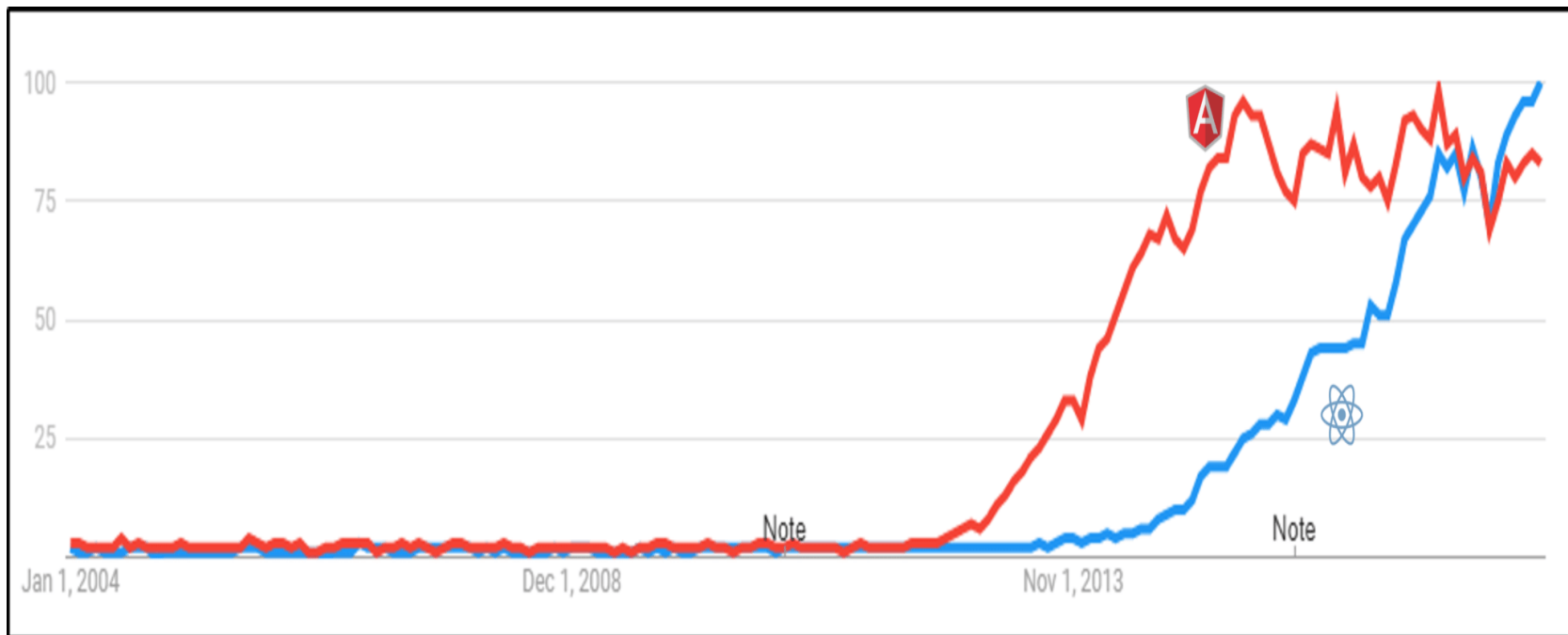


Vue

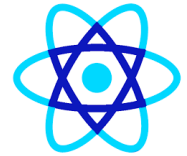




- ***Even though React is young, it is giving a neck to neck competition to Angular***

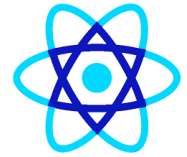


SPA vs Traditional (Multi-Page) Applications



- **1 Traditional Web Applications (MPA)**
- **MPA = Multi-Page Application**
- **How it works**
 - Every user action (click, submit) sends a request to the server
 - Server returns a **new HTML page**
 - Browser reloads the entire page
- **Example stack**
 - JSP / Servlet
 - PHP
 - ASP.NET MVC
 - Django / Rails (server-rendered)
- **Characteristics**
 - Full page reload on each request
 - UI logic mostly on the server
 - Slower user experience
 - Easier SEO by default

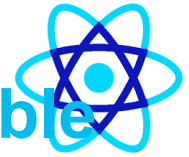
SPA vs Traditional (Multi-Page) Applications



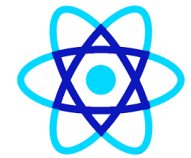
- **[2] Single Page Applications (SPA)**
- **SPA loads once, updates dynamically**
- **How it works**
 - Browser loads **one HTML page**
 - JavaScript (React / Angular / Vue) updates content dynamically
 - Backend exposes **APIs (REST / GraphQL)**
 - No full page reloads
- **Example stack**
 - React / Angular / Vue
 - Backend: Spring Boot, Node.js, .NET Web API
- **Characteristics**
 - Smooth, app-like experience
 - Faster navigation
 - Client-side routing
 - UI logic on the browser



SPA vs Traditional Apps – Comparison Table



Feature	Traditional App (MPA)	SPA
Page Reload	Every request	Only once
Speed	Slower	Faster
User Experience	Basic	App-like
Backend Role	HTML + Data	APIs only
Frontend Logic	Minimal JS	Heavy JS
SEO	Easy	Needs SSR (Next.js)
Scalability	Limited	High
Mobile-friendly	Average	Excellent



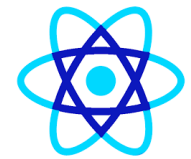
✓ When to use Traditional Apps

- Simple websites
- Content-heavy sites (blogs, news)
- Minimal interactivity
- SEO-critical pages

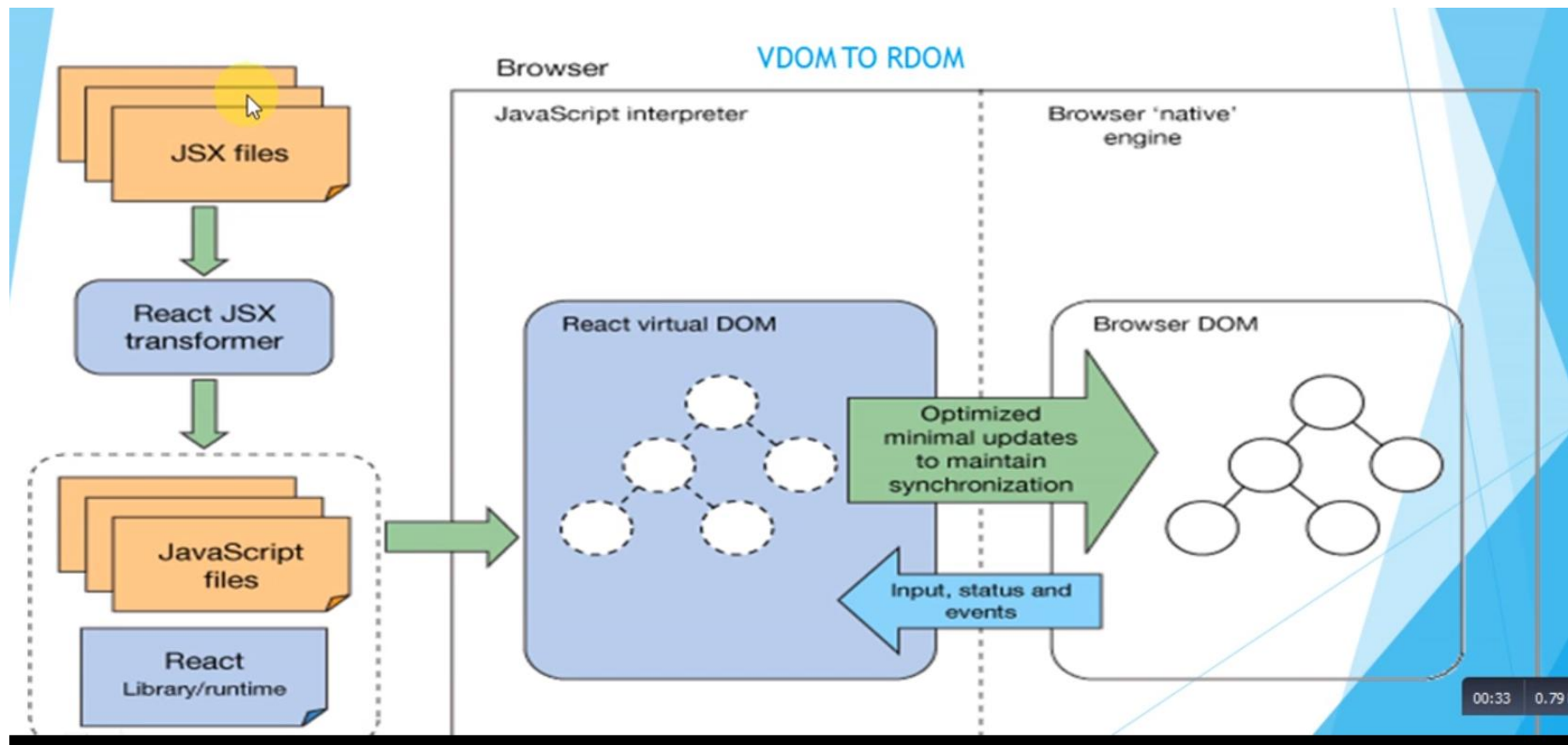


When to use SPA

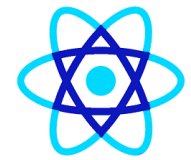
- Dashboards & admin portals
- Banking, insurance, enterprise apps
- Real-time apps (chat, analytics)
- Mobile-first applications



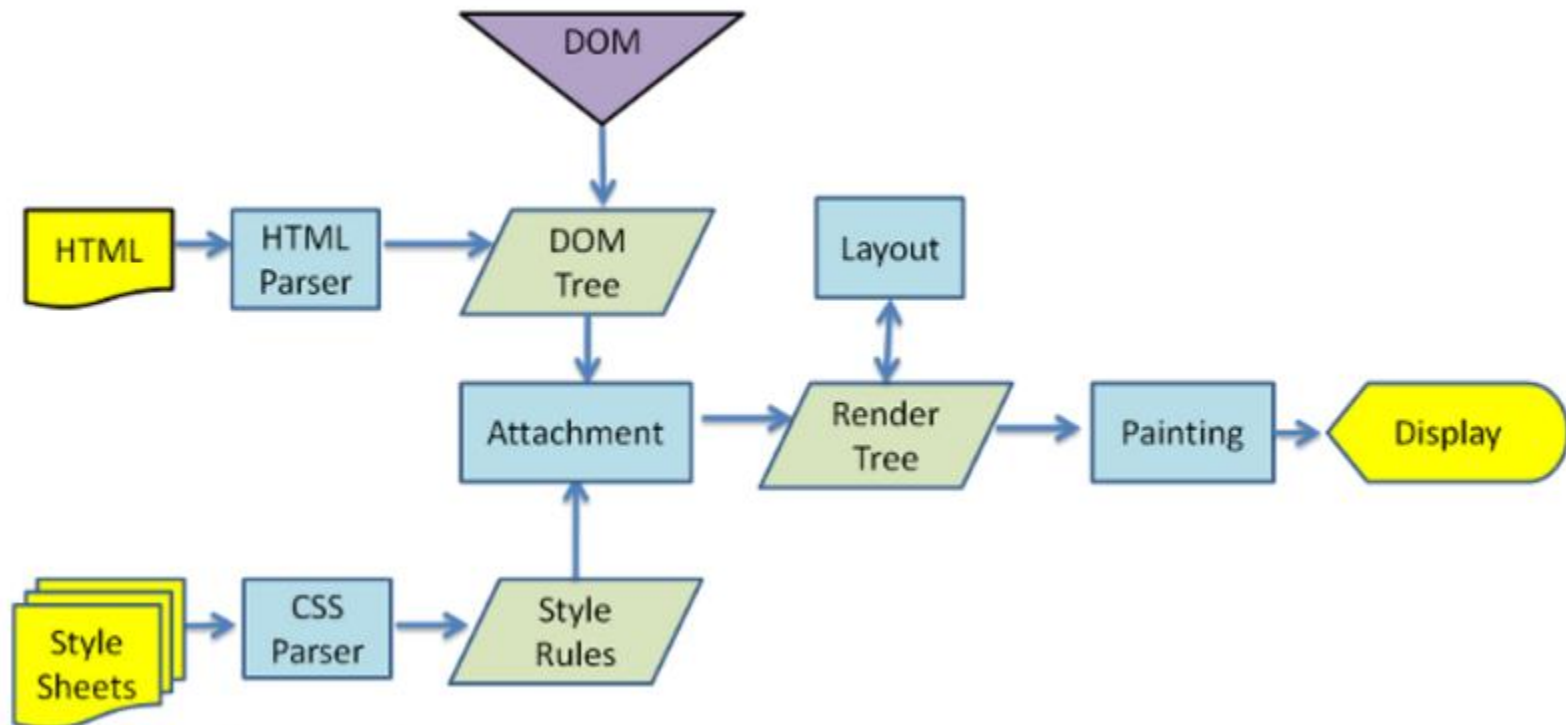
Reactjs Architecture

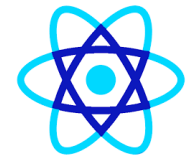


00:33 0.79



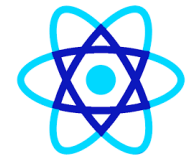
Why updating Real DOM is slow:





Why updating Real DOM is slow:

-  **Why the Real DOM is Slow**
 - DOM manipulation involves **layout, repaint, reflow**
 - Frequent updates cause performance bottlenecks
 - Direct DOM access blocks the browser rendering pipeline

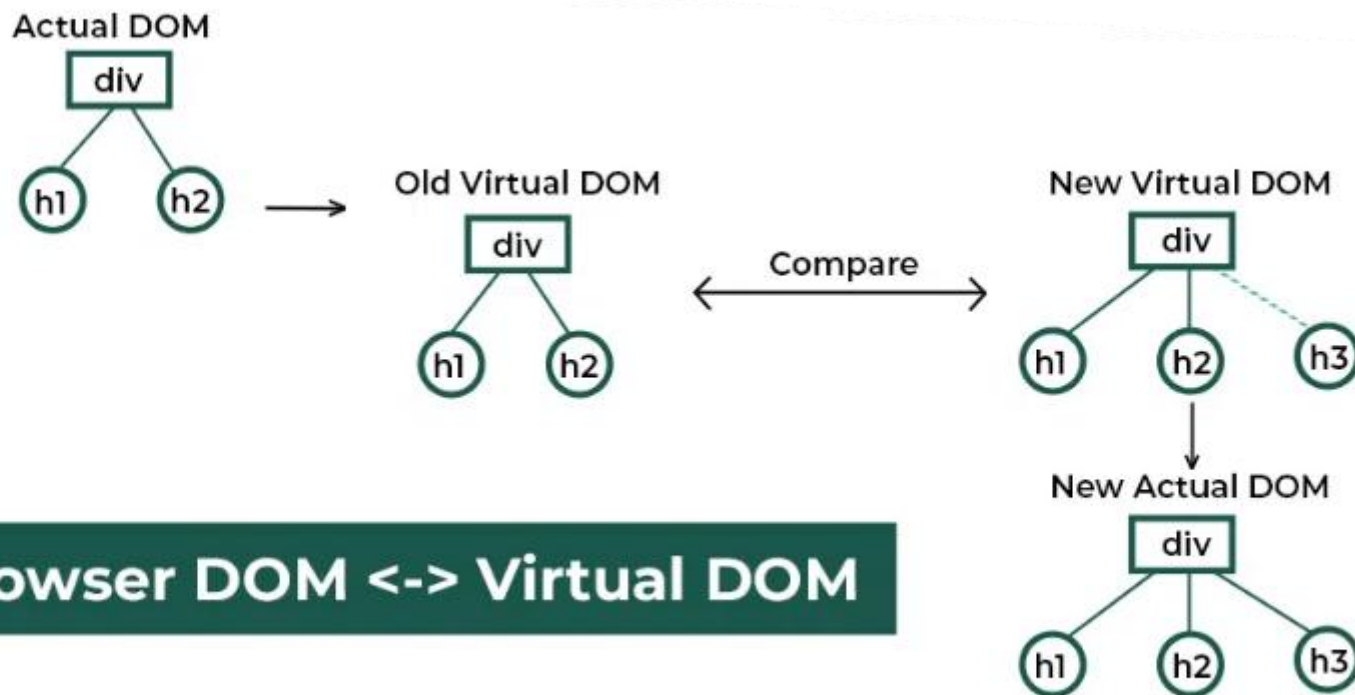


Virtual DOM

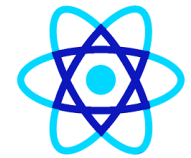
- The **Virtual DOM** is a **lightweight, in-memory copy of the real browser DOM** used by libraries like **React** to make UI updates **faster and more efficient**.
- Instead of updating the real DOM directly every time data changes (which is slow), React:
- Updates the **Virtual DOM**
- Compares it with the **previous Virtual DOM**
- Updates **only the changed parts** in the real DOM



Virtual DOM

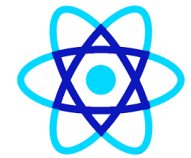


Browser DOM <-> Virtual DOM



How Virtual DOM Works (step by step)

-  **How Virtual DOM Works (Step-by-Step)**
- **Initial Render**
 - React creates a Virtual DOM tree from your components
 - It renders this to the real DOM
- **State / Props Change**
 - Component state or props change
 - A **new Virtual DOM** is created
- **Diffing (Reconciliation)**
 - React compares **old VDOM vs new VDOM**
 - Finds the **minimal set of changes**
- **Batch Update**
 - Only the changed nodes are updated in the real DOM
- 👉 **Result: Fast UI updates with minimal DOM operations**



Virtual DOM vs Real DOM

Virtual DOM	Real DOM
It is a lightweight copy of the original DOM	It is a tree representation of HTML elements
It is maintained by JavaScript libraries	It is maintained by the browser after parsing HTML elements
After manipulation it only re-renders changed elements	After manipulation, it re-render the entire DOM
Updates are lightweight	Updates are heavyweight
Performance is fast and UX is optimised	Performance is slow and the UX quality is low
Highly efficient as it performs batch updates	Less efficient due to re-rendering of DOM after each update



⚡ Why Virtual DOM is Faster

Feature	Real DOM	Virtual DOM
Update cost	Expensive	Cheap (JS object)
DOM operations	Many	Minimal
Performance	Slower	Faster
Re-render scope	Whole subtree	Only changed nodes

Without Virtual DOM

js

```
document.getElementById("count").innerText = count;
```

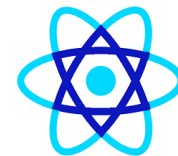
➡ Direct DOM update every time

With Virtual DOM (React)

jsx

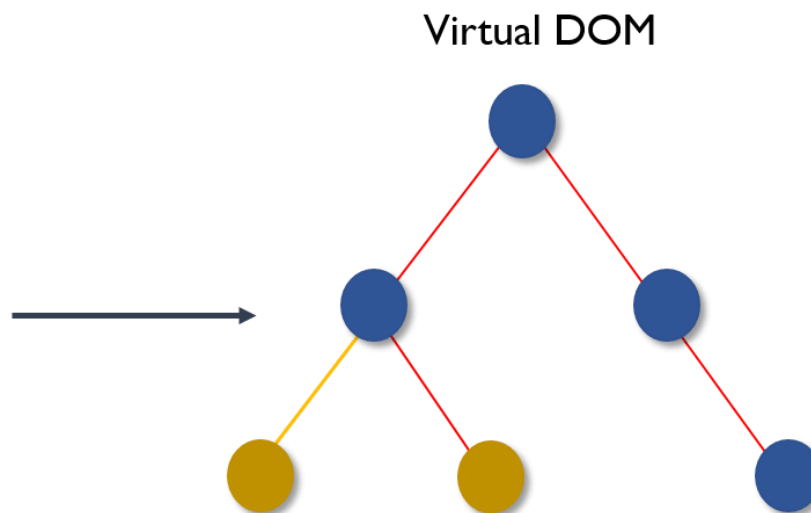
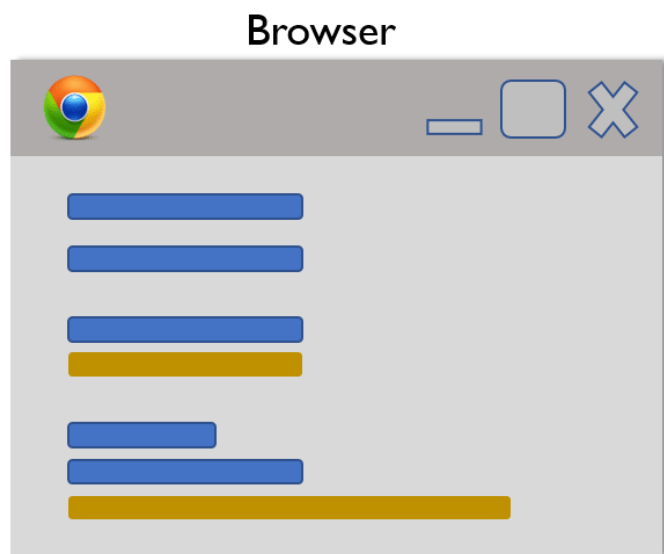
```
setCount(count + 1);
```

➡ React figures out what actually changed



Virtual DOM

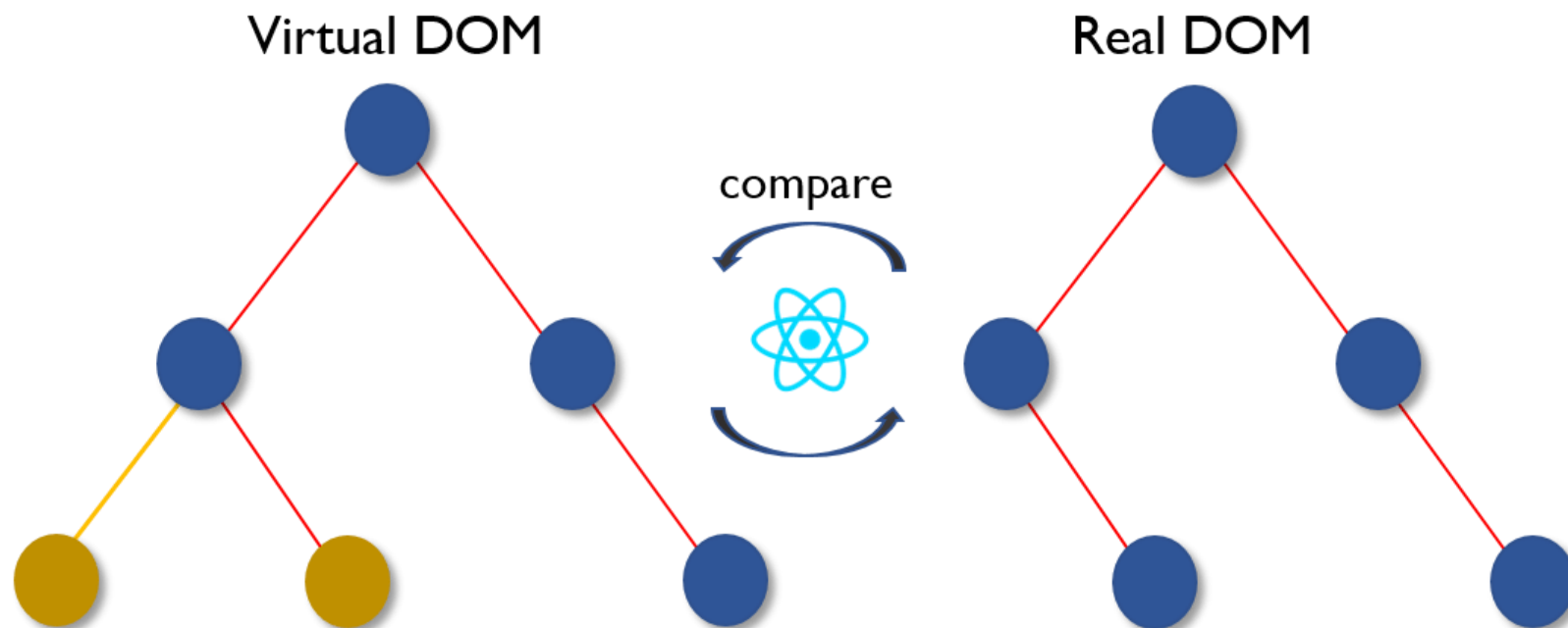
- Whenever any underlying data changes, the entire UI is re-rendered in Virtual DOM representation.

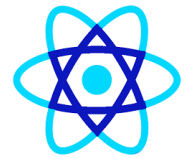




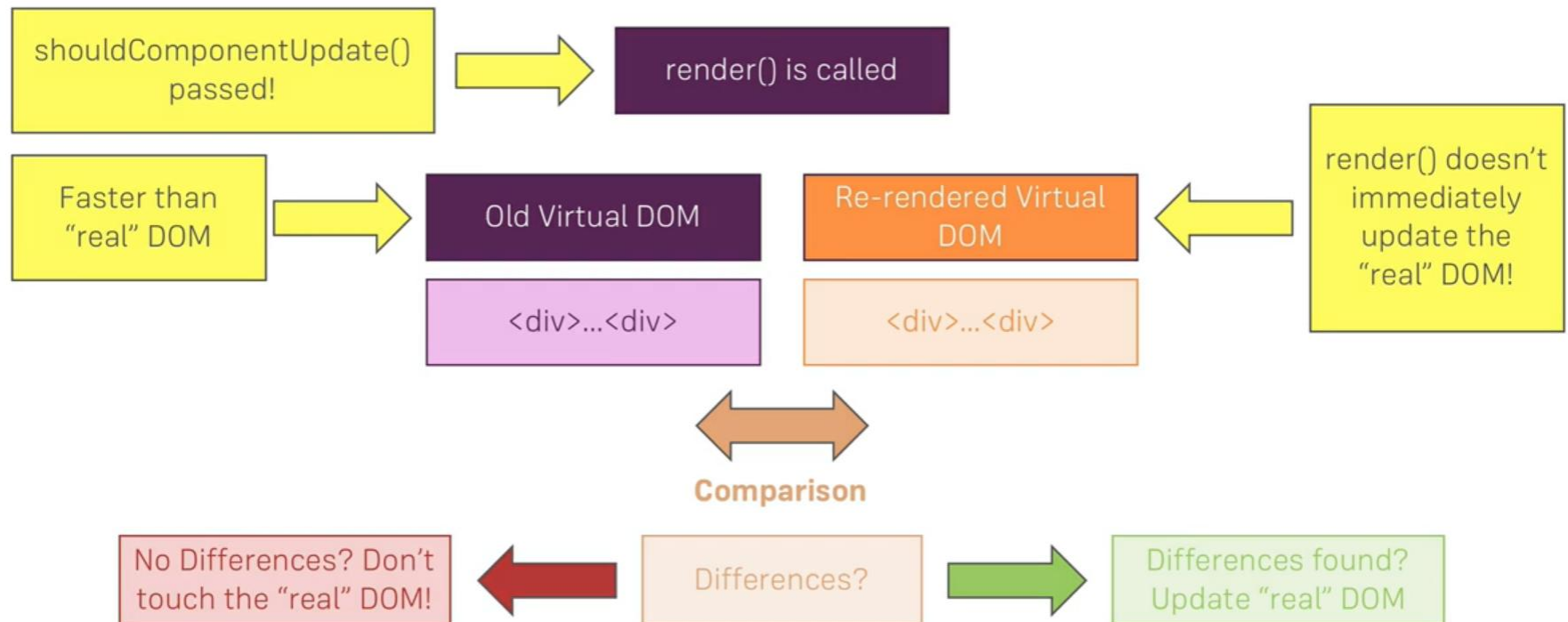
Virtual DOM

- Then the difference between the previous DOM representation and the new one is calculated..





How React Updates The DOM

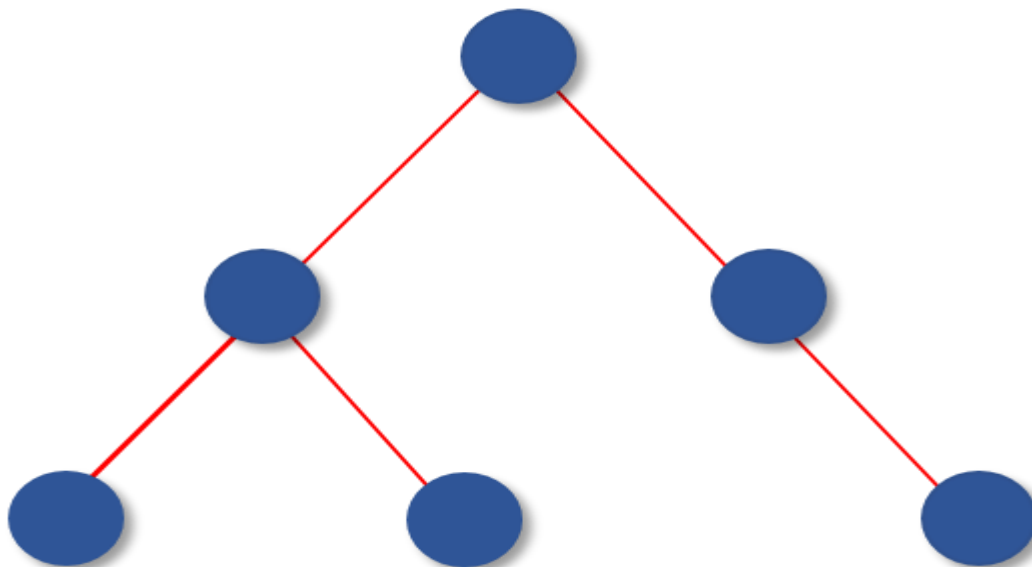




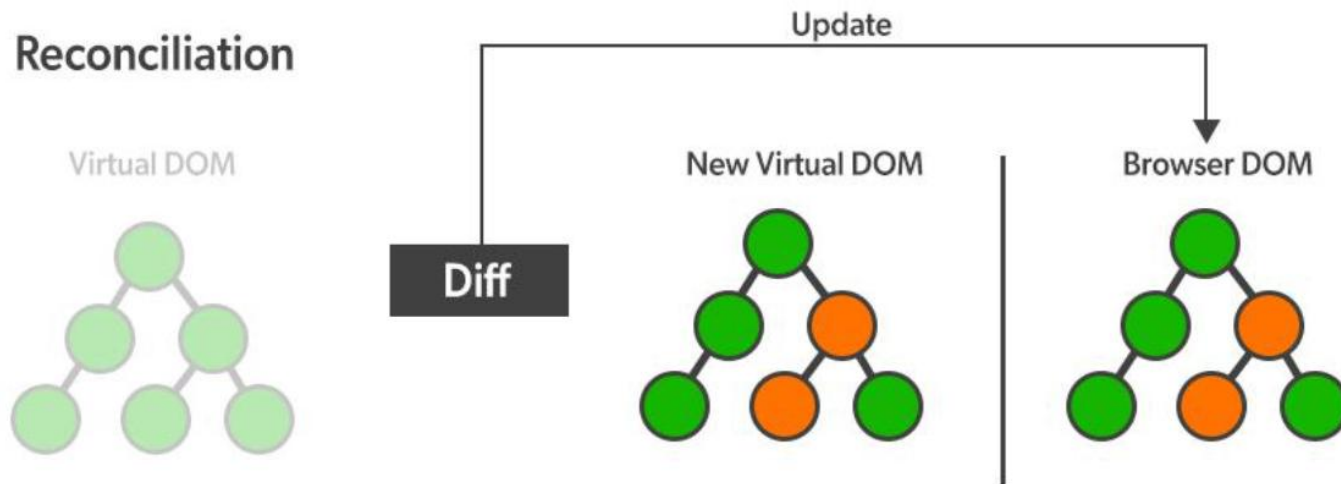
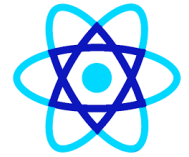
Virtual DOM

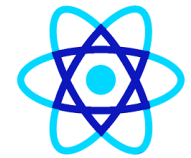
- Once the calculations are done, the real DOM will be updated with only the things that have actually changed. You can think of it as a patch. As patches are applied only to the affected area, similarly, the virtual DOM acts as patches and are applied to the elements which are updated or changed, in the real DOM.

Real DOM (updated)



Virtual DOM





Reactjs Architecture

Feature	Real DOM	Virtual DOM
Nature	Browser DOM	JS object (in memory)
Update speed	Slow	Fast
Re-render	Whole DOM / large parts	Only changed nodes
Memory	No extra memory	Uses additional memory
UI performance	Poor for frequent updates	Optimized
Direct access	Yes (document.getElementById)	No direct access
Used by	Vanilla JS, jQuery	React, Vue



Reactjs Architecture

- **1 JSX Files (Top-Left)**
- **What it shows:**
 -  JSX files
- **Meaning:**
- Developers write UI using **JSX** (HTML-like syntax in JavaScript)
- Example:
- `<button>Save</button>`
- **JSX is not understood by browsers directly.**



Reactjs Architecture

- **[2] React JSX Transformer (Build Time)**
- **What it shows:**
 -  React JSX transformer
- **Meaning:**
- JSX is converted into **plain JavaScript**
- Done by tools like:
 - **Babel**
 - **Vite**
 - **Webpack**
- JSX → `React.createElement()`
- `React.createElement("button", null, "Save")`
- ✓ Happens **before runtime**



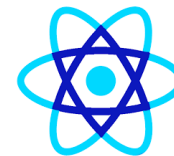
Reactjs Architecture

- **3 JavaScript Files + React Runtime**
- **What it shows:**
 - 📁 JavaScript files
 - 📦 React Library / runtime
- **Meaning:**
- Transformed JS + React runtime are loaded into the browser
- React runtime handles:
 - Component lifecycle
 - State updates
 - Virtual DOM creation



Reactjs Architecture

- **4 Browser – JavaScript Interpreter**
- **What it shows:**
 -  Browser
 -  JavaScript interpreter
- **Meaning:**
- Browser executes JavaScript using:
 - Chrome → V8
 - Firefox → SpiderMonkey
- React code now runs inside browser memory



Reactjs Architecture

- **5 React Virtual DOM (Core Concept)**
- **What it shows:**
 - React virtual DOM
- **Meaning:**
- Virtual DOM is a **lightweight JavaScript object tree**
- It represents **UI structure in memory**, NOT the real browser DOM



Reactjs Architecture

- **5 React Virtual DOM (Core Concept)**

- Example:

- {

- type: "div",

- children: [

- { type: "h1", text: "Hello" }

-]

- }

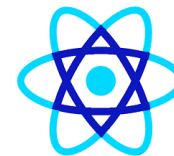
- ✓ Very fast to create and compare

- ✓ No browser repaint yet



Reactjs Architecture

- **6 Input, State & Events (Bottom Arrow)**
- **What it shows:**
 -  Input, status and events
- **Meaning:**
- User actions:
 - Click
 - Input
 - Scroll



Reactjs Architecture

- **[7] Diffing & Reconciliation**
- **What it shows:**
 - ➔ Optimized minimal updates to maintain synchronization
- **Meaning:**
- React compares:
 - Old Virtual DOM
 - New Virtual DOM
- Finds **only the differences**
- This process is called:
 - **Diffing**
 - **Reconciliation**
- ✓ Avoids full DOM re-render
- ✓ Improves performance



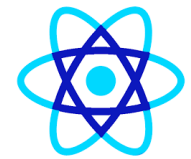
Reactjs Architecture

- **8 Browser Native Engine**
- **What it shows:**
 -  Browser 'native' engine
- **Meaning:**
- React now instructs the browser:
 - What exactly to update
 - Where to update
- Browser executes minimal DOM operations



Reactjs Architecture

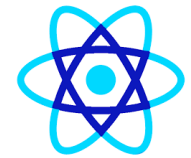
-  **Real DOM (RDOM)**
- **What it shows:**
 -  Browser DOM
- **Meaning:**
- Actual HTML DOM tree
- Only **changed nodes** are updated
- Browser repaints UI
- ✓ Faster than traditional full reload
- ✓ Smooth SPA experience



Reactjs Compilation



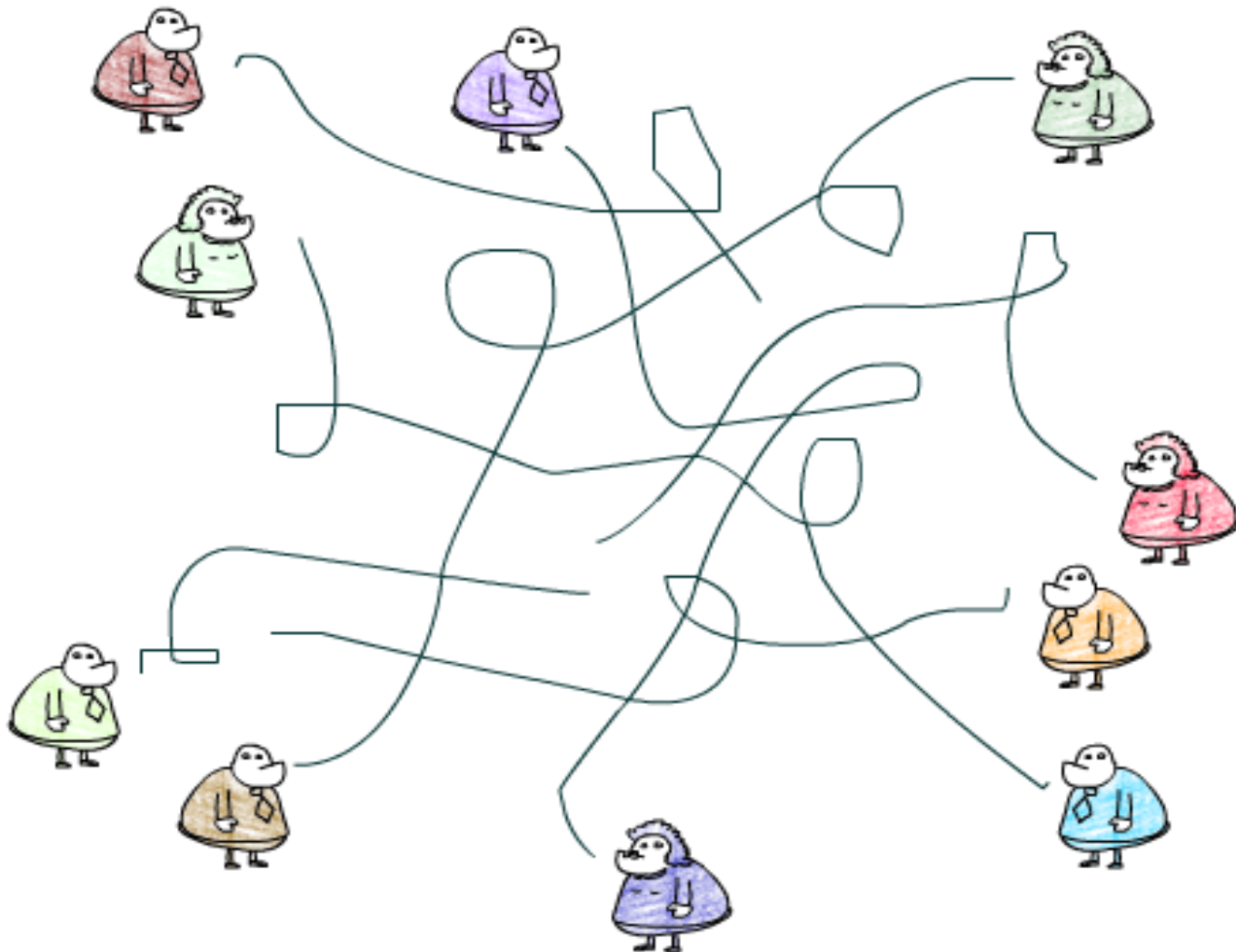
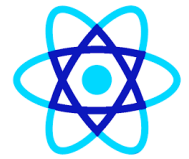
ESM (ES Modules)
HMR Hot Module Replacement



Why This Architecture is Powerful

Feature	Benefit
Virtual DOM	High performance
Diffing	Minimal updates
One-way data flow	Predictable UI
Component tree	Reusability
Browser-agnostic	Works everywhere

Why State Management

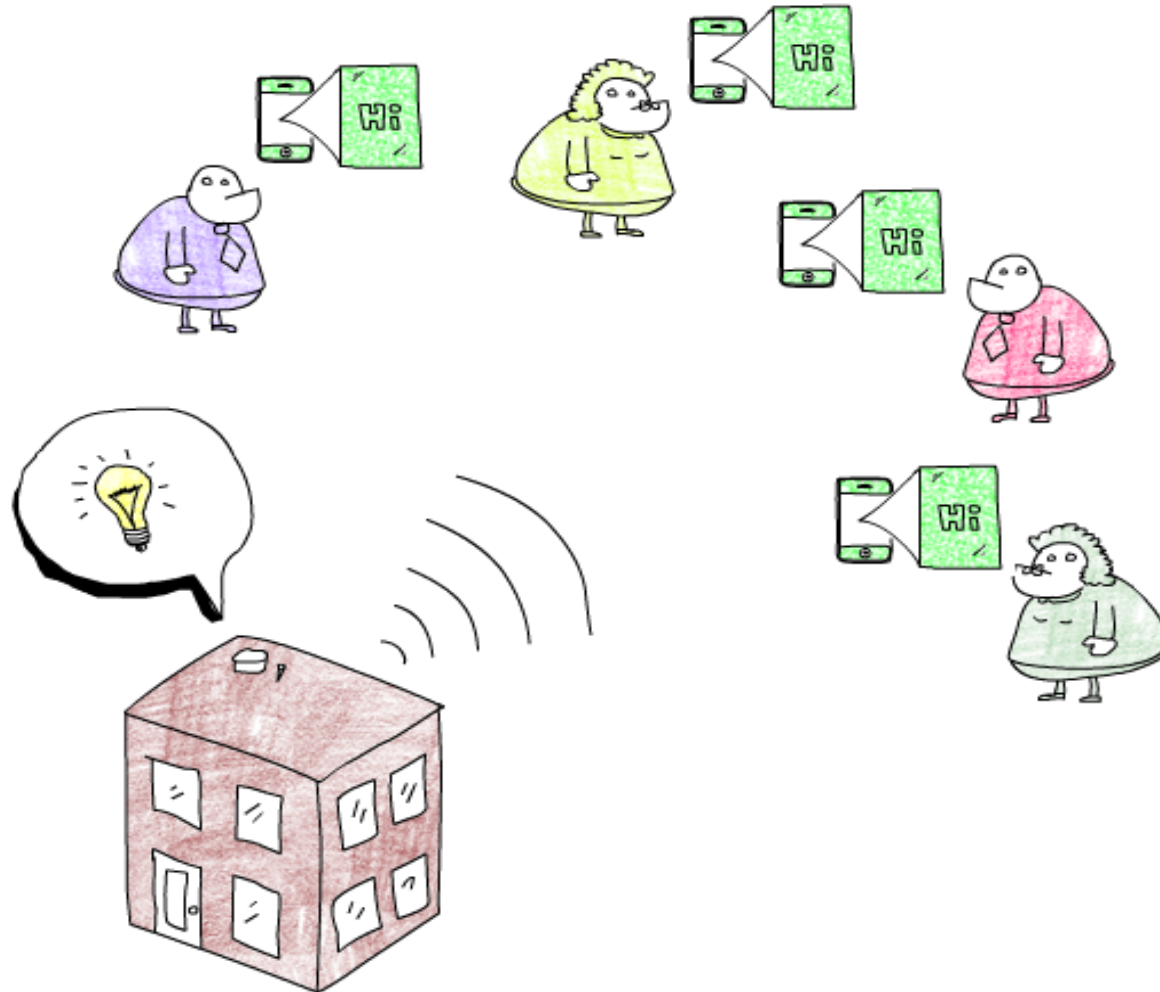
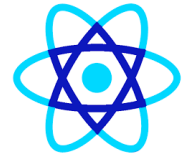


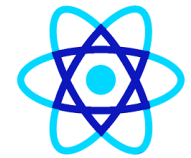


Why State Management

-  Problem shown in the image: *Without Redux (or centralized state)*
- What's happening here
- Components **share state directly**
- Data flows:
 - Parent → child
 - Child → sibling
 - Sibling → unrelated component
- Communication becomes **unpredictable and hard to track**
- This is called:
-  **Prop Drilling + Tight Coupling**
- Component A → B → C → D → E

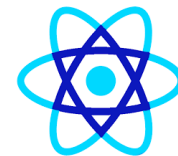
Why State Management





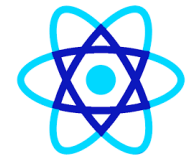
Why State Management

-  Red building (bottom-left) → **Redux Store**
- **What it represents**
- The **single source of truth**
- Holds the **entire application state**
 - store = {
 - user,
 - cart,
 - orders,
 - theme
 - }
- ✓ Only one store
- ✓ Centralized
- ✓ Predictable



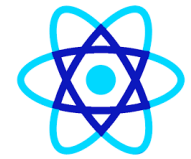
Why State Management

-  **Green blocks (Hi) → State data**
- **What they represent**
- Pieces of global state
- Examples:
 - Logged-in user
 - Notifications
 - Cart count
 - Dashboard data
- These values **live in the store**, not inside components.



Why State Management

-  **Colored characters → React Components**
- Each character:
 - Needs some data
 - May trigger state changes
 - **Is independent of other components**
- ✓ No direct coupling
 - ✓ Easy to reuse
 - ✓ Easy to test



Why State Management

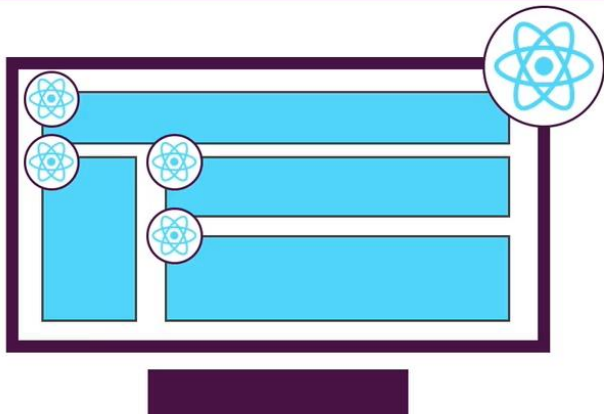
- 🧑 **Colored characters → React Components**
- Each character:
 - Needs some data
 - May trigger state changes
 - **Is independent of other components**
- ✓ No direct coupling
 - ✓ Easy to reuse
 - ✓ Easy to test



Two Kinds of Applications

Single Page Applications

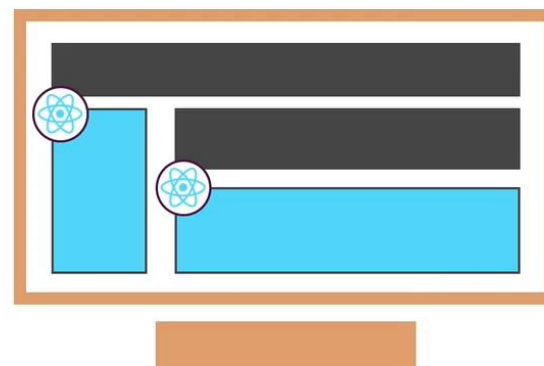
Only ONE HTML Page, Content is (re)rendered on Client



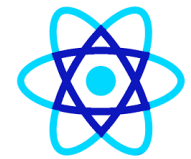
Typically only ONE ReactDOM.render() call

Multi Page Applications

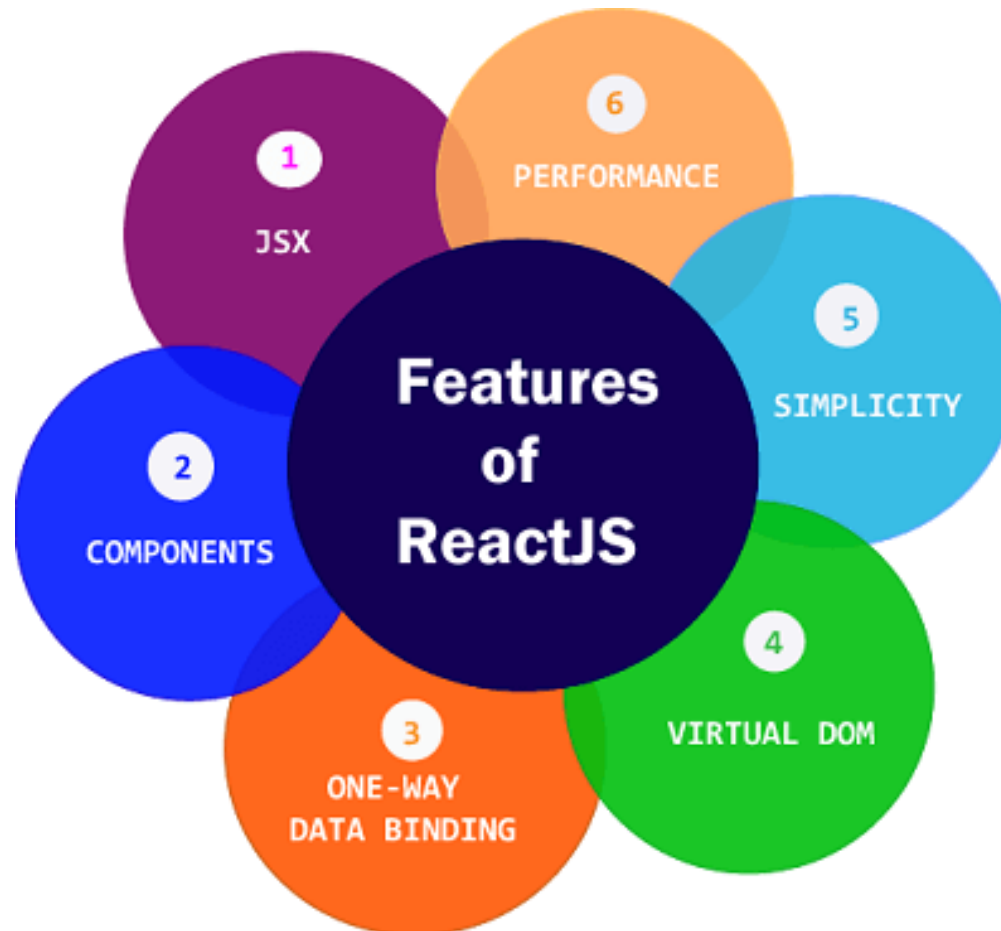
Multiple HTML Pages, Content is rendered on Server



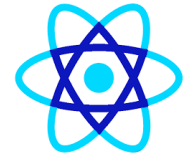
One ReactDOM.render() call per "widget"

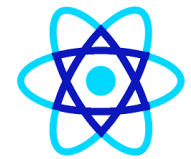


Features of React

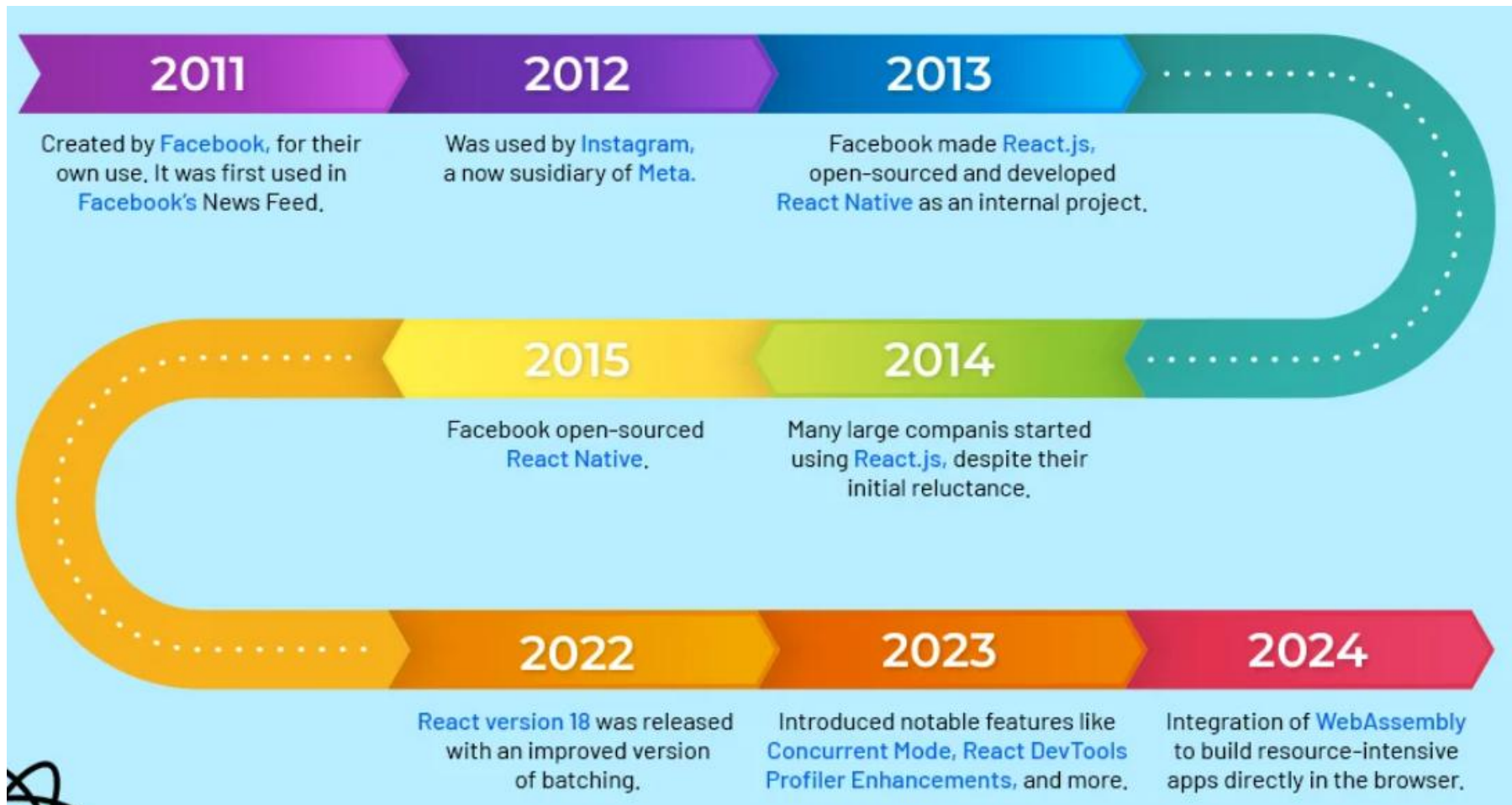


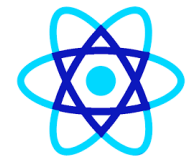
Features Of React



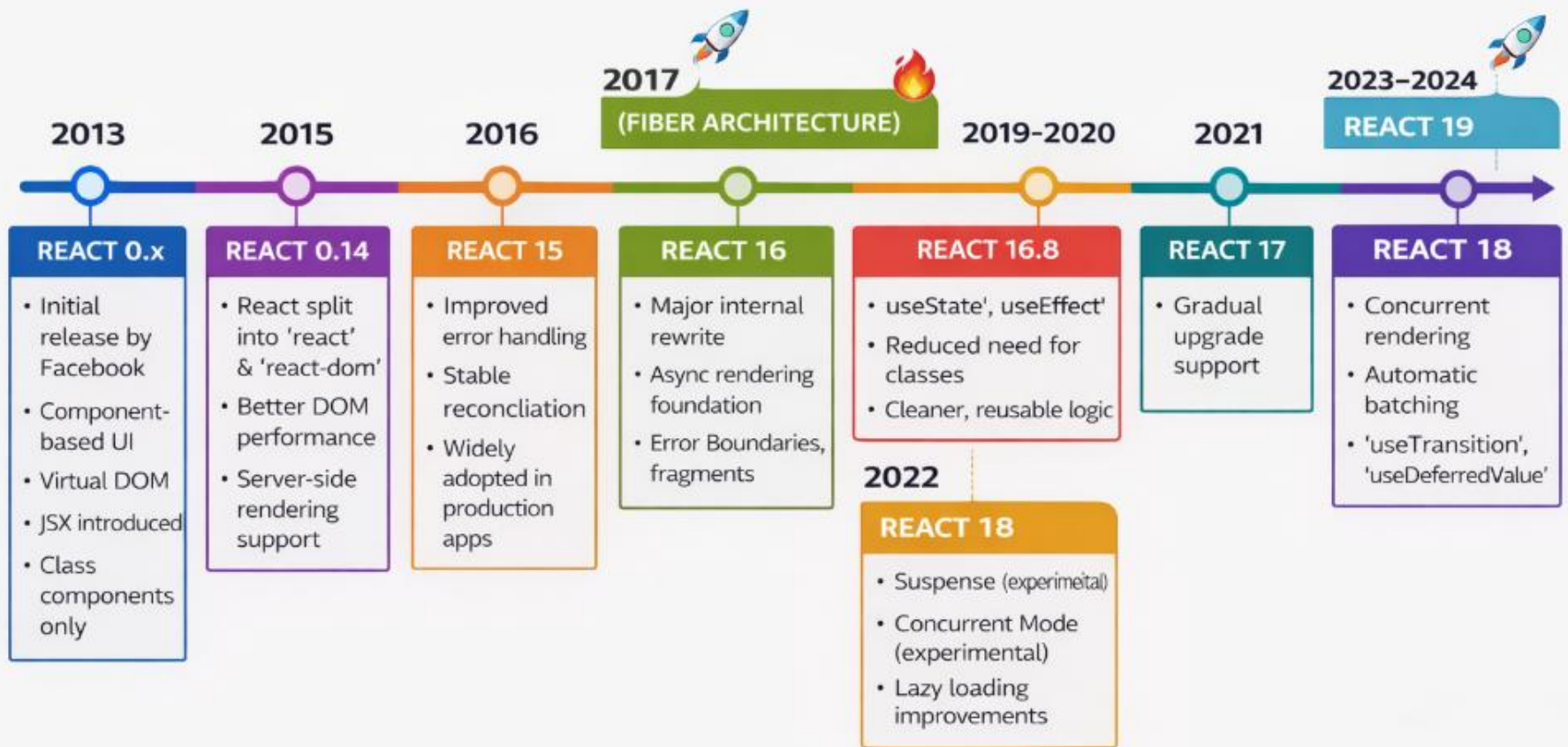


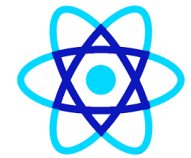
History and Evolution of Reactjs





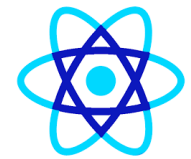
History and Evolution of Reactjs



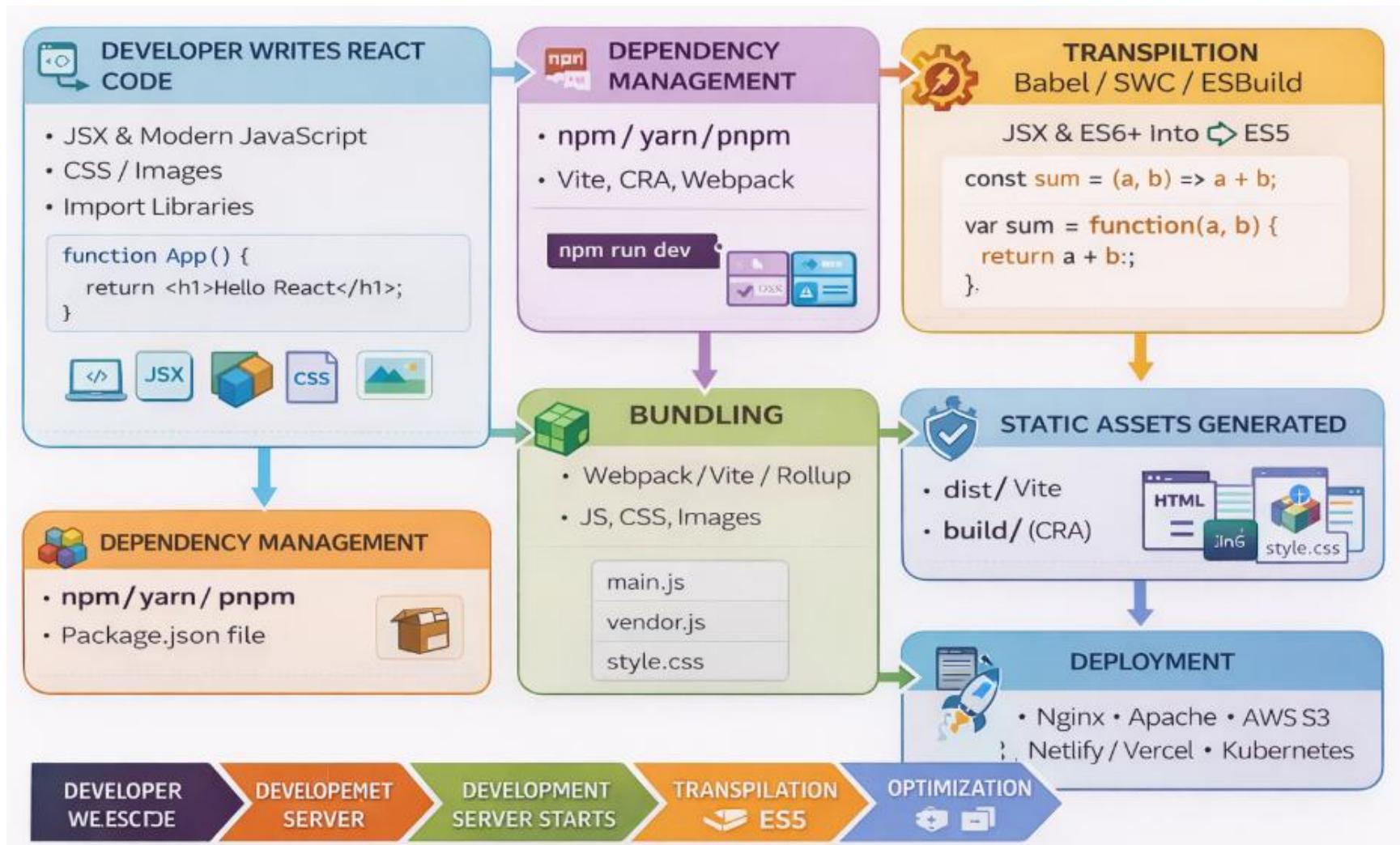


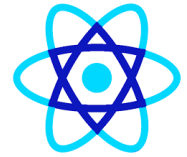
Quick Overview

Era	Version	Key Highlight
Early	0.x – 15	Virtual DOM, JSX
Major Shift	16	Fiber architecture
DX Revolution	16.8	Hooks
Stability	17	Gradual upgrades
Modern React	18	Concurrent rendering
Future	19	Server-first React

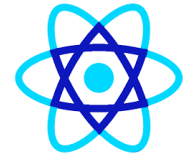


Using a Build Workflow

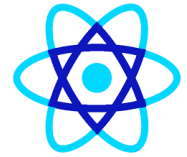




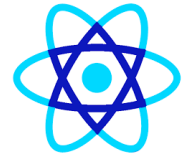
- **❶ Prerequisites (Must Have)**
- **✓ Operating System**
- Windows / macOS / Linux
- **✓ Node.js (Mandatory)**
- React runs on **Node.js**
- Includes **npm** (Node Package Manager)
- **◆ Recommended:**
- **Node.js LTS (18.x or 20.x)**
- Check installation:
- `node -v`
- `npm -v`



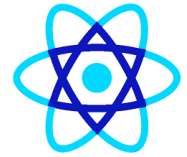
- **2 Code Editor**
- **✓ Recommended Editors**
- **VS Code** (most popular)
- WebStorm
- IntelliJ IDEA
- **🔌 Useful VS Code Extensions**
- ES7+ React Snippets
- Prettier
- ESLint
- Auto Rename Tag



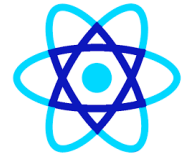
- **3 Browser & Dev Tools**
- **✓ Browser**
- Google Chrome (recommended)
- **🔧 React Developer Tools**
- Install browser extension:
- React Developer Tools (Chrome / Firefox)
- Used for:
- Inspecting components
- Checking props & state



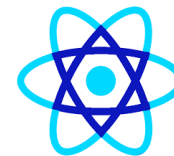
- **4 Create React Application (3 Ways)**
- **Option 1: Vite (Recommended – Modern)**
- Fastest & industry preferred
- `npm create vite@latest my-react-app`
- `cd my-react-app`
- `npm install`
- `npm run dev`
- Runs on:
- `http://localhost:5173`



- **4 Create React Application (3 Ways)**
- **Option 2: Create React App (CRA – Older)**
 - `npx create-react-app my-app`
 - `cd my-app`
 - `npm start`
 - Runs on:
 - `http://localhost:3000`
 - **⚠ Slower than Vite, but still common in legacy projects**



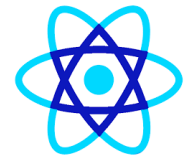
- **4 Create React Application (3 Ways)**
- **Option 3: Next.js (Framework-based)**
 - Used for SSR, SEO, production apps
 - `npx create-next-app my-app`
 - `cd my-app`
 - `npm run dev`



5 Project Structure (Vite Example)

pgsql

```
my-react-app/  
├─ node_modules/  
├─ public/  
├─ src/  
│   ├─ App.jsx  
│   ├─ main.jsx  
│   └─ index.css  
├─ package.json  
└─ vite.config.js
```



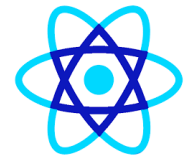
6 Verify React Is Working

Edit `App.jsx`:

jsx

```
function App() {  
  return <h1>React Setup Successful 🚀</h1>;  
}  
  
export default App;
```

Save → browser auto refreshes ✓



7 Build for Production

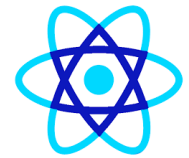
```
bash
```

```
npm run build
```

Output:

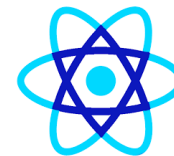
- `dist/` (Vite)
- `build/` (CRA)

👉 This folder is **deployment-ready**



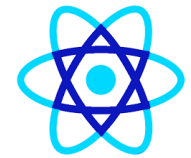
8 Optional (Professional Setup)

- ESLint + Prettier
- TypeScript
- Git & GitHub
- Docker (optional)
- CI/CD (GitHub Actions)



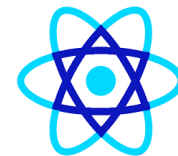
What SWC does

- SWC can:
- ☒ Compile **ES6+ → ES5**
- ☒ Transform **JSX / TSX**
- ☒ Compile **TypeScript → JavaScript**
- ☒ Enable **React Fast Refresh**
- ☒ Minify JavaScript



SWC(Speedy Web Compiler) vs Babel

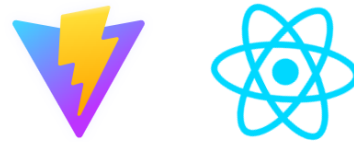
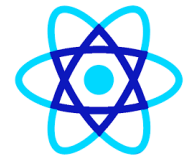
Feature	SWC	Babel
Language	Rust	JavaScript
Speed	⚡ Very Fast	🐢 Slower
JSX	✓	✓
TypeScript	✓	✓
Plugin ecosystem	Limited	Very rich
Configuration	Minimal	Complex



.npmrc settings

- npm config set strict-ssl=false
- npm config set proxy
http://username:password@proxyname:port
- npm config set https-proxy
http://username:password@proxyname:port
- npm config set registry <http://registry.npmjs.org/>
- Or
- Npm config edit
- proxy = http://username:password@proxyname:port
- https-proxy = http://username:password@proxyname:port
- Registry= <http://registry.npmjs.org/>

Loading Reactjs in Browser

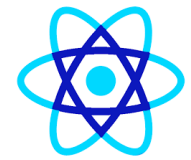


Vite + React

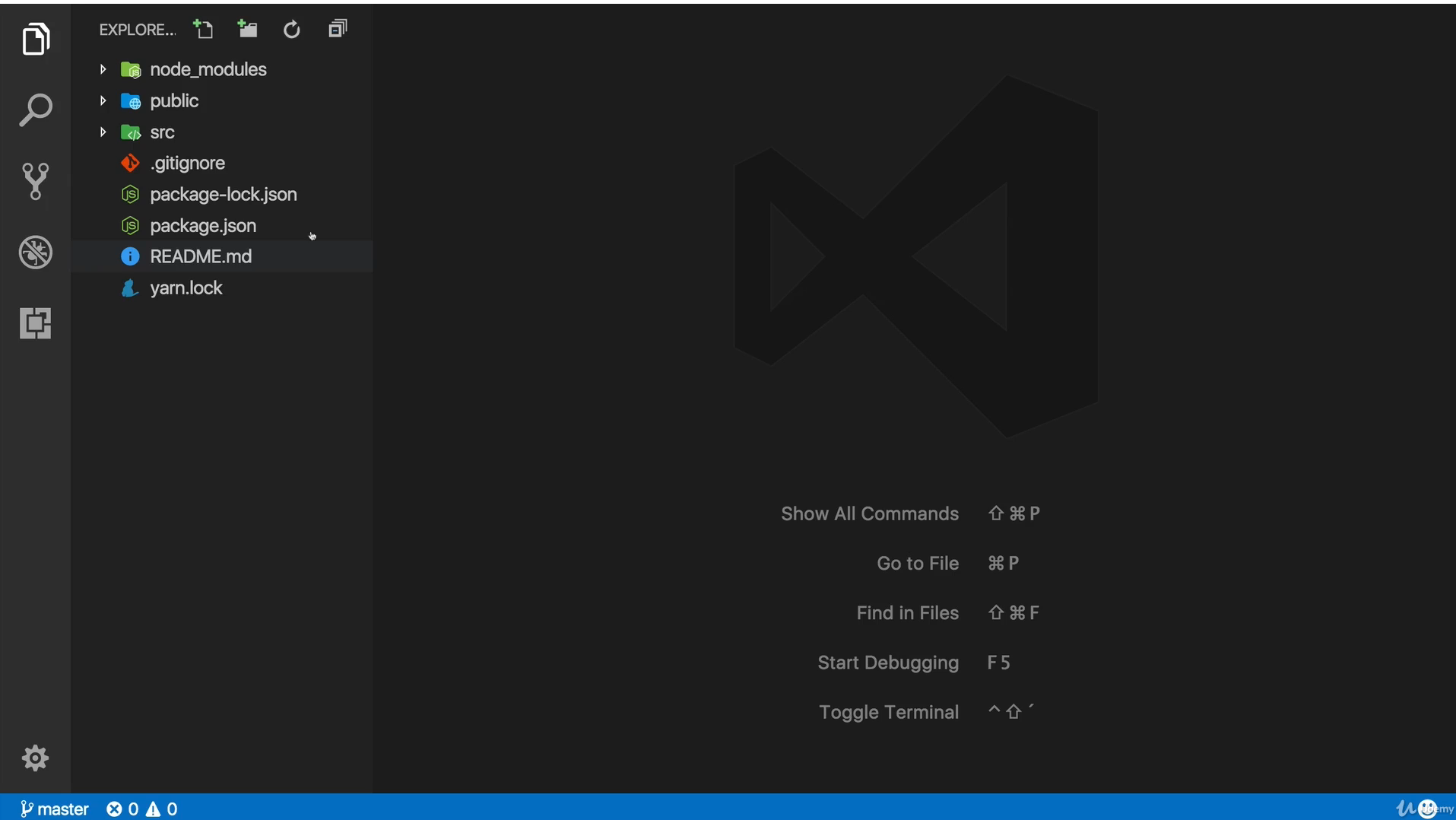
count is 0

Edit `src/App.jsx` and save to test HMR

Click on the Vite and React logos to learn more



Open Project in Editor

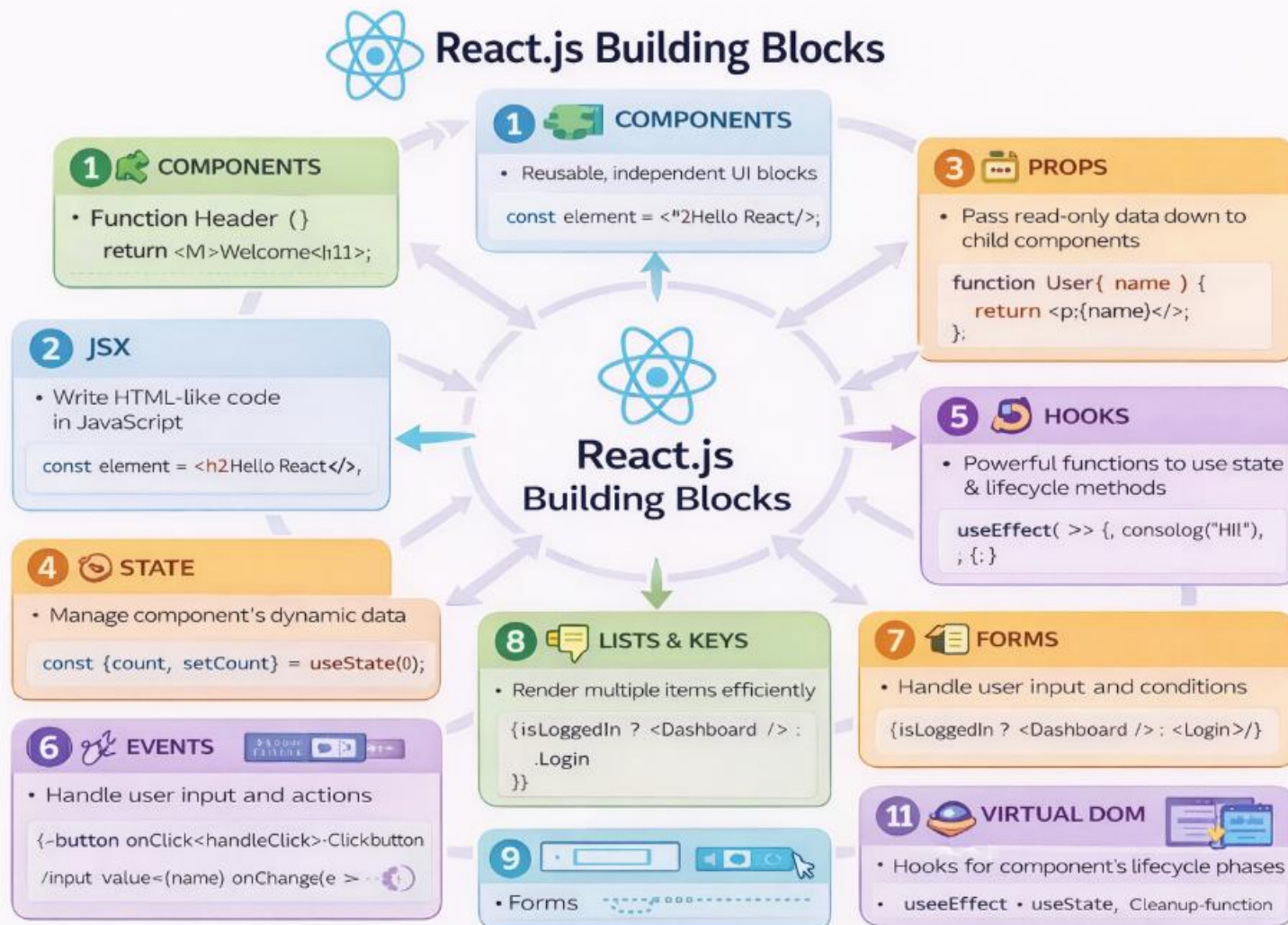
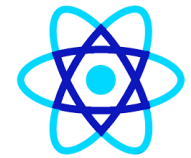




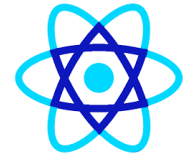
Building Blocks

- Components
- Props
- State
- State Lifecycle
- Event handling
- Keys

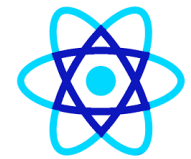
Building Blocks



Planning React App



- 1 Component Tree / Component Structure
- 2 Application State (Data)
- 3 Components vs Containers



Planning React App

```
{
  "name": "react-complete-guide",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "react": "^16.0.0",
    "react-dom": "^16.0.0",
    "react-scripts": "1.0.13"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test --env=jsdom",
    "eject": "react-scripts eject"
  }
}
```

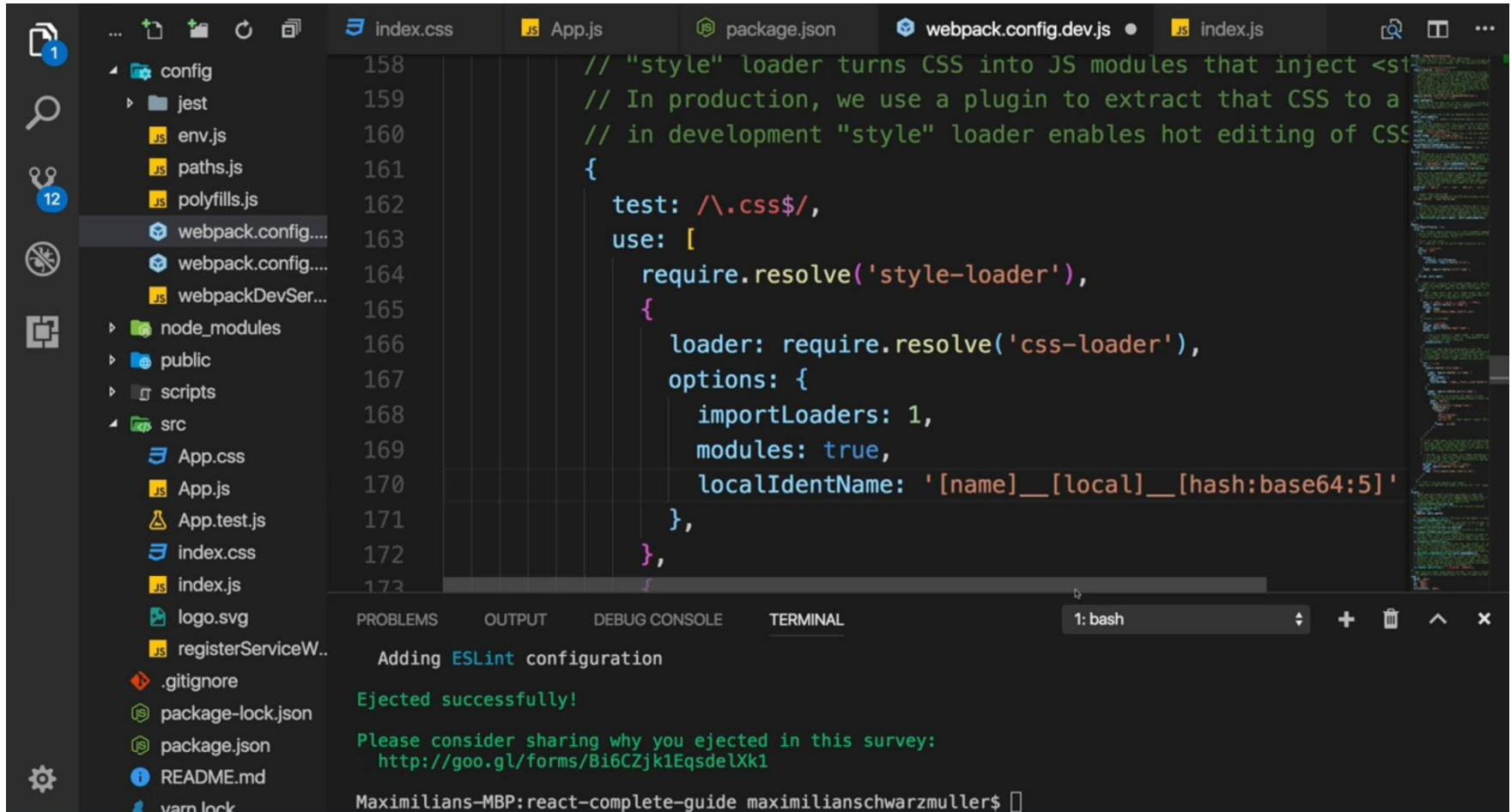
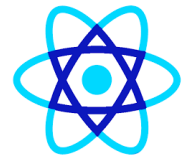
Maximilians-MBP:react-complete-guide maximilianschwarzmueller\$ npm run eject

> react-complete-guide@0.1.0 eject /Users/maximilianschwarzmueller/development/reactjs/tutorials/udemy/react-complete-guide

> react-scripts eject

? Are you sure you want to eject? This action is permanent. (y/N)

Planning React App



The screenshot shows a VS Code editor with the following files open in the tabs: index.css, App.js, package.json, webpack.config.dev.js, and index.js. The left sidebar shows a file explorer with a project structure including config, jest, node_modules, public, scripts, and src. The main editor displays the webpack.config.dev.js file, which is configured to use the 'style-loader' and 'css-loader' for CSS files. The terminal at the bottom shows the output of the 'eject' command, indicating a successful ejection and providing a survey link.

```
// "style" loader turns CSS into JS modules that inject <style> tags
// In production, we use a plugin to extract that CSS to a separate file
// in development "style" loader enables hot editing of CSS
{
  test: /\.css$/,
  use: [
    require.resolve('style-loader'),
    {
      loader: require.resolve('css-loader'),
      options: {
        importLoaders: 1,
        modules: true,
        localIdentName: '[name]__[local]__[hash:base64:5]'
      }
    },
  ],
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

1: bash

Adding ESLint configuration

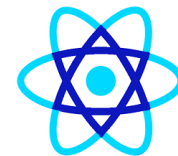
Ejected successfully!

Please consider sharing why you ejected in this survey:
<http://goo.gl/forms/Bi6CZjk1EqsdeIXk1>

Maximilians-MBP:react-complete-guide maximilianschwarzmueller\$



80



Atoms, Molecules and Organisms

- Examples of Atoms:
 - A button component (Button)
 - A label (Label)
 - An input field (TextField)
 - Icons (Icon)
 - Text elements like headings (Heading, Text)

jsx

```
import React from 'react';

const Button = ({ label, onClick }) => {
  return <button onClick={onClick}>{label}</button>;
};

export default Button;
```



Atoms, Molecules and Organisms

- Molecules are formed by combining two or more atoms.
- They are slightly more complex than atoms but are still simple and often represent small, reusable components that encapsulate specific functionality.
- Examples of Molecules:
 - A form input field with a label and error message (combining atoms like Label, Input, ErrorMessage)
 - A search bar (Input + Button)
 - A card with text and an image (Image + Text)



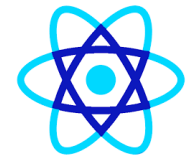
Atoms, Molecules and Organisms

jsx

```
import React from 'react';
import Button from './Button'; // An Atom
import TextField from './TextField'; // An Atom

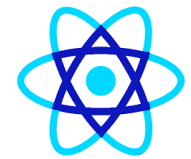
const SearchBar = ({ placeholder, buttonText, onSearch }) => {
  return (
    <div>
      <TextField placeholder={placeholder} />
      <Button label={buttonText} onClick={onSearch} />
    </div>
  );
};

export default SearchBar;
```

Atoms, Molecules and Organisms

- Organisms are more complex UI components that consist of molecules and/or atoms working together to form a relatively distinct section of the interface.
- They represent larger and more complex UI sections, like headers, forms, or cards with multiple functionalities.
- Examples of Organisms:
 - A navbar that includes multiple links, search bars, and user profile components (composed of molecules like SearchBar, NavLink, UserAvatar)
 - A user profile section that includes an image, name, and contact information
 - A complete form with various input fields, buttons, and labels



Atoms, Molecules and Organisms

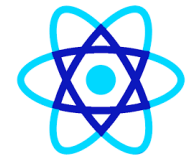
jsx

 Copy code

```
import React from 'react';
import SearchBar from './SearchBar'; // A Molecule
import UserProfile from './UserProfile'; // Another Molecule

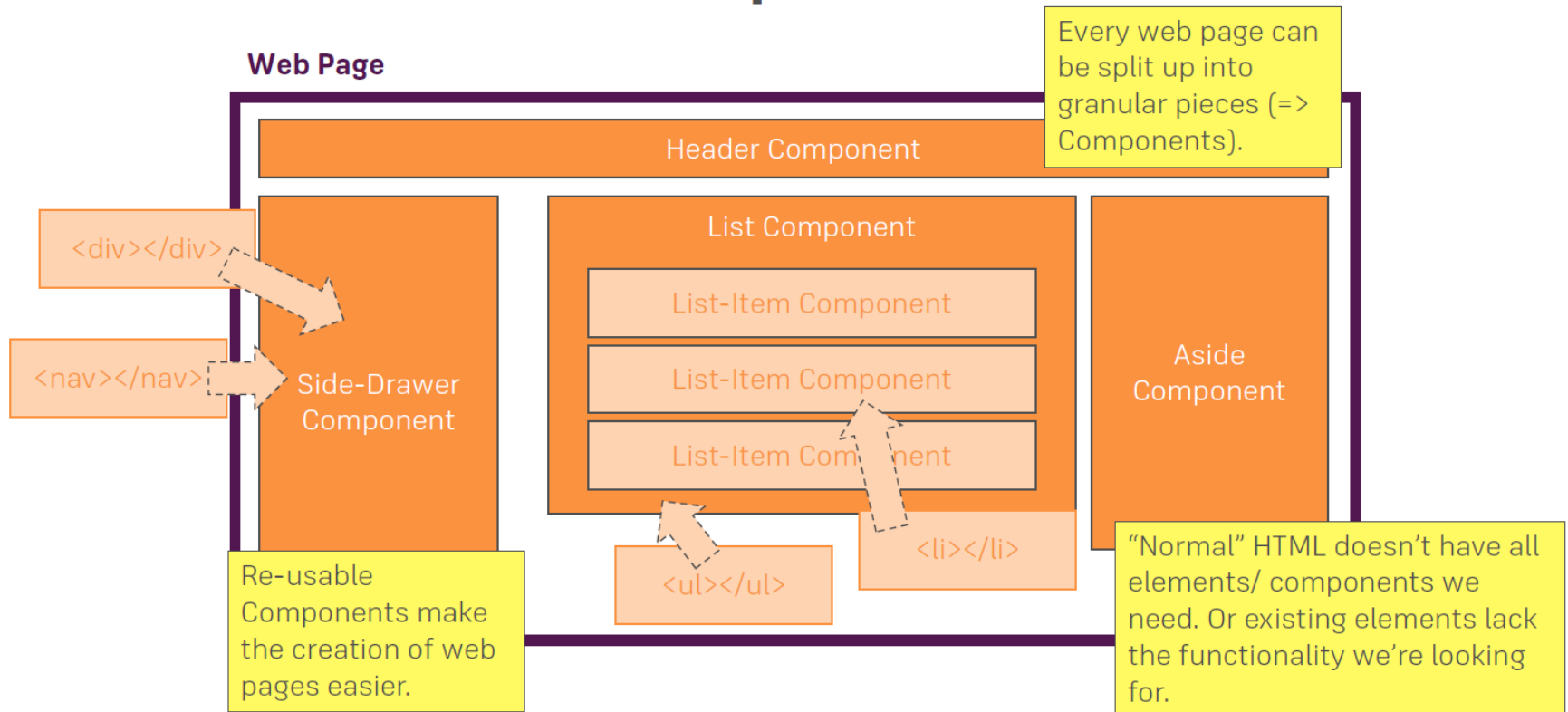
const Header = () => {
  return (
    <header>
      <h1>My App</h1>
      <SearchBar placeholder="Search..." buttonText="Go" onSearch={() => {}} />
      <UserProfile name="John Doe" avatarUrl="/avatar.png" />
    </header>
  );
};

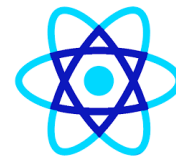
export default Header;
```



Application Architecture

Components

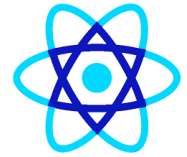




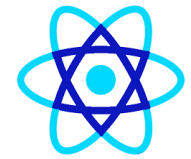
- Components are independent and reusable bits of code.
- They serve the same purpose as JavaScript functions.
- It works in isolation and returns HTML.
- In other words, we can say that every application we develop in React is made up of pieces called components.

Functional Components

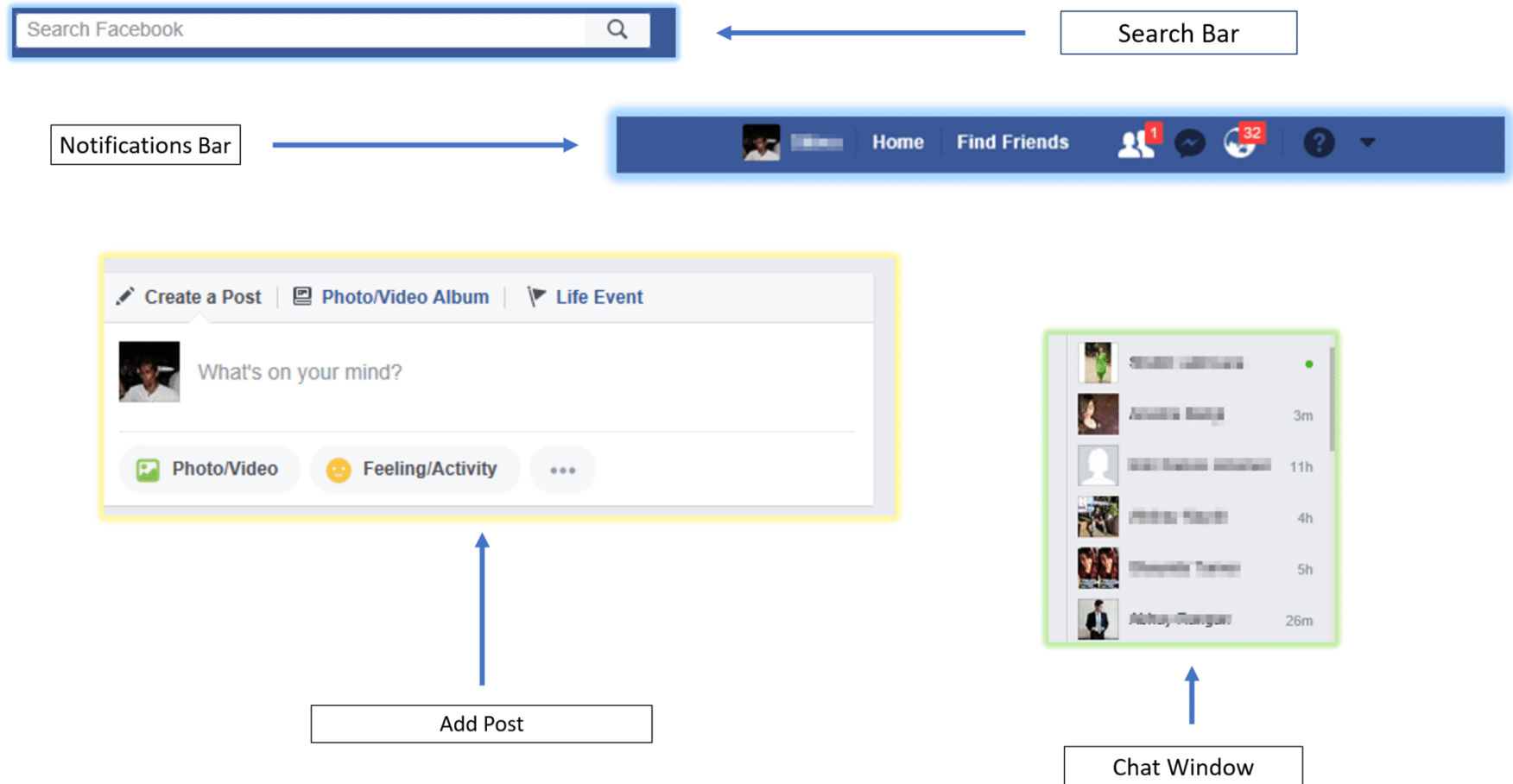
Class Components

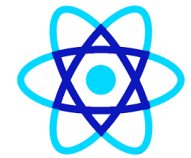


- The entire application can be modeled as a set of independent components.
- Different components are used to serve different purposes.
- This enables us to keep logic and views separate.
- React renders multiple components simultaneously.
- Components can be either stateful or stateless.



Facebook Components





Two types of components

Stateless vs Stateful Components

Stateful (Containers)

class XY extends Component



Access to State



Lifecycle Hooks

Access State and Props via **"this"**

`this.state.XY` & `this.props.XY`

Use only if you need to manage State or
access to Lifecycle Hooks!

Stateless

`const XY = (props) => { ... }`



Access to State



Lifecycle Hooks

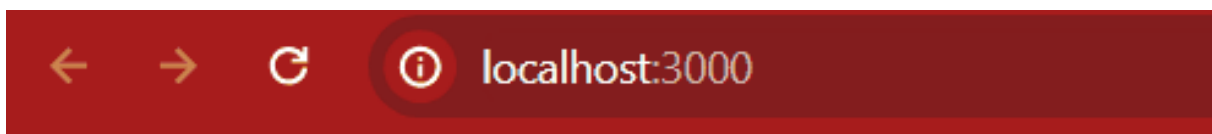
Access Props via **"props"**

`props.XY`

Use in all other Cases



State Management



You've clicked 0 times!

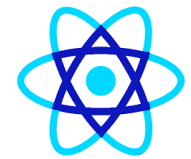
The number of times you have clicked is even!

Click me



Component

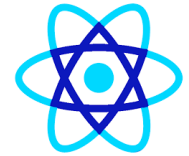
- Components can be generated using cli tool.
- Go to root of the project and execute the following command
- `npx generate-react-cli component Logo`



Component

```
export class BankApp extends Component{  
  no usages  
  constructor(props, context) {  
    super(props, context);  
    //console.log("Entering  constructor");  
    //step 1 create state  
    this.state={  
      currentTime:new Date(),  
    }  
  }  
  //step2  
  1+ usages  
  timerEvent=() : void =>{  
    this.setState( state: {  
      currentTime:new Date()  
    })  
  }  
}
```

Component Lifecycle



Only available in Stateful Components!

`constructor()`

`componentWillReceiveProps()`

`componentWillUpdate()`

`componentDidCatch()`

`componentWillUnmount()`

`componentWillMount()`

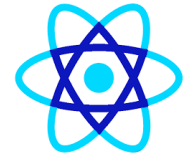
`shouldComponentUpdate()`

`componentDidUpdate()`

`componentDidMount()`

`render()`

Component Lifecycle



`constructor()`

`componentWillReceiveProps()`

`componentWillUpdate()`

`componentDidCatch()`

`componentWillUnmount()`

`componentWillMount()`

`shouldComponentUpdate()`

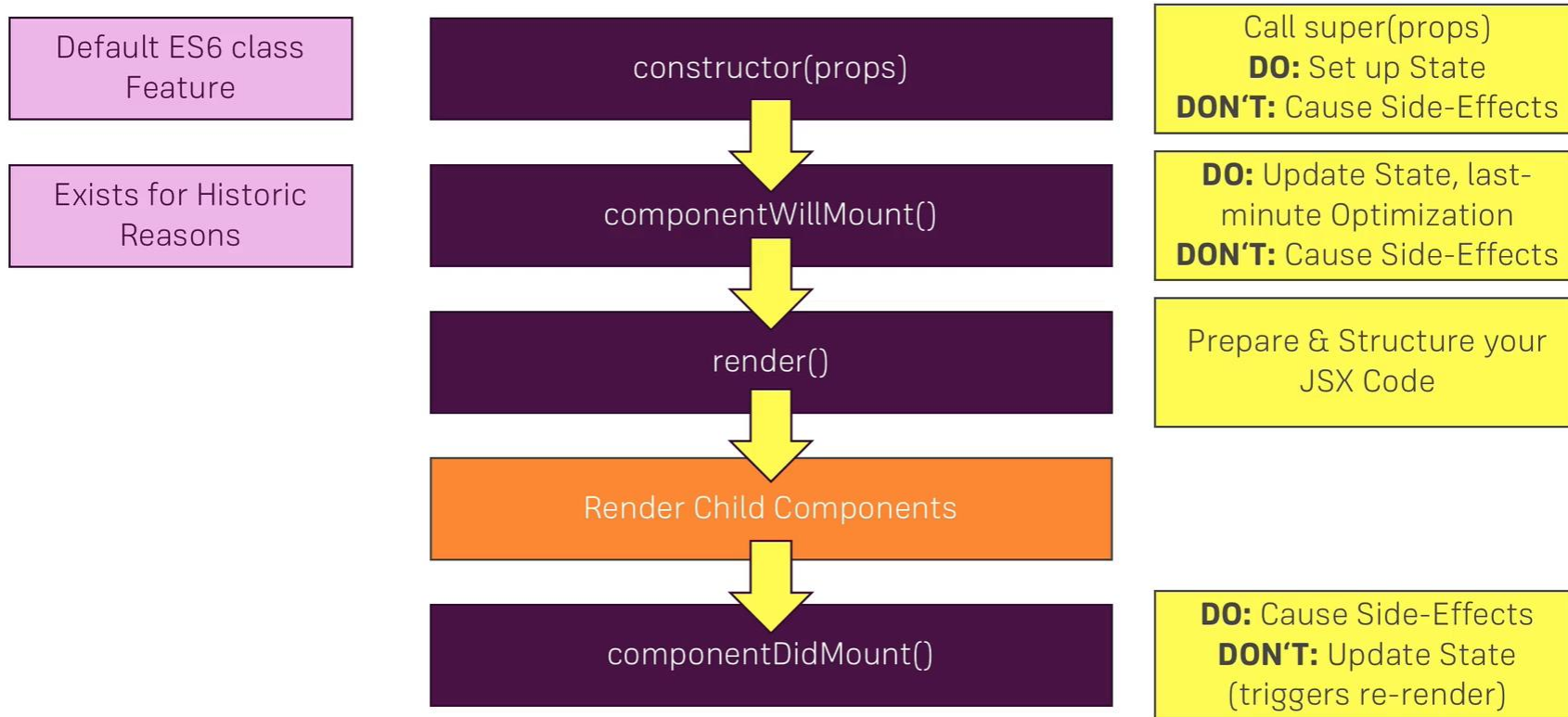
`componentDidUpdate()`

`componentDidMount()`

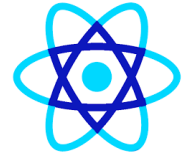
`render()`



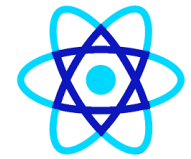
Component Lifecycle



React Component Lifecycle



-
- Initial Phase
 - Updating Phase
 - Props change Phase
 - Unmounting Phase

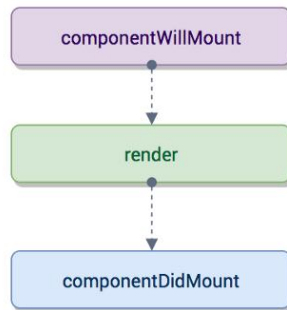


React Component Lifecycle

Initialization

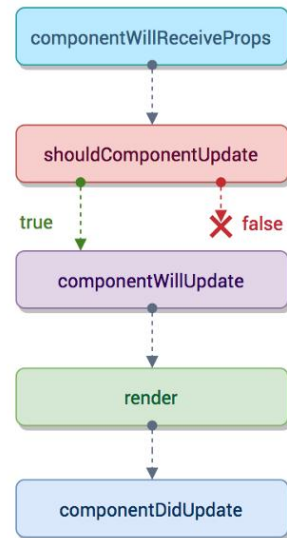
setup props and state

Mounting

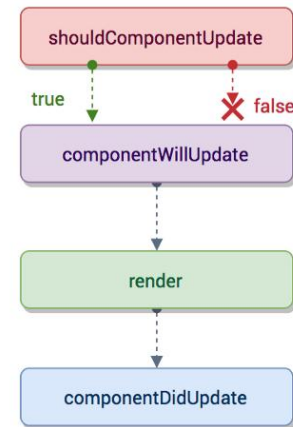


Updation

props

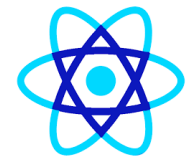


states

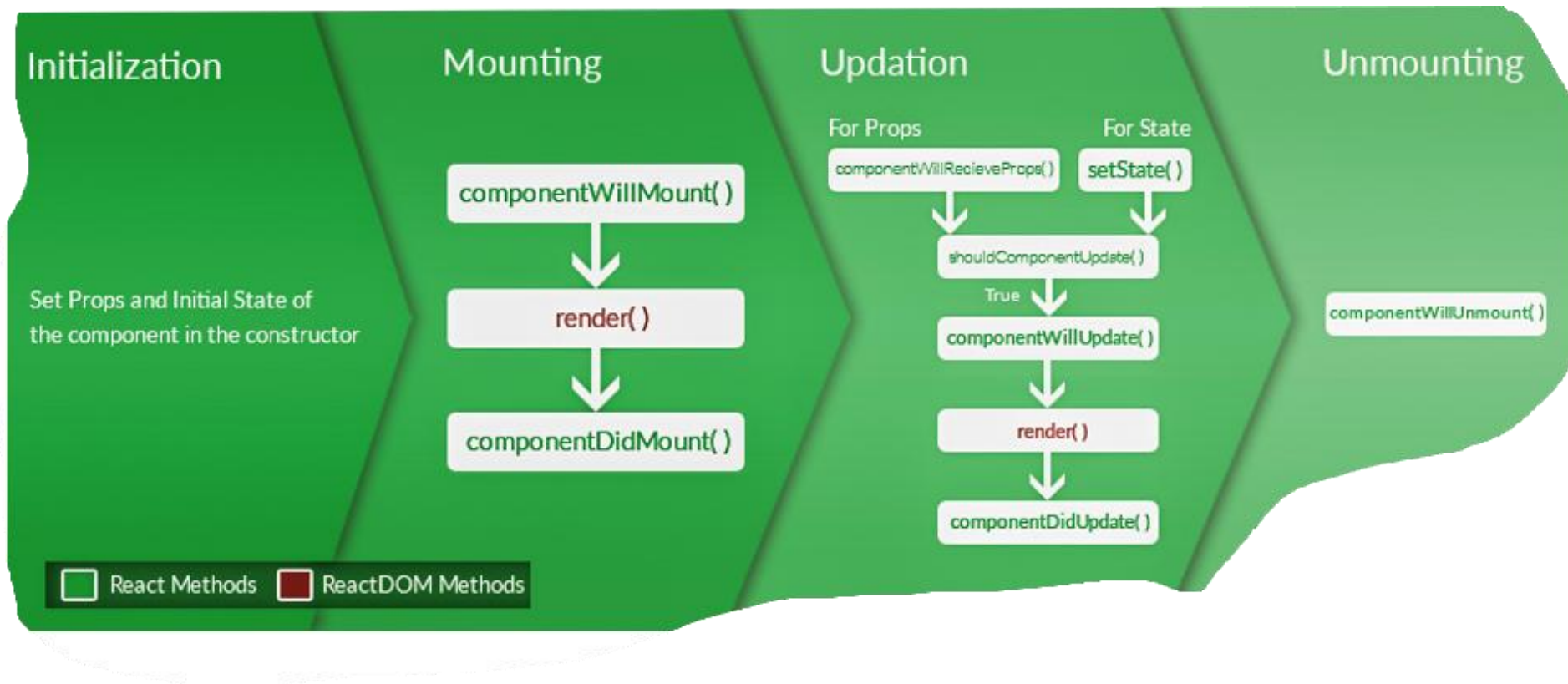


Unmounting

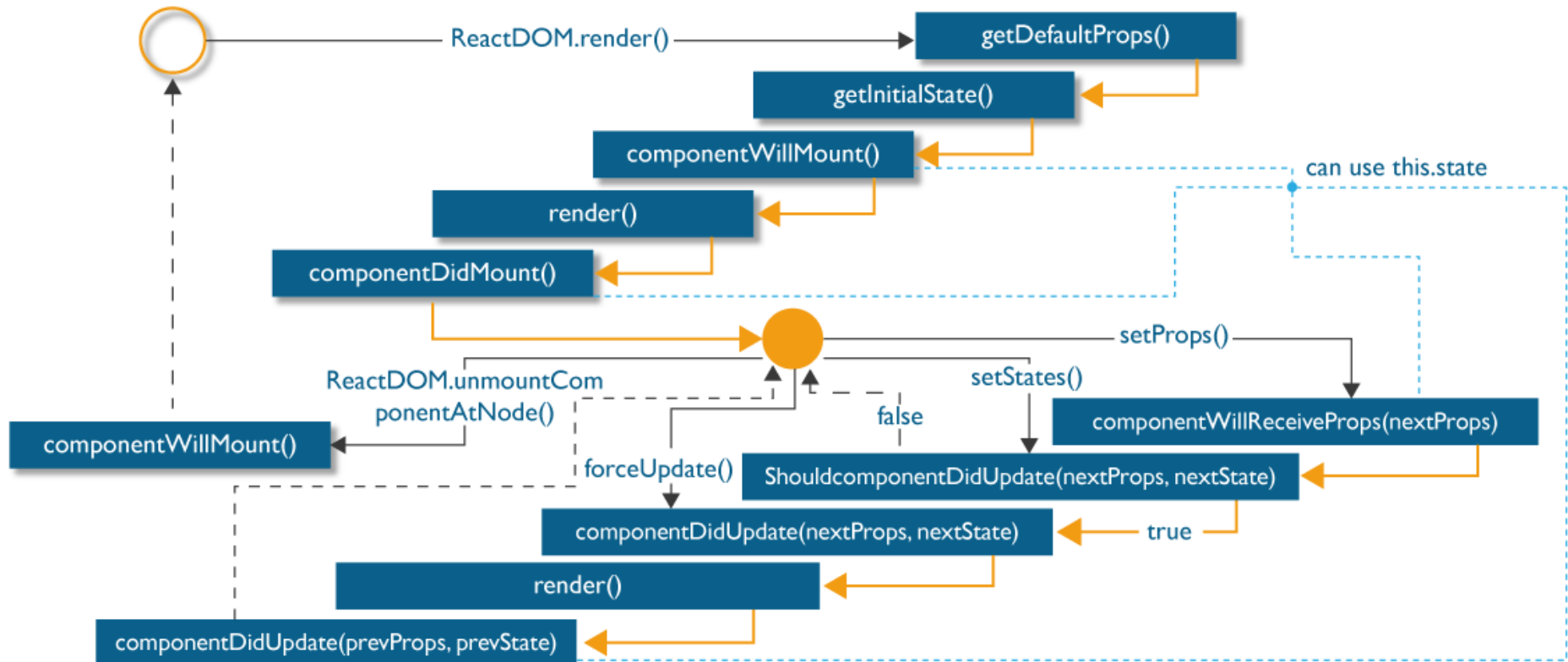
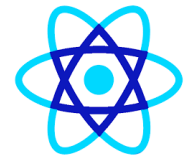


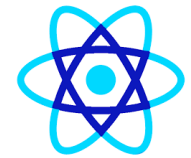


React Component Lifecycle



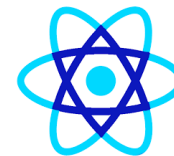
React Component Lifecycle





Initial Phase

- ***getDefaultProps():***
 - This method is used to specify the default value of **this.props**.
 - It gets called before your component is even created or any props from the parent are passed into it.
- ***getInitialState():***
 - This method is used to specify the default value of **this.state** before your component is created.
- ***componentWillMount():***
 - This is the last method that you can call before your component gets rendered into the DOM.
 - But if you call **setState()** inside this method your component will not re-render.
- ***render():***
 - This method is responsible for returning a single root HTML node and must be defined in each and every component.
 - You can return **null** or **false** in case you don't want to render anything.
- ***componentDidMount():***
 - Once the component is rendered and placed on the DOM, this method is called. Here you can perform any DOM querying operations.



Updating Phase

- ***shouldComponentUpdate():***
 - Using this method you can control your component's behavior of updating itself.
 - If you return a true from this method, the component will update. Else if this method returns a false, the component will skip the updating.
- ***componentWillUpdate():***
 - This method is called just before your component is about to update. In this method, you can't change your component state by calling **this.setState**.
- ***render():***
 - If you are returning false via **shouldComponentUpdate()**, the code inside **render()** will be invoked again to ensure that your component displays itself properly.
- ***componentDidUpdate():***
 - Once the component is updated and rendered, then this method is invoked. You can put any code inside this method, which you want to execute once the component is updated.



Props Change Phase

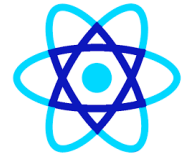
- ***componentWillReceiveProps():***
 - This method returns one argument which contains the new prop value that is about to be assigned to the component.
 - *Rest of the lifecycle methods behave identically to the methods which we saw in the previous phase.*
- ***shouldComponentUpdate()***
- ***componentWillUpdate()***
- ***render()***
- ***componentDidUpdate()***



The Unmounting Phase

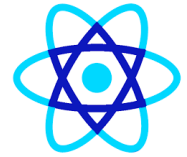
- ***componentWillUnmount():***
 - Once this method is invoked, your component is removed from the DOM permanently.
 - In this method, you can perform any clean-up related tasks like removing event listeners, stopping timers, etc.

2 Functional Components (Modern & Recommended)



-  **Definition**
 - A **functional component** is a **JavaScript function** that:
 - Returns JSX
 - Uses **Hooks** for state and lifecycle
 - No `this` keyword
 - Simpler and cleaner

2 Functional Components (Modern & Recommended)



Syntax with Hooks

jsx

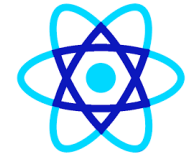
```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

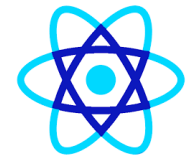
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}

export default Counter;
```

2 Functional Components (Modern & Recommended)



Lifecycle	Hook
Mount	<code>useEffect(() => {}, [])</code>
Update	<code>useEffect(() => {}, [deps])</code>
Unmount	<code>useEffect(() => { return () => {} }, [])</code>



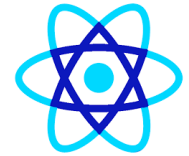
Key Differences(Class vs Functional)

Feature	Class Component	Functional Component
Syntax	Verbose	Simple
State	this.state	useState()
Lifecycle	Methods	Hooks
this keyword	Yes	No
Performance	Slightly slower	Slightly faster
Code reuse	HOCs, Render props	Custom Hooks
Future support	Legacy	Recommended

⚡ Why Functional Components Are Preferred Today

-  Less boilerplate
-  Easier to test
-  Better performance optimizations
-  Cleaner lifecycle handling
-  Works better with modern tools (Vite, SWC)

When to Use What?



Scenario	Choice
New project	✓ Functional
Legacy codebase	Class
Simple UI	Functional
Heavy lifecycle logic	Functional + Hooks



Example Same Component Both Types

Class

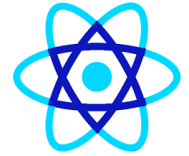
jsx

```
class Hello extends React.Component {  
  render() {  
    return <h1>Hello {this.props.name}</h1>;  
  }  
}
```

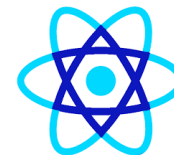
Functional

jsx

```
const Hello = ({ name }) => <h1>Hello {name}</h1>;
```



- ♦ **What is an Expression?**
- An **expression** is anything that produces a value. In JSX, expressions are written inside `{ }`.
-  **Allowed**
- Variables
- Function calls
- Ternary operators
- Mathematical operations



jsx

```
const name = "React";
```

```
<h1>Hello {name}</h1>
```

```
<h2>{10 + 5}</h2>
```

```
<p>{isLoggedIn ? "Welcome" : "Please Login"}</p>
```

✗ Not Allowed

- `if`, `for`, `while` statements
- `switch` statements

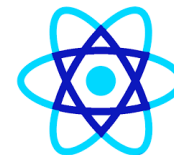
! Use ternary or logical operators instead.



2 JSX Attributes

- ♦ **What are JSX Attributes?**
- Attributes are used to **pass values to elements or components.**

HTML	JSX
class	className
for	htmlFor
onclick	onClick



2 JSX Attributes

Attribute Examples

jsx

```

<button disabled={true}>Submit</button>
<input type="text" maxLength={10} />
```

Dynamic Attributes

jsx

```
<button className={isActive ? "active" : "inactive"}>
  Save
</button>
```




3 JSX Styles

- ◆ **Inline Styles in JSX**
- Styles are written as a **JavaScript object**.
- ✓ camelCase property names
- ✓ Values as strings or numbers

jsx

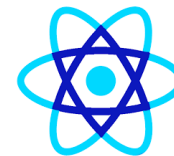
```
const style = {  
  color: "blue",  
  fontSize: "20px",  
  backgroundColor: "lightgray"  
};  
  
<h2 style={style}>Hello JSX</h2>
```



Inline Style (Direct)

jsx

```
<h1 style={{ color: "red", marginTop: 20 }}>  
  React Style  
</h1>
```



◆ Styling with className (Recommended)

jsx

```
<h2 className="title">Welcome</h2>
```

css

```
.title {  
  color: green;  
  font-size: 24px;  
}
```

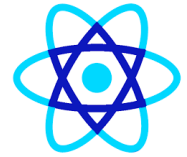


Common JSX Rules (Very Important)

-  **Must Follow**
-  Single parent element
-  Self-closing tags
- `<img />`
- `<input />`
-  JavaScript inside `{}`

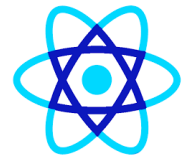


JSX Cheat Sheet (Interview Ready)

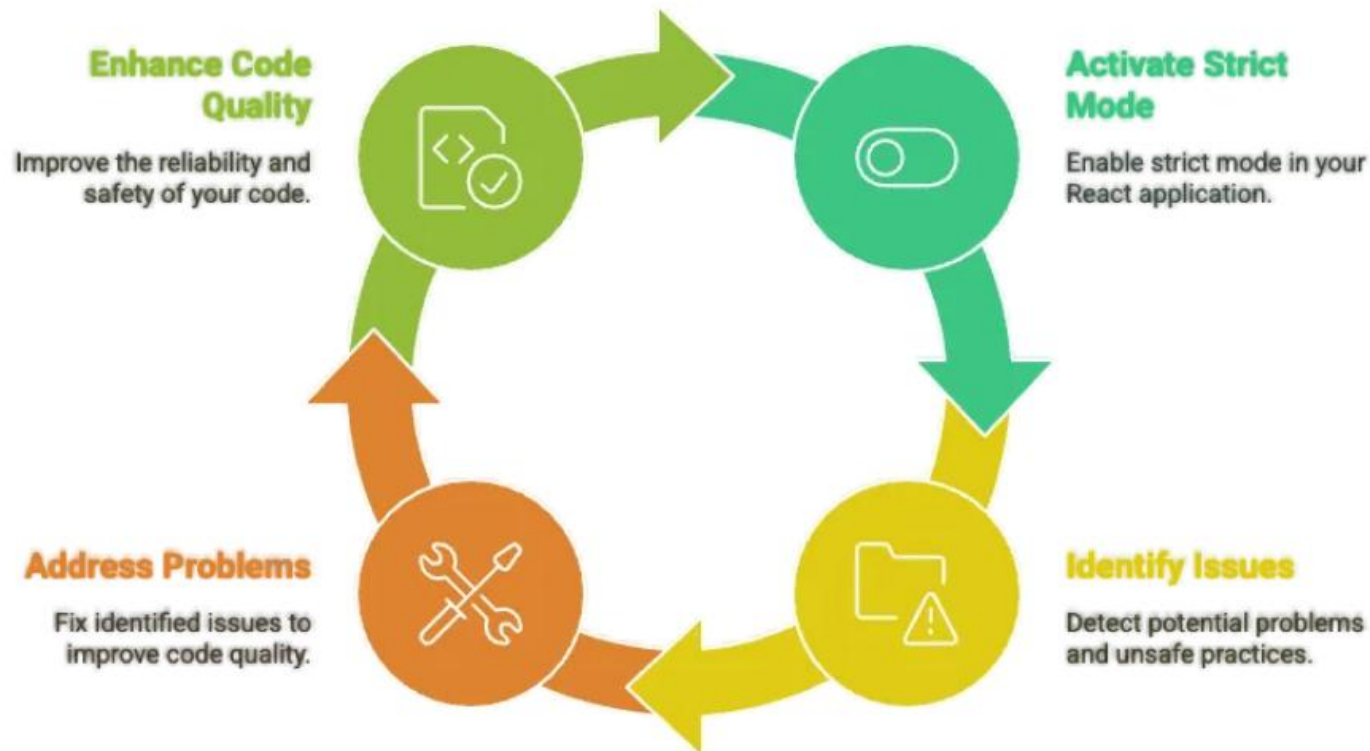


Topic	Key Point
Expressions	Use {}
Attributes	camelCase
Styles	JS object
Classes	className
Conditions	Ternary / &&

React.StrictMode



React Strict Mode Cycle





React.StrictMode

- **StrictMode** is a **development-only tool** that:
 - Finds unsafe side effects
 - Warns about deprecated APIs
 - Helps you write future-proof React code
- 👉 **It does NOT affect production builds**



React.StrictMode

- **[2] Where StrictMode is in a Vite App**
- **main.jsx (default Vite React template)**
- `import { StrictMode } from 'react'`
- `import { createRoot } from 'react-dom/client'`
- `import './index.css'`
- `import App from './App.jsx'`
- `createRoot(document.getElementById('root')).render(
 <StrictMode>
 <App />
 </StrictMode>,
)`
-



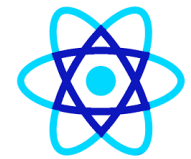
React.StrictMode

- **③ What StrictMode Actually Does**
- **↺ Double Invokes (DEV only)**
- StrictMode intentionally runs certain things **twice** to detect side effects:
- Component render
- useEffect (setup + cleanup + setup)
- useState initializer
- useReducer initializer
- **!** This is **expected behavior**, not a bug.



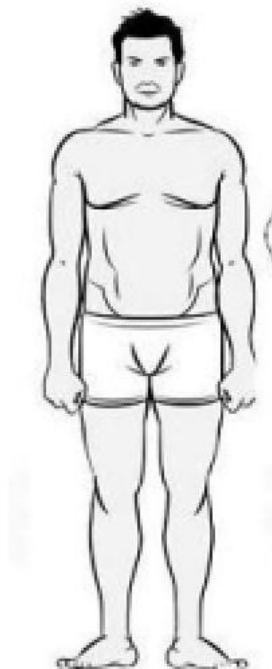
React.StrictMode

Feature	Vite	CRA
StrictMode default	✓ Yes	✓ Yes
Fast refresh	Faster	Slower
Dev behavior	Same	Same
Production impact	None	None



Require vs Import

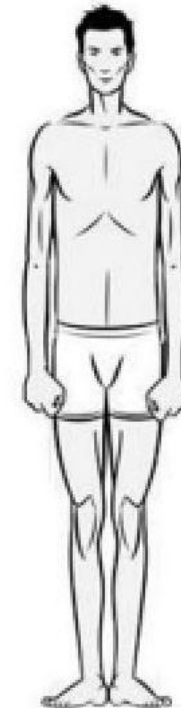
require



```
//----- lib.js -----  
function add(x, y) {  
  return x + y  
}  
module.exports = {  
  add: add,  
};  
  
//----- main.js -----  
var add = require('lib').add;  
console.log(add(1,2));  
|
```



import



```
//----- lib.js -----  
export function add(x, y) {  
  return x + y  
}  
  
//----- main.js -----  
import { add } from 'lib';  
console.log(add(1,2));
```



Different types of export

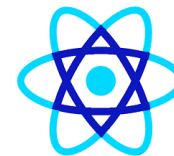
- 1— Named exports (several per module)
- 2— Default exports (one per module)
- 3— Mixed named & default exports
- 4— Cyclical Dependencies



Named Export

```
//----- lib.js -----  
export const sqrt = Math.sqrt;  
export function square(x) {  
  return x * x;  
}  
export function diag(x, y) {  
  return sqrt(square(x) + square(y));  
}
```

```
//----- main.js -----  
import { square, diag } from 'lib';  
console.log(square(11)); // 121  
console.log(diag(4, 3)); // 5
```



Named Export

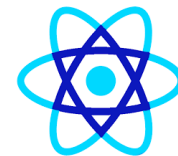
or

//----- main.js -----

import * as lib from 'lib';

console.log(lib.square(11)); // 121

console.log(lib.diag(4, 3)); // 5



Default exports (one per module)

- `//----- myFunc.js -----`
- `export default function () { ... };`

- `//----- main1.js -----`
- `import myFunc from 'myFunc';`
- `myFunc();`



Mixed Default and Named Exports

- `//----- underscore.js -----`
- `export default function (obj) {`
- `...`
- `};`
- `export function each(obj, iterator, context) {`
- `...`
- `}`
- `export { each as forEach };`

- `//----- main.js -----`
- `import _, { each } from 'underscore';`
- `...`



Cyclical Dependency

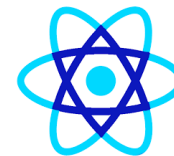
- `// lib.js`
- `import Main from 'main';`
- `var lib = {message: "This Is A Lib"};`
- `export { lib as Lib };`

- `// main.js`
- `import { Lib } from 'lib';`
- `export default class Main {`
- `// ....`
- `}`



React Form Design

- **1. What Formik actually does**
- Formik helps you manage:
 - **form state** – values, errors, touched fields, isSubmitting, etc.
 - **validation** – required fields, patterns, custom rules (often with Yup)
 - **submit** – central onSubmit handler
 - **UX helpers** – dirty, isValid, resetForm(), etc.



React Form Design

- **2. Setup**
 - npm install formik yup
 - # or
 - yarn add formik yup
- We'll use:
 - **Formik** – form state + helpers
 - **Yup** – schema-based validation



React Form Design

- **3. Basic Formik usage (hook style)**

First, the simplest possible form using useFormik:

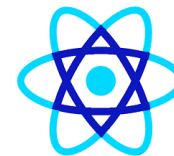
```
import React from "react";
import { useFormik } from "formik";
function SimpleLoginForm() {
  const formik = useFormik({
    initialValues: {
      email: "",
      password: "",
    },
    onSubmit: (values) => {
      console.log("Form submitted:", values);
    },
  });
}
```



React Form Design

- **3. Basic Formik usage (hook style)**

```
return (  
  <form onSubmit={formik.handleSubmit}>  
    <label>  
      Email  
      <input  
        type="email"  
        name="email"  
        value={formik.values.email}  
        onChange={formik.handleChange}  
        onBlur={formik.handleBlur}  
      />  
    </label>  
  
    <button type="submit">Login</button>  
  </form>
```



React Form Design

- Core ideas:
 - `initialValues` defines form fields.
 - `formik.values` holds current values.
 - `formik.handleChange` and `formik.handleBlur` are wired to each input.
 - `formik.handleSubmit` is attached to `<form onSubmit>`.



React Form Design

- **4. Adding validation with Yup**
 - Now let's design a **User Registration form** with:
 - name
 - email
 - password
 - confirmPassword
 - role (select)
 - acceptTerms (checkbox)

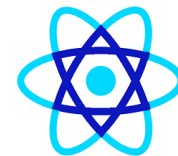


React Form Design

- **4. Adding validation with Yup**

- 4.1 Define validation schema**

```
import * as Yup from "yup";  
const registrationSchema = Yup.object({  
  name: Yup.string()  
    .trim()  
    .required("Name is required"),  
  email: Yup.string()  
    .email("Invalid email format")  
    .required("Email is required"),  
  password: Yup.string()  
    .min(8, "Password must be at least 8 characters")  
    .required("Password is required"),
```

React Form Design

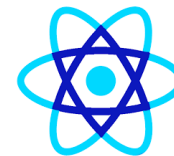
- **4. Adding validation with Yup**

```
confirmPassword: Yup.string()  
  .oneOf([Yup.ref("password")], "Passwords must match")  
  .required("Please confirm your password"),  
role: Yup.string()  
  .oneOf(["user", "admin"], "Invalid role")  
  .required("Role is required"),  
acceptTerms: Yup.boolean()  
  .oneOf([true], "You must accept the terms & conditions"),  
});
```



React Form Design

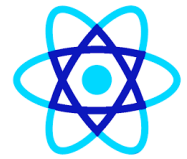
- **UX best practices with Formik**
- **Disable submit** when:
 - `isSubmitting` is true
 - or `!dirty` (nothing changed)
 - or `!isValid` (validation errors)
- **Show errors only after blur/change**
Formik tracks `touched`. Using `meta.touched && meta.error` prevents errors flashing as soon as the form renders.



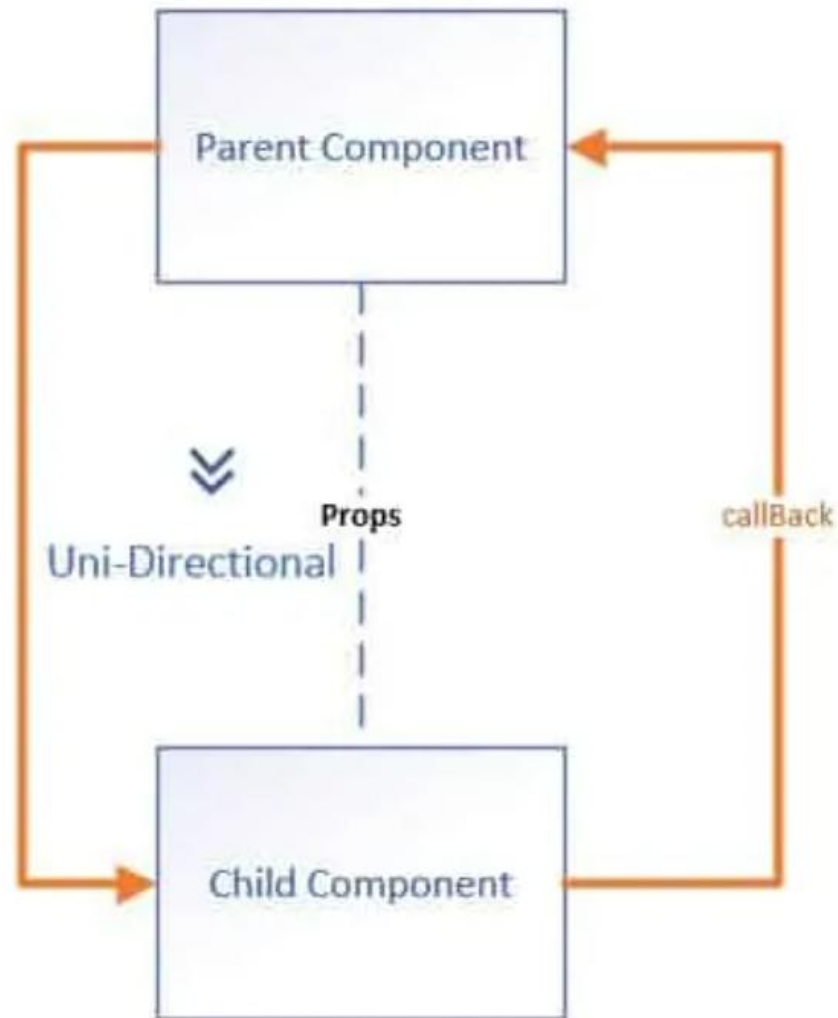
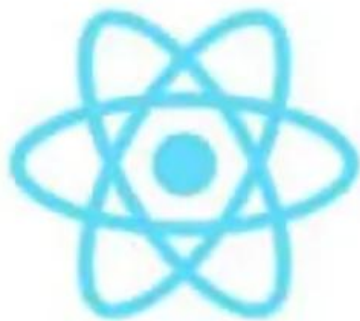
React Form Design

- **UX best practices with Formik**
- **Inline vs global errors**
 - Use `<ErrorMessage>` or `useField` to show inline errors.
 - You can also show a global error (e.g., API error) in `onSubmit` using `setStatus` or `setErrors`.
- **Reset after submit**
 - `resetForm()` inside `onSubmit` once API succeeds.
- **Loading indicator**
 - Use `isSubmitting` to show loading text/spinner on the button

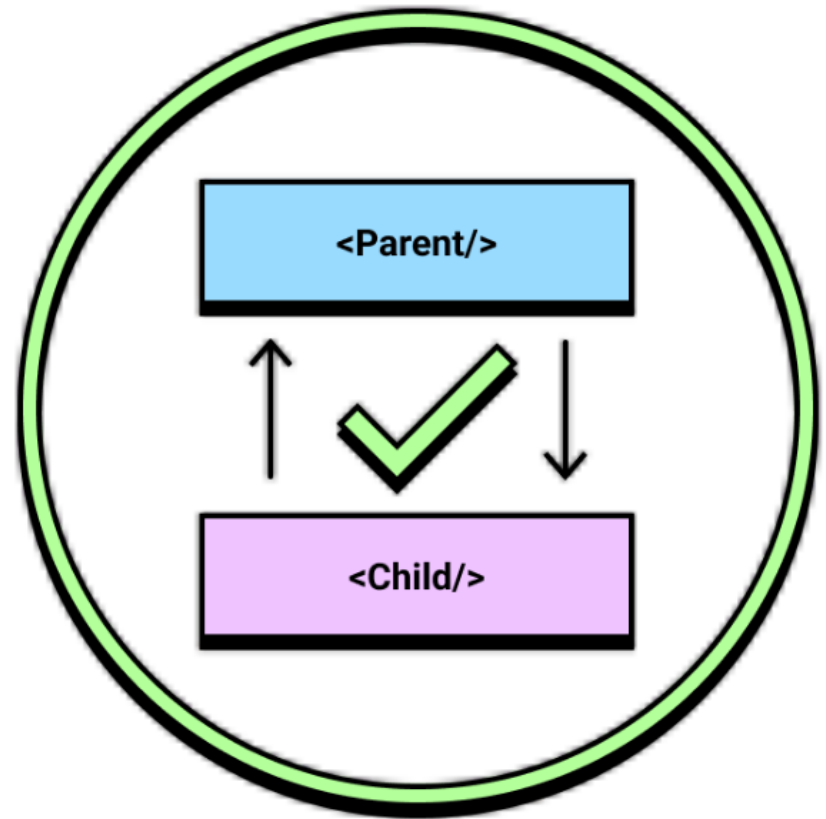
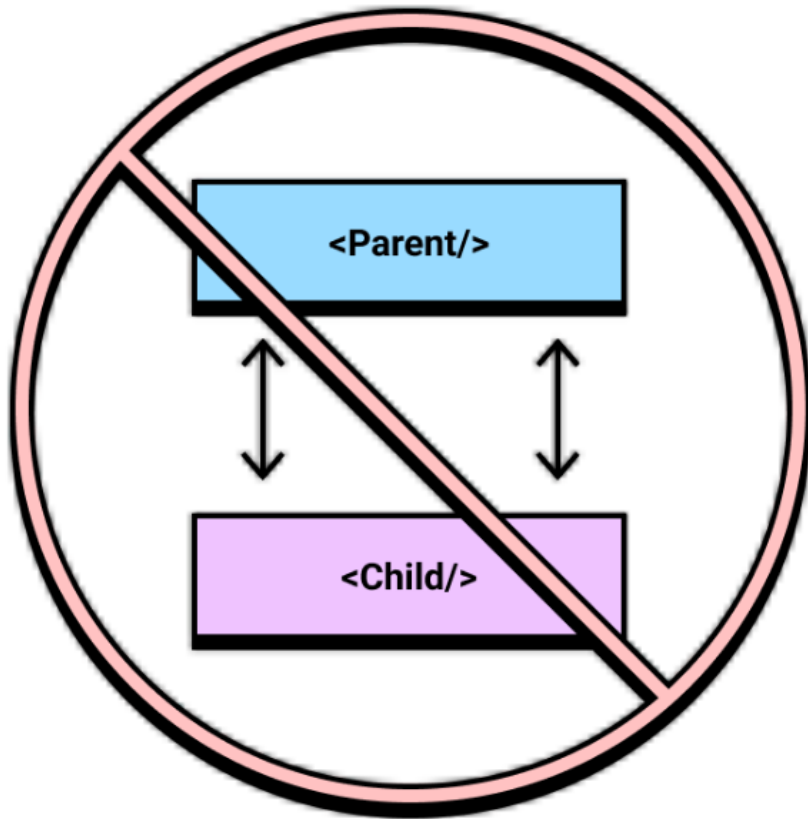
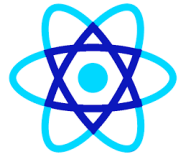
Props



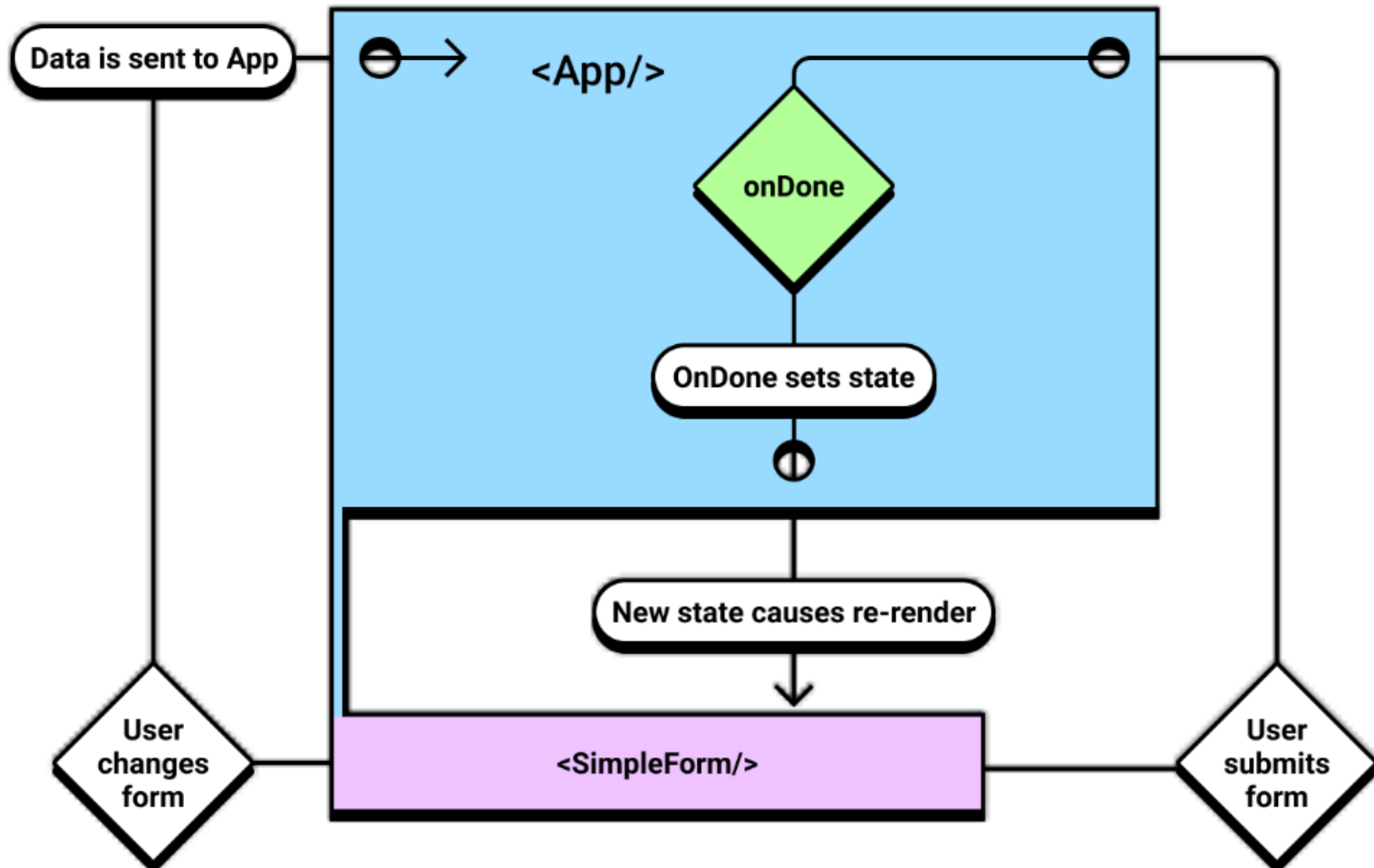
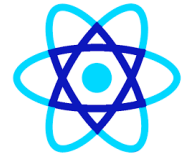
React



Props



Props

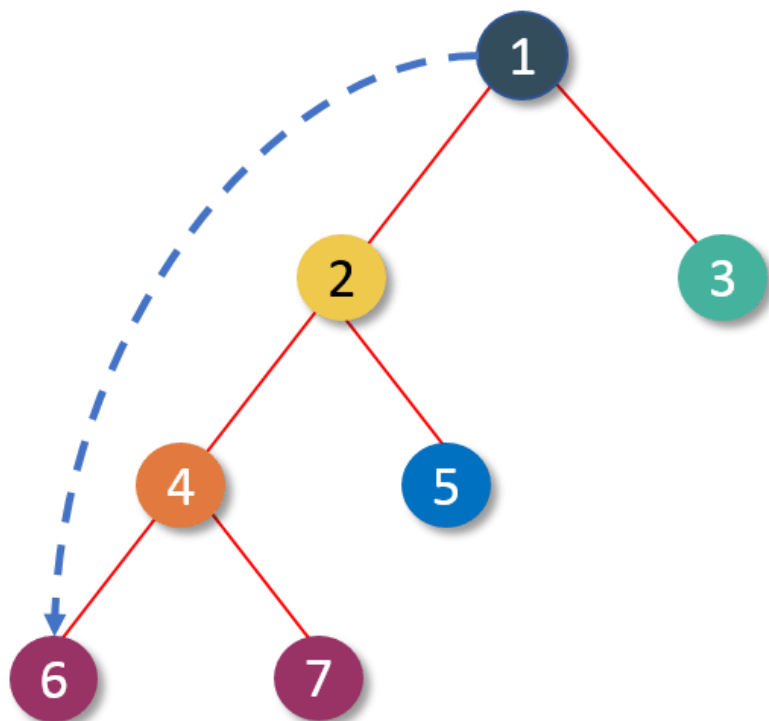




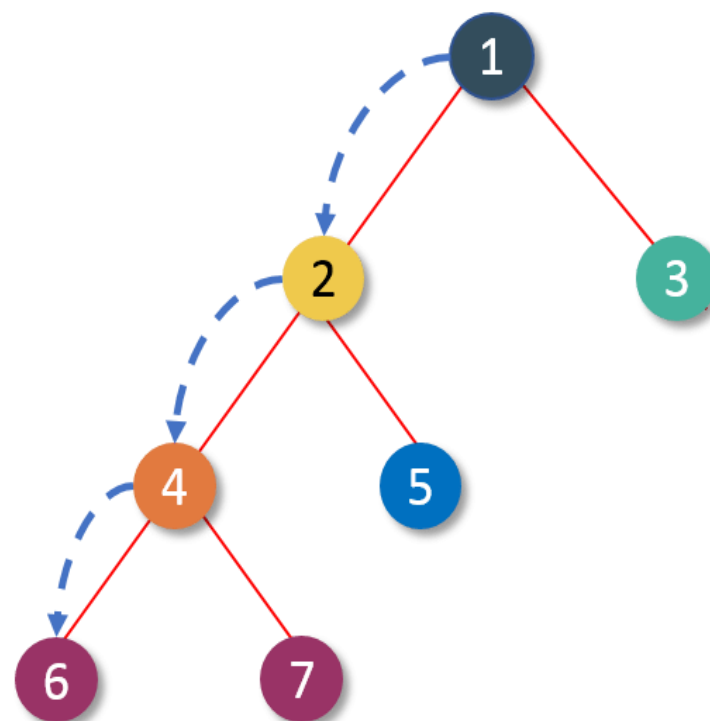
Props

Prop is a way of passing data from parent to child component.

UI Tree



UI Tree





Props

- ♦ **What are Props?**
- **Props (properties) are read-only data passed from a parent component to a child component.**
- 👉 **Child cannot modify props.**



Props

Example: Passing Props

```
jsx

function Parent() {
  return <Child name="React" />;
}

function Child(props) {
  return <h2>Hello {props.name}</h2>;
}
```

Key Points about Props

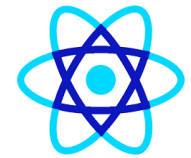
- ✓ Passed from parent to child
- ✓ Immutable (read-only)
- ✓ Used for configuration & display

Destructuring Props

```
jsx

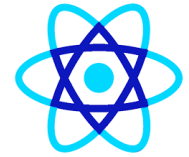
function Child({ name }) {
  return <h2>Hello {name}</h2>;
}
```

React Props Validation



SN	PropsType	Description
1.	PropTypes.any	The props can be of any data type.
2.	PropTypes.array	The props should be an array.
3.	PropTypes.bool	The props should be a boolean.
4.	PropTypes.func	The props should be a function.
5.	PropTypes.number	The props should be a number.
6.	PropTypes.object	The props should be an object.
7.	PropTypes.string	The props should be a string.

React Props Validation

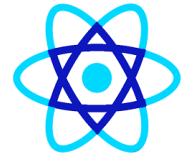


8.	<code>PropTypes.symbol</code>	The props should be a symbol.
9.	<code>PropTypes.instanceOf</code>	The props should be an instance of a particular JavaScript class.
10.	<code>PropTypes.isRequired</code>	The props must be provided.
11.	<code>PropTypes.element</code>	The props must be an element.
12.	<code>PropTypes.node</code>	The props can render anything: numbers, strings, elements or an array (or fragment) containing these types.
13.	<code>PropTypes.oneOf()</code>	The props should be one of several types of specific values.
14.	<code>PropTypes.oneOfType([PropTypes.string, PropTypes.number])</code>	The props should be an object that could be one of many types.



State

- **[2] State — Managing Dynamic Data with useState**
- **◆ What is State?**
- **State is data owned by a component that can change over time.**
- **👉 When state changes, React re-renders the component.**



Example: useState

jsx

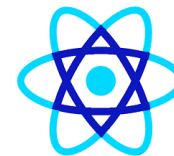
```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

Key Points about State

- ✓ Local to component
- ✓ Mutable using setter function
- ✓ Triggers re-render



Props vs State

Feature	Props	State
Who owns it	Parent	Component itself
Mutable	✗ No	✓ Yes
Purpose	Pass data	Manage data
Triggers re-render	✗	✓



Props vs State

SN	Props	State
1.	Props are read-only.	State changes can be asynchronous.
2.	Props are immutable.	State is mutable.
3.	Props allow you to pass data from one component to other components as an argument.	State holds information about the components.
4.	Props can be accessed by the child component.	State cannot be accessed by child components.
5.	Props are used to communicate between components.	States can be used for rendering dynamic changes with the component.
6.	Stateless component can have Props.	Stateless components cannot have State.
7.	Props make components reusable.	State cannot make components reusable.
8.	Props are external and controlled by whatever renders the component.	The State is internal and controlled by the React Component itself.



4 One-Way Data Flow (Very Important Concept)

◆ What is One-Way Data Flow?

React follows **unidirectional data flow**:

nginx

Parent → Child → UI

- Data flows **downward only**
- Child cannot change parent state directly



4 One-Way Data Flow (Very Important Concept)

🔧 Example: Child Updating Parent (Correct Way)

```
jsx

function Parent() {
  const [count, setCount] = useState(0);

  return <Child count={count} onIncrement={() => setCount(count + 1)} />;
}

function Child({ count, onIncrement }) {
  return (
    <>
      <p>{count}</p>
      <button onClick={onIncrement}>+</button>
    </>
  );
}
```

- ✓ Parent owns state
- ✓ Child triggers update via callback



4 One-Way Data Flow (Very Important Concept)

-  **Why One-Way Data Flow Matters**
-  Predictable behavior
-  Easier debugging
-  Better performance
-  Clear ownership of data



4 One-Way Data Flow (Very Important Concept)

-  **Common Beginner Mistakes**
-  Trying to modify props
-  Sharing state unnecessarily
-  Updating state directly (state++)



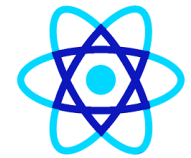
Index.js

- npm start
- Below adding individual links for above commands for node @8.9.4
- @node_modules/react-scripts/bin/react-scripts.js >> line number 35
- @node_modules/react-scripts/scripts/start.js >> line number 46
- @node_modules/react-scripts/config/webpack.config.dev.js >> line number 21, 102
- @node_modules/react-scripts/config/paths.js >> line number 56



Atoms, Molecules and Organisms

- In ReactJS and Atomic Design, components can be organized into three main categories: Atoms, Molecules, and Organisms.
- This pattern helps in building reusable and scalable UI components in a structured and hierarchical way.
- 1. Atoms are the simplest, most fundamental building blocks in our application.
- They represent the smallest and most basic UI components that cannot be broken down any further.
- They often correspond to basic HTML elements like buttons, inputs, labels, etc.



Recaptcha Admin Console

Google reCAPTCHA

← Register a new site



Get started with reCAPTCHA

Add advanced features like MFA, spam/fraud protection & Google Cloud integration.

- ✓ Up to 10,000 assessments/month at no cost
- ✓ No Credit Card required

<https://www.google.com/recaptcha/admin/create>

Label ⓘ

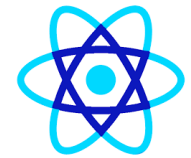
e.g. example.com

0 / 50

reCAPTCHA type ⓘ

- ☐ Score based (v3) Verify requests with a score
- ☒ Challenge (v2) Verify requests with a challenge

Recaptcha Admin Console



reCAPTCHA type (i)

- ☐ Score based (v3) Verify requests with a score
- ☒ Challenge (v2) Verify requests with a challenge
 - ☒ "I'm not a robot" Checkbox Validate requests with the "I'm not a robot" checkbox
 - ☐ Invisible reCAPTCHA badge Validate requests in the background

Domains (i)

+ localhost

Google Cloud Platform

It looks like you've used Google Cloud before. To get started, we'll create a new project and enable the necessary APIs.

Project Name*

 vebdentalcare

13 / 30



Recaptcha Admin Console

Label ⓘ

chitapp

7 / 50

reCAPTCHA type: v2 Checkbox

[VIEW IN CLOUD CONSOLE](#) ↗

reCAPTCHA keys ^

Use this site key in the HTML code your site serves to users. [See client side integration](#)

🔑 COPY SITE KEY

6LeE5U4qAAAAABdXr-9Lb53OA-nTJdrEnvRprFAf

Use this secret key for communication between your site and reCAPTCHA. [See server side integration](#)

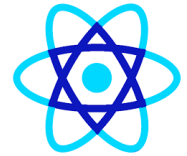
🔑 COPY SECRET KEY

6LeE5U4qAAAAANUJjZYVI5NB8vgg1efdF54WYRs8

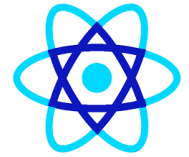
Domains ⓘ

✕ localhost

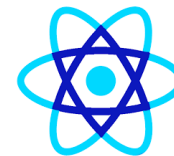
Recaptcha HTML Content Security Policy



```
<head>
<meta charset="utf-8" />
<link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<meta http-equiv="Content-Security-Policy" content="script-src 'self' 'nonce-abc123' https://apis.google.com;">
<meta http-equiv="Content-Security-Policy" content="script-src 'self' 'nonce-abc123' https://apis.google.com https://www.google.c
<meta name="theme-color" content="#000000" />
<meta
  name="description"
  content="Web site created using create-react-app"
/>
<link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
<!--
  manifest.json provides metadata used when your web app is installed on a
  user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
-->
<script nonce="abc123" src="https://www.google.com/recaptcha/api.js?onload=onloadcallback&render=explicit" async defer></script>
```



- Before Hooks (React <16.8), only **class components** could:
 - ✓ store state
 - ✓ use lifecycle methods
 - ✓ access context
- Now function components can do all thes



React Hook

- In React, Hooks gives functional components access to use the states and manage side effects.
- They were introduced to React 16.8.
- They let developers use state and other React features without writing a class
- For example, The state of a component It is important to note that hooks are not used inside the classes.



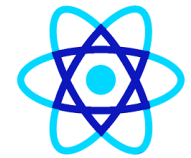
When to use React Hooks

- If you have created a function component in React and later need to add state to it, you used to have to convert it to a class component.
- However, you can now use a Hook within the existing function component to add a state.
- This way, you can avoid having to rewrite your code as a class component.



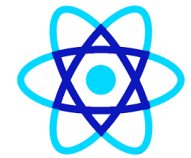
Types of React Hooks

- The Built-in React Hooks are:
 - State Hooks
 - Context Hooks
 - Ref Hooks
 - Effect Hooks
 - Performance Hooks
 - Resource Hooks
 - Other Hooks



React State Hooks

- State Hooks stores and provide access to the information.
- To add state in Components we use:
 - useState Hook: useState Hooks provides state variable with direct update access.
 - useReducer Hook: useReducer Hook provides a state variable along with the update logic in reducer function.



React Context Hooks

- Context hooks make it possible to access the information without being passed as a prop.
- useContext Hooks:
 - shares the data as a global data with passing down props in component tree.
 - `const context = useContext(myContext);`



React Ref Hooks

- Refs creates the variable containing the information not used for rendering e.g. DOM nodes.
- useRef:
 - Declares a reference to the DOM elements mostly a DOM Node.
 - useImperativeHandle: It is an additional hook that declares a customizable reference
 - `const textRef = useRef(null);`



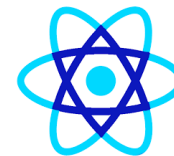
Effect Hooks:

- Effects connect the components and make it sync with the system.
- It includes changes in browser DOM, networks and other libraries.
- Runs side-effects (API calls, timers, subscriptions).
- `useEffect`:
 - `useEffect` Hook connects the components to external system
 - `useLayoutEffect`: used to measure the layout, fires when the screen rerenders.
 - `useInsertionEffect`: used to insert the CSS dynamically, fires before the changes made to DOM.



Effect Hooks:

```
useEffect(()->{  
  // Code to be executed  
}, [dependencies] )
```



React Performance Hooks

- Performance hooks are a way to skip the unnecessary work and optimise the rendering performance.
 - `useMemo`: return a memoized value reducing unnecessary computations.
 - `useCallback`: returns a memoized callback that changes if the dependencies are changed.
 - `const memoizedValue = useMemo(functionThatReturnsValue, arrayDependencies)`



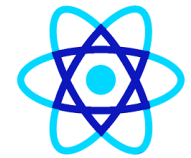
React Resource Hooks:

- It allows component to access the resource without being a part of their state e.g., accessing a styling or reading a promise.
 - use: used to read the value of resources e.g., context and promises.
 - `const data = use(dataPromise);`



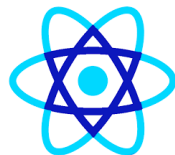
Benefits of using React Hooks

- Hooks can improve code reusability and make it easier to split complex components into smaller functions.
 - Simpler, cleaner code: Functional components with hooks are often more concise and easier to understand than class components.
 - Better for complex UIs: Hooks make it easier to manage state and side effects in components with intricate logic.
 - Improved maintainability: Code using hooks is often easier to test and debug.



Why the need for ReactJs Hooks ?

- There are multiple reasons responsible for the introduction of the Hooks which may vary depending upon the experience of developers in developing React application.
- Needs for react hooks are:
 - Use of 'this' keyword
 - Reusable stateful logics
 - Simplifying complex scenarios



Why the need for ReactJs Hooks ?

```
// Filename - index.js

import React, { useState } from "react";
import ReactDOM from "react-dom/client";

function App() {
  const [click, setClick] = useState(0);

  // Using array destructuring here
  // to assign initial value 0
  // to click and a reference to the function
  // that updates click to setClick
  return (
    <div>
      <p>You clicked {click} times</p>
      <button onClick={() => setClick(click + 1)}>
        Click me
      </button>
    </div>
  );
}

const root = ReactDOM.createRoot(
  document.getElementById("root")
);
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```



Custom Hooks Using Reuse

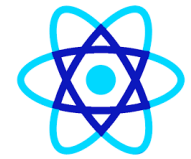
```
function useSimpleCategory( title, items) {  
  const [search, setSearch] = useState("");  
  
  const filteredItems = useMemo(() => {  
    const q = search.trim().toLowerCase();  
    if (!q) return items;  
  
    return items.filter((item) => {  
      const name = String(item?.name ?? "").toLowerCase();  
      const category = String(item?.category ?? "").toLowerCase();  
      const price = String(item?.price ?? "").toLowerCase(); // optional  
  
      return (  
        name.includes(q) ||  
        category.includes(q) ||  
        price.includes(q)  
      );  
    });  
  }, [items, search]);  
  
  return { title, search, setSearch, items: filteredItems };  
}
```



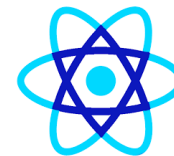

Custom Hooks Using Reuse

- ✓ Custom hooks **must**:
 - start with use
 - be called **unconditionally**
 - not be treated as a component
- useMemo is a React hook that remembers (memoizes) the result of a calculation and recomputes it only when its dependencies change.

Differences between React Hook Vs Class Component



Feature	Class Components	React Hooks
State Management	<code>this.state</code> and lifecycle methods	<code>useState</code> and <code>useEffect</code>
Code Structure	Spread across methods, can be complex	Smaller, focused functions
Reusability	Difficult to reuse logic	Easy to create and reuse custom hooks
Learning Curve	Familiar to OOP developers	Requires different mindset than classes
Error Boundaries	Supported	Not currently supported
Third-party Libraries	Some libraries rely on them	May not all be compatible yet



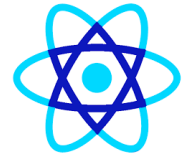
React Router

Npm install react-router-dom

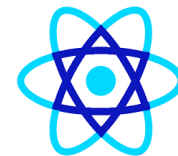
Index.js

```
const app=(  
  <BrowserRouter>  
    <App/>  
  </BrowserRouter>  
)  
  
ReactDOM.render(app,  
document.getElementById('root'));
```

Different types of routers in react router



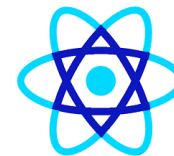
- Static Routing: For simple page navigation.
- Nested Routing: For hierarchically structured routes.
- Dynamic Routing: Handling variable URLs.
- Protected Routes: Routes restricted to authenticated users.
- Redirects: Automatically navigating to other routes.
- Lazy Loading: Improving performance by loading components on demand.
- Hash Router: URL-based routing using hashes.
- Memory Router: In-memory routing for testing or environments without a URL.
- Query Params: Handling query strings in URLs.



Static Routing

- Use Case: The most common routing type used in SPAs for navigating between static pages (e.g., home, about, contact).

```
jsx Copy code  
  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
import Home from './Home';  
import About from './About';  
import Contact from './Contact';  
  
const App = () => (  
  <Router>  
    <Routes>  
      <Route path="/" element={<Home />} />  
      <Route path="/about" element={<About />} />  
      <Route path="/contact" element={<Contact />} />  
    </Routes>  
  </Router>  
>);  
  
export default App;
```



Nested Routing

- Use Case: When you want to have child routes (nested pages) inside a parent route.
- For example, a blog page can have multiple blog posts.



Nested Routing

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
```

[Copy code](#)

```
const Blog = () => (
```

```
  <div>
```

```
    <h2>Blog</h2>
```

```
    <Outlet /> { /* This will render the nested routes */ }
```

```
  </div>
```

```
);
```

```
const BlogPost = ({ postId }) => <div>Blog Post {postId}</div>;
```

```
const App = () => (
```

```
  <Router>
```

```
    <Routes>
```

```
      <Route path="/blog" element={<Blog />}>
```

```
        <Route path="post1" element={<BlogPost postId="1" />} />
```

```
        <Route path="post2" element={<BlogPost postId="2" />} />
```

```
      </Route>
```

```
    </Routes>
```

```
  </Router>
```

```
);
```



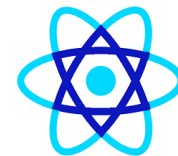


Dynamic Routing

- Use Case: Dynamic routes are useful for handling variable parts of the URL, such as showing different pages or posts based on dynamic IDs or slugs (e.g., viewing a specific product, user profile, or blog post).

```
jsx Copy code  
  
import { BrowserRouter as Router, Routes, Route, useParams } from 'react-router-dom';  
  
const Product = () => {  
  const { productId } = useParams();  
  return <div>Product ID: {productId}</div>;  
};  
  
const App = () => (  
  <Router>  
    <Routes>  
      <Route path="/product/:productId" element={<Product />} />  
    </Routes>  
  </Router>  
>;  
  
export default App;
```





Protected Routes (Private Routes)

- Use Case: When you want to restrict certain routes to authenticated users only. For example, a dashboard or profile page should only be accessible after logging in.

```
jsx Copy code

import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';

const isAuthenticated = false; // Set this to true if the user is authenticated

const ProtectedRoute = ({ children }) => {
  return isAuthenticated ? children : <Navigate to="/login" />;
};

const Dashboard = () => <div>Dashboard: Protected Content</div>;
const Login = () => <div>Login Page</div>;

const App = () => (
  <Router>
    <Routes>
      <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
      <Route path="/login" element={<Login />} />
    </Routes>
  </Router>
);
```





Redirects

- Use Case: When you need to automatically redirect the user to another route. This is often used for login redirects, handling 404 pages, or other cases where navigation needs to be automated.

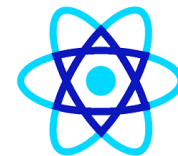
jsx

Copy code

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';

const App = () => (
  <Router>
    <Routes>
      <Route path="/old-path" element={<Navigate to="/new-path" />} />
      <Route path="/new-path" element={<div>New Path Content</div>} />
      <Route path="*" element={<div>404 Not Found</div>} /> {/* Catch-all for undefined routes */}
    </Routes>
  </Router>
);

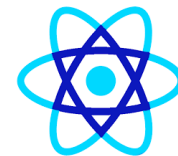
export default App;
```



Lazy Loading (Code-Splitting Routes)

- Use Case: For large applications where you want to load components lazily to improve performance. Pages are loaded only when needed.

```
jsx Copy code  
  
import React, { Suspense, lazy } from 'react';  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
const Home = lazy(() => import('./Home'));  
const About = lazy(() => import('./About'));  
const Contact = lazy(() => import('./Contact'));  
  
const App = () => (  
  <Router>  
    <Suspense fallback={<div>Loading...</div>}>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="/contact" element={<Contact />} />  
      </Routes>  
    </Suspense>  
  </Router>  
>);
```



Lazy Loading (Code-Splitting Routes)

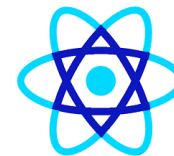
- Use Case: For large applications where you want to load components lazily to improve performance. Pages are loaded only when needed.

```
jsx Copy code  
  
import React, { Suspense, lazy } from 'react';  
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';  
  
const Home = lazy(() => import('./Home'));  
const About = lazy(() => import('./About'));  
const Contact = lazy(() => import('./Contact'));  
  
const App = () => (  
  <Router>  
    <Suspense fallback={<div>Loading...</div>}>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="/contact" element={<Contact />} />  
      </Routes>  
    </Suspense>  
  </Router>  
>);
```



Memory Router

- A router which doesn't change the URL in your browser instead it keeps the URL changes in memory
- It is very useful for testing and non browser environments 
- But in browser, It doesn't have history. So you can't go back or forward using browser history 



Memory Router

- Use Case: Primarily used for testing or non-browser environments. It keeps the history of your "navigation" in memory (does not interact with the browser URL).

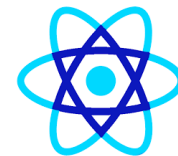
jsx

 Copy code

```
import { MemoryRouter, Routes, Route } from 'react-router-dom';

const App = () => (
  <MemoryRouter>
    <Routes>
      <Route path="/" element={<div>Home</div>} />
      <Route path="/about" element={<div>About</div>} />
    </Routes>
  </MemoryRouter>
);

export default App;
```



Query Params Handling

- Use Case: Useful when your route needs to handle query parameters (e.g., /search?query=react).

```
jsx
import { BrowserRouter as Router, Routes, Route, useSearchParams } from 'react-router-dom'

const Search = () => {
  const [searchParams] = useSearchParams();
  const query = searchParams.get('query');

  return <div>Search query: {query}</div>;
};

const App = () => (
  <Router>
    <Routes>
      <Route path="/search" element={<Search />} />
    </Routes>
  </Router>
);

export default App;
```



Hash Router

- A router which uses client side hash routing.
- Whenever, there is a new route get rendered, it updated the browser URL with hash routes. (eg., `/#/about`)
- Hash portion of the URL won't be handled by server, server will always send the `index.html` for every request and ignore hash value. Hash value will be handled by react router.
- It is used to support legacy browsers which usually doesn't support HTML `pushState` API 
- It doesn't need any configuration in server to handle routes 
- This route isn't recommended by the team who created react router package. Use it only if you need to support legacy browsers or don't have server logic to handle the client side routes 



Hash Router

- Use Case: Useful for applications that are hosted on static file servers where the server doesn't handle requests for all URLs (e.g., GitHub Pages).

jsx

Copy code

```
import { HashRouter as Router, Routes, Route } from 'react-router-dom';
import Home from './Home';
import About from './About';

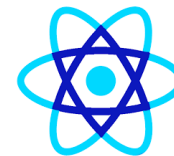
const App = () => (
  <Router>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
    </Routes>
  </Router>
);

export default App;
```



Browser Router

- The widely popular router and a router for modern browsers which use HTML5 pushState API. (i.e., pushState, replaceState and popState API).
- It routes as normal URL in browser, you can't differentiate whether it is server rendered page or client rendered page through the URL.
- It assumes, your server handles all the request URL (eg., /, /about) and points to root index.html. From there, BrowserRouter take care of routing the relevant page.
- It accepts forceRefresh props to support legacy browsers which doesn't support HTML5 pushState API 



React Router

Npm install react-router-dom

Index.js

```
const app=(  
  <BrowserRouter>  
    <App/>  
  </BrowserRouter>  
)  
  
ReactDOM.render(app,  
document.getElementById('root'));
```



Uncontrolled Element

```
import React, {useRef, useState} from 'react';
```

1+ usages

```
function NewVacancy(){
```

```
  const [age : string , setAge] = useState( initialState: '' );
```

```
  const nameInputRef : MutableRefObject<null> = useRef( initialValue: null );
```

1+ usages

```
  const handleSubmit = (event) : void => {
```

```
    event.preventDefault();
```

```
    //uncontrolled element
```

```
    const name = nameInputRef.current.value;
```

```
    alert(`Name: ${name}, Age: ${age}`);
```

```
  };
```

```
  return (
```

```
    <form onSubmit={handleSubmit}>
```

```
      <div>
```

```
        <label>Name: </label>
```

```
        <input type="text" ref={nameInputRef} />
```

```
      </div>
```

```
      <div>
```

```
        <label>Age: </label>
```

```
        <input
```

```
          type="number"
```

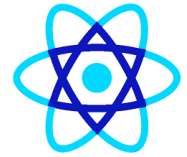
```
          value={age}
```

```
          onChange={(e : ChangeEvent<HTMLInputElement> ) : void => setAge(e.target.value)} // Controlled input
```

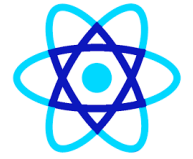
```
        />
```

```
      </div>
```

```
      <button type="submit">Submit</button>
```



- Apollo: Easy to set up and use GraphQL client.
- Axios: Promise based HTTP client for the browser and node.js.
- Relay Modern - A JavaScript framework for building data-driven React applications.
- Request: Simplified HTTP request client.
- Superagent: A lightweight “isomorphic” library for AJAX requests.

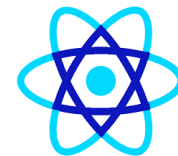


-
- `npm install axios --save`



HOC (High Order Components)

- Higher-Order Components (HOCs) in React are a pattern used for reusing component logic.
- An HOC is a function that takes a component as an argument and returns a new component with enhanced behavior.
- This allows developers to share common functionality between components without duplicating code.



HOC (High Order Components)

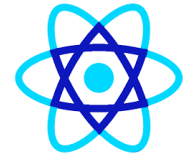
- What is a Higher-Order Component?
 - A Higher-Order Component is essentially a function that Receives a component as an input.
 - Returns a new component that is a wrapped version of the original component with added or modified behavior.
- HOCs are often used for tasks such as:
 - Code reuse and logic abstraction.
 - Managing side effects like data fetching.
 - Handling permissions or authentication.
 - Modifying props



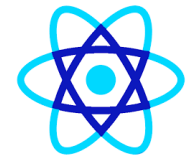
Why use HOCs?

- HOCs help you:
 - Share logic between components
 - Keep components **clean & focused**
 - Avoid code duplication
 - Implement **cross-cutting concerns**
- Common use cases:
 - Authentication / Authorization
 - Logging & analytics
 - Data fetching
 - Permission checks
 - Error handling
 - Theming

vs HOC vs Hooks

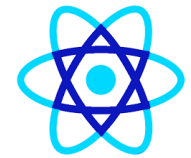


Feature	HOC	Hooks
Logic reuse	✓	✓
Wrapper components	✗ Adds extra layers	✓ No wrapper
Readability	⚠ Can be complex	✓ Cleaner
Modern React	⚠ Legacy pattern	✓ Preferred



Context API vs Redux

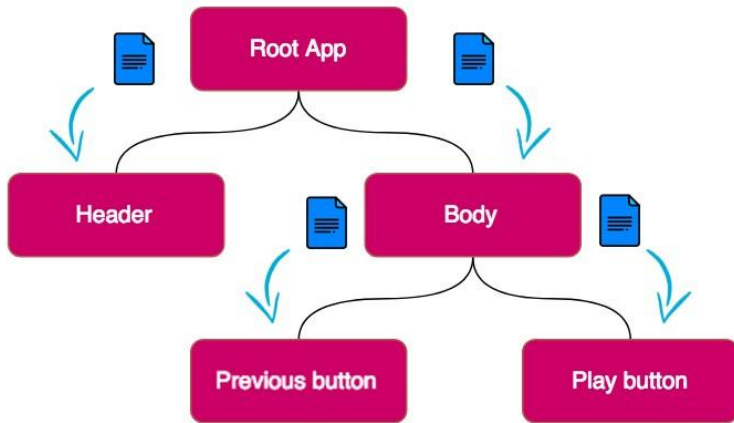
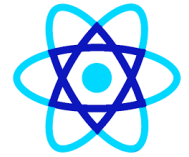
Context API	Redux
Built-in tool	Install the libraries
Requires minimal setup	Extensive setup
It is meant for static data which is not often refreshed or updated	Works for both static and Dynamic
Adding new contexts requires creation from scratch	Easily extendible due to ease of adding new data/actions after the initial setup



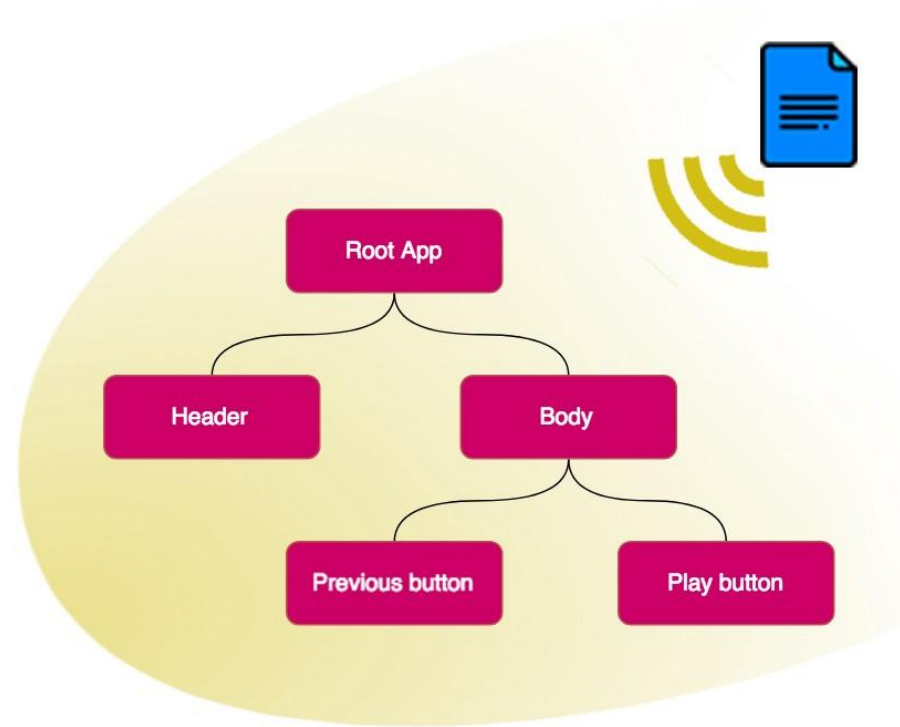
Context API vs Redux

Context API	Redux
Debugging is hard because it is recommended for nested react components	Dev tool for redux
UI logic and state are in the same component	UI logic and state separated from component

Context API

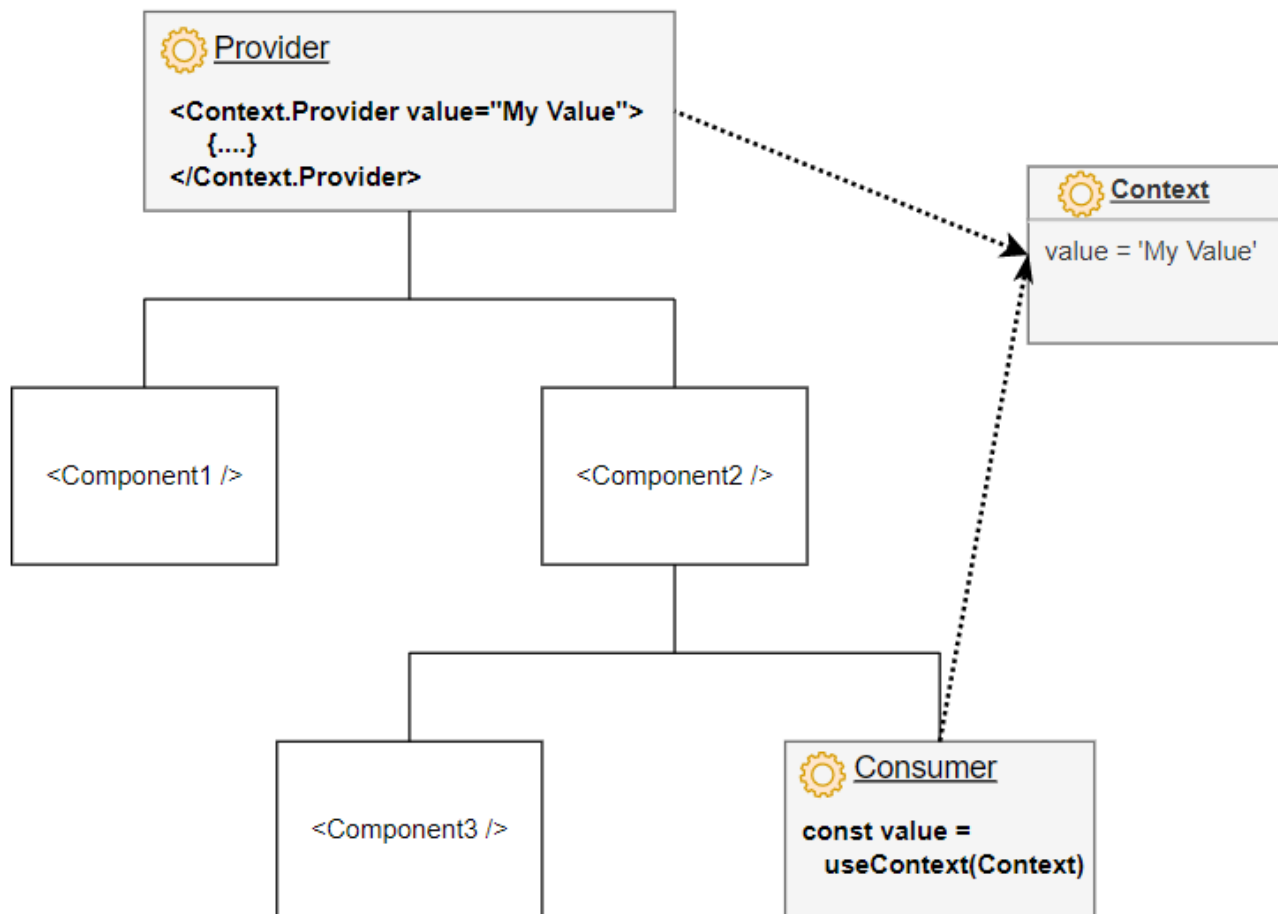
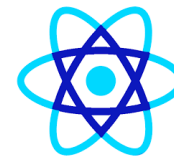


Passing prop to each level



The data in Context available to every component

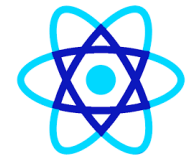
Context API





What is Redux

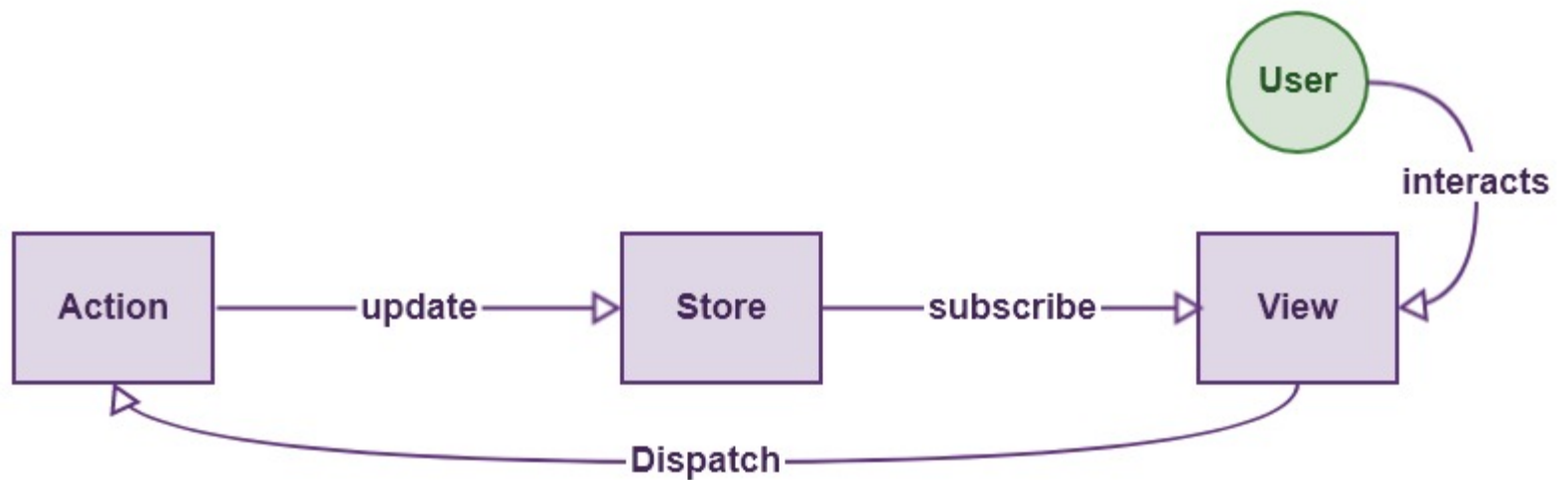
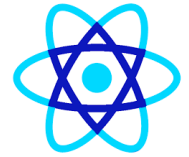
- Redux is a **state management library** for JavaScript apps (very common with React).
- It solves: **many components need the same data + predictable updates.**

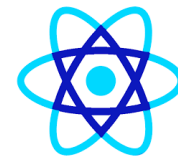


Why Redux is needed

- In React, state is usually:
 - local (useState)
 - shared via props (prop drilling)
 - shared via Context
- Redux is useful when:
 - many screens need same state (user, cart, products)
 - you need predictable updates, debugging, time-travel, logs
 - state changes are complex (cart, checkout, auth)
- Example in your app:
 - cartItems needed in Header + Cart page + Checkout page
 - selectedCategory needed in sidebar + product list
 - userLogin needed everywhere

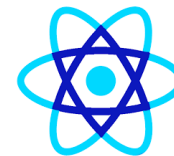
Core concepts of Redux





Core concepts of Redux

- **1 Store**
- A **single object** that holds the entire application state
- Acts as the **single source of truth**
- {
 - auth: {...},
 - cart: {...},
 - user: {...}
- }



Core concepts of Redux

- **2 Action**
- A **plain JavaScript object**
- Describes *what happened*
- { type: "ADD_ITEM", payload: item }



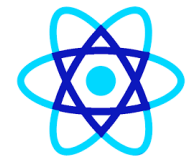
Core concepts of Redux

- **3 Reducer**
- A **pure function**
- Decides *how the state changes*
- $(\text{state}, \text{action}) \Rightarrow \text{newState}$



Core concepts of Redux

- **4 Dispatch**
- The method used to send actions to the store
- `dispatch({ type: "LOGOUT" })`



Why Redux

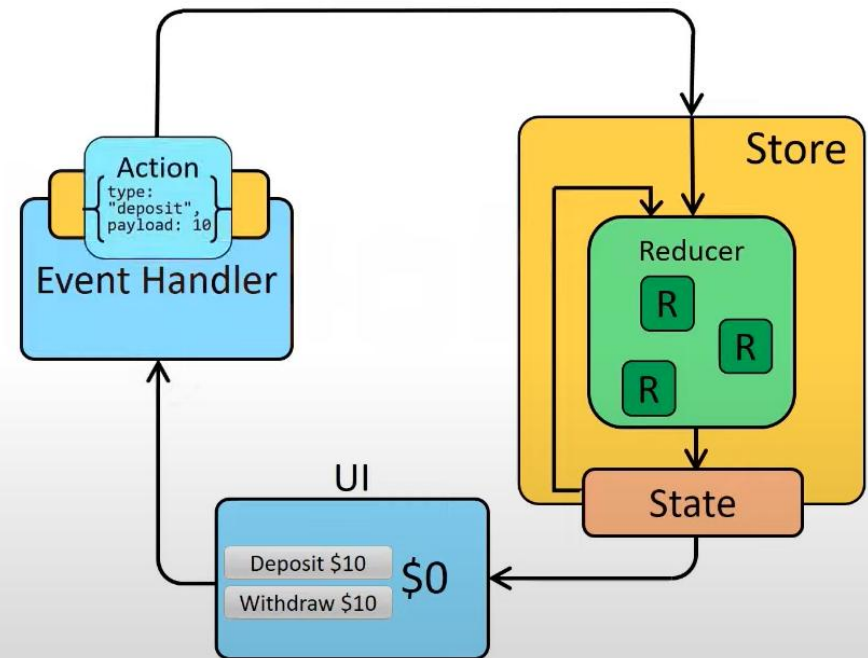
Centralized State Management

Predictable State Changes

Debugging

Developer Experience

Scalability





Why Redux

- **1 Centralized State Management**
- All application state lives in **one place (Store)**
- No scattered state across components
- 👉 Easy to access, update, and reason about



Why Redux

- **2 Predictable State Changes**
- State changes happen **only through actions & reducers**
- No hidden or accidental updates
- 👉 Same input (action) → same output (state)



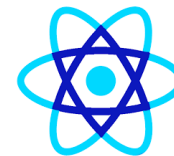
Why Redux

- **3 Debugging**
- Every action is logged
- You can inspect:
 - Previous state
 - Action
 - Next state
- 👉 Enables **Redux DevTools & time-travel debugging**



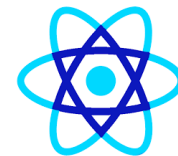
Why Redux

- **4 Developer Experience**
 - Clear structure: **Action** → **Reducer** → **Store**
 - Easier teamwork in large projects
 - Cleaner, readable code (especially with Redux Toolkit)
- **5 Scalability**
 - Works well for **large and complex applications**
 - Easy to add new features without breaking existing ones



Why Redux

-  **UI (Bottom – “Deposit / Withdraw”)**
- This is your **React UI component**.
- Example:
 - Deposit \$10
 - Withdraw \$10
 - Balance: \$0
- **UI does NOT change state directly.**



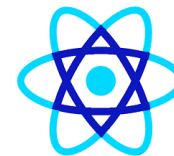
Why Redux

-  **Event Handler**
- Triggered by user actions (button click).
- Example:
- `dispatch({`
- `type: "deposit",`
- `payload: 10`
- `});`
- ✓ Converts UI events into Redux **actions**



Why Redux

-  **Action**
- A **plain JavaScript object** describing what happened:
 - {
 - type: "deposit",
 - payload: 10
 - }
- ✓ No logic
- ✓ Just data



Why Redux

-  **Store (Yellow Box)**
- **The Redux Store:**
 - Holds the entire application state
 - Single source of truth
 - Inside the store:
- **Reducers**
- **State**



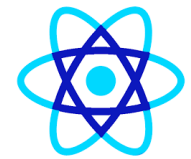
Why Redux

-  **Reducer (Green “R” blocks)**
- Reducers:
- Receive (state, action)
- Return **new state**
- Example:
 - `function balanceReducer(state = 0, action) {`
 - `if (action.type === "deposit") {`
 - `return state + action.payload;`
 - `}`
 - `return state;`
 - `}`
- ✓ Pure function
- ✓ No mutation

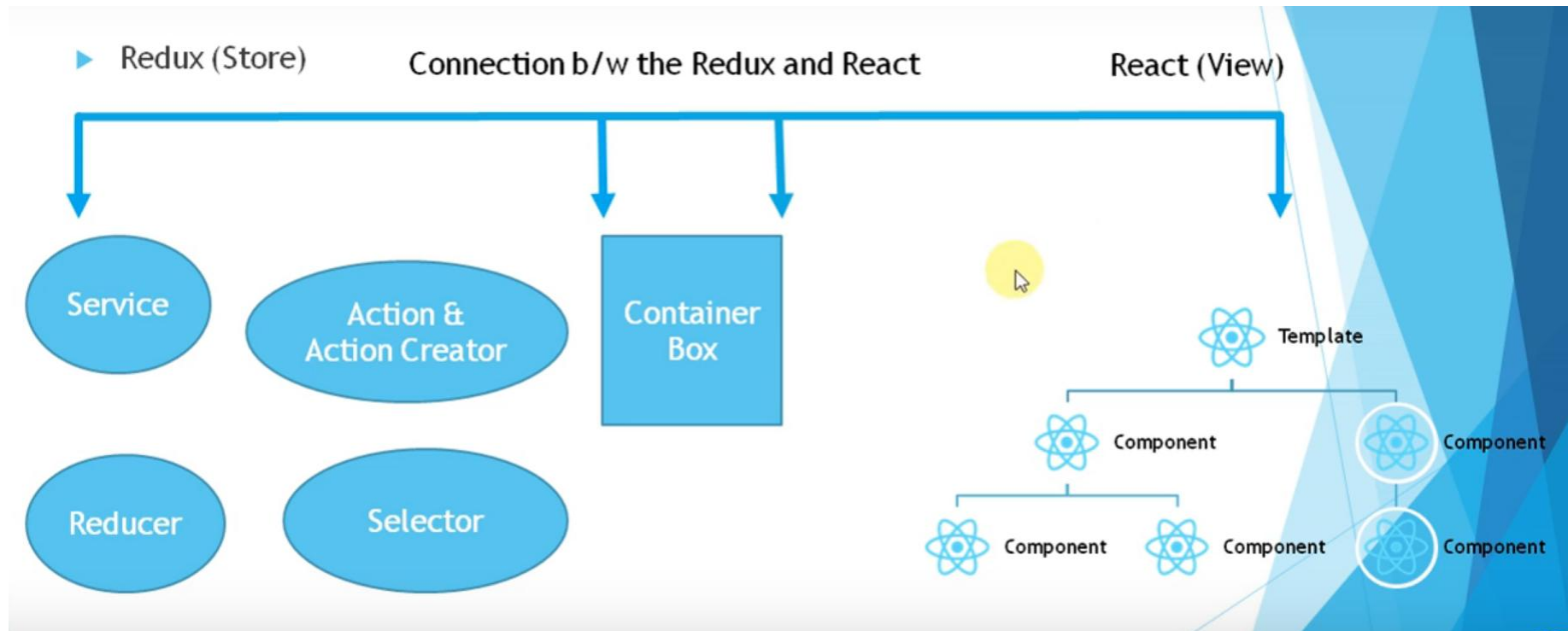


Why Redux

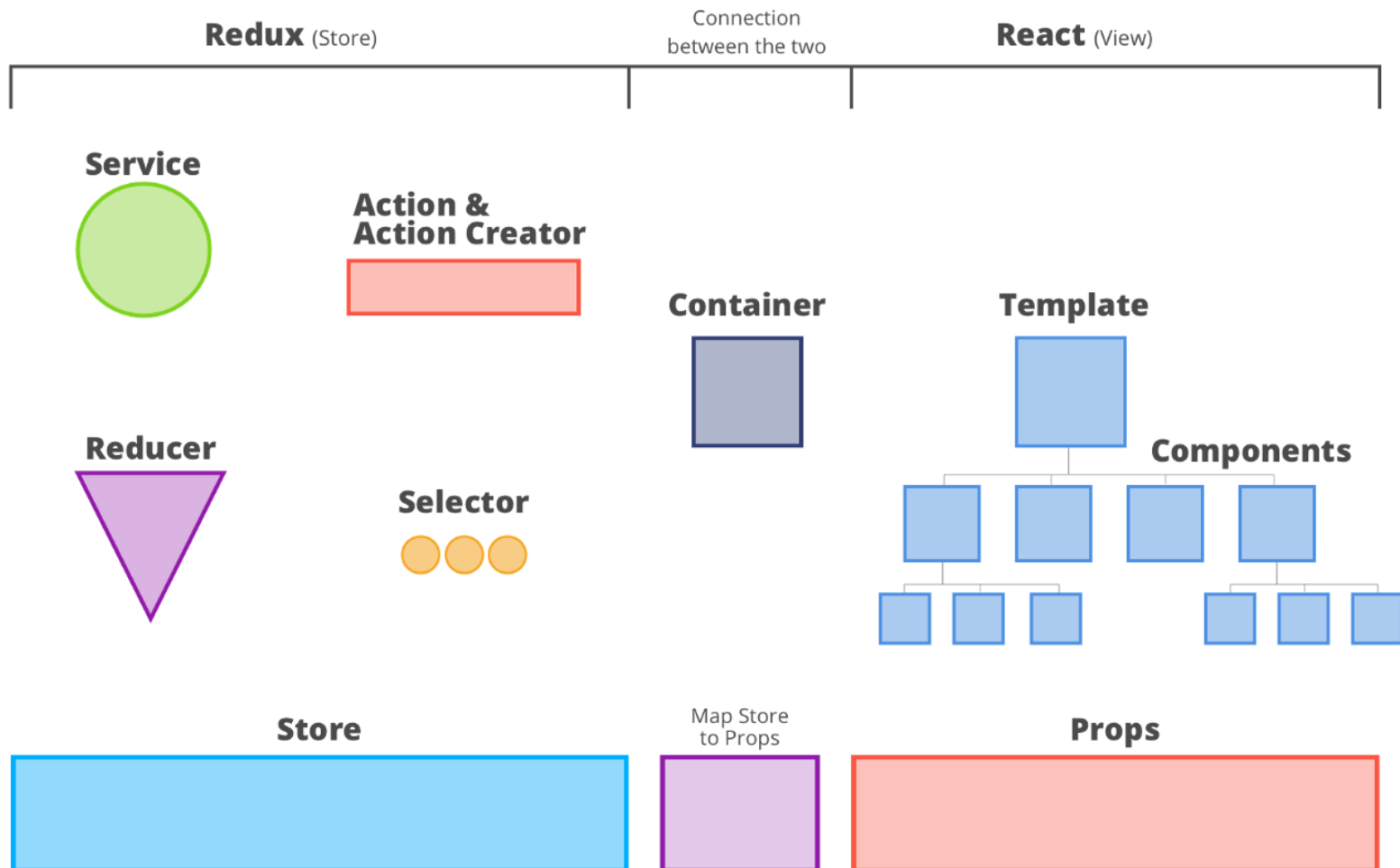
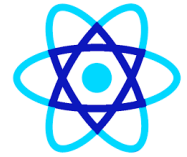
-  **State**
- Updated result from reducers
- Stored in Redux Store
- Example:
- `state = { balance: 10 }`

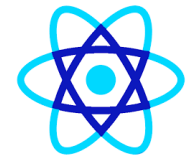


Redux With React Architecture

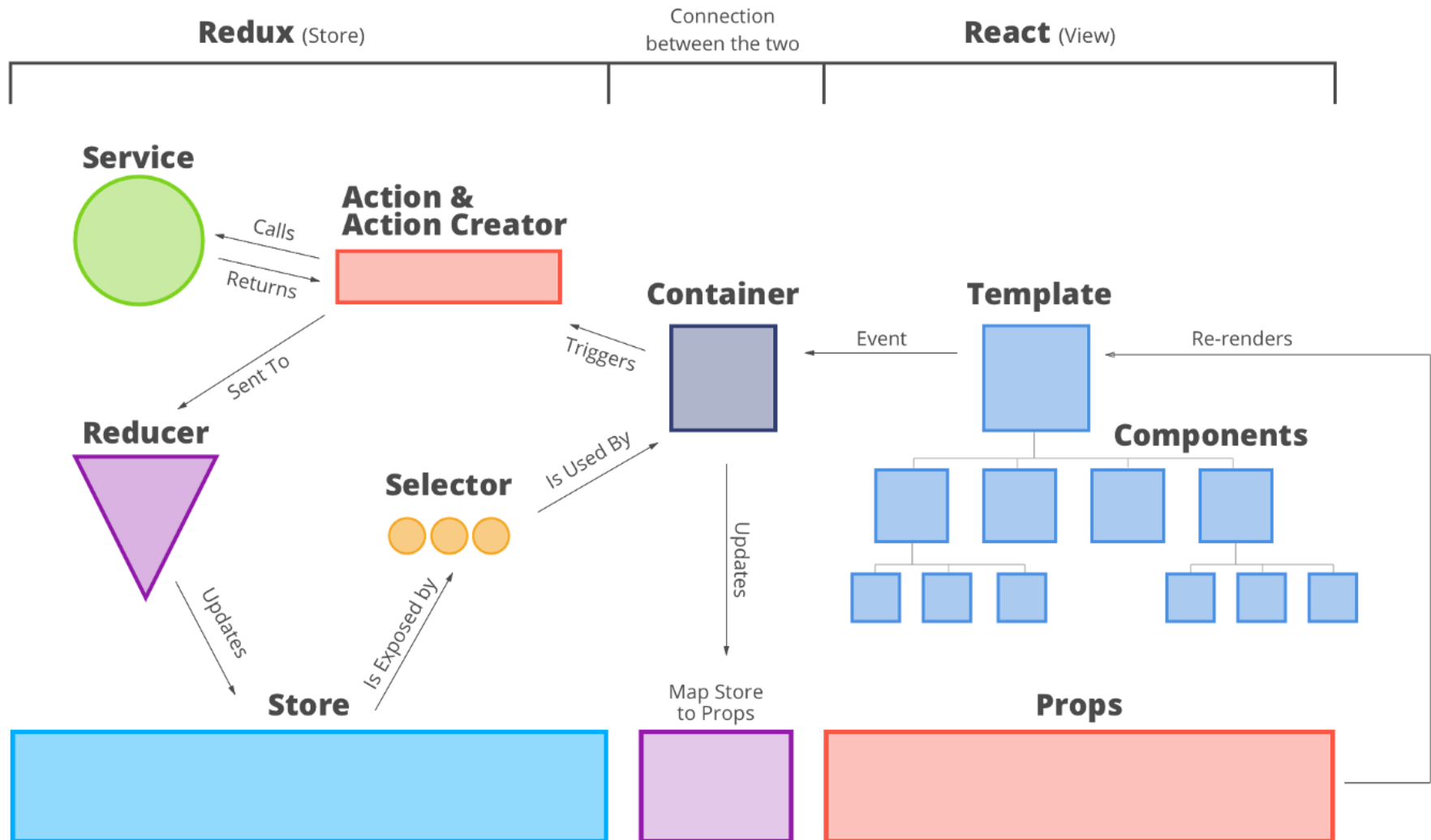


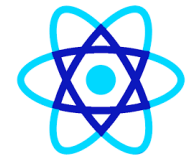
Why Redux





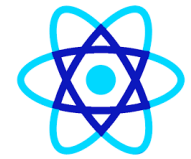
Why Redux





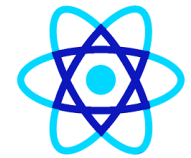
End to End Flow

- **5 Full Data Flow (End-to-End)**
-  **Step-by-step flow**
- User clicks a button in **Component**
- Event goes to **Container**
- Container **dispatches Action**
- Action calls **Service** (API)
- Service returns data
- Action is sent to **Reducer**



End to End Flow

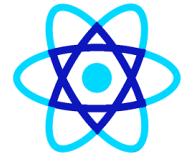
- **5 Full Data Flow (End-to-End)**
- Reducer creates **new state**
- Store is updated
- Selector extracts required data
- Container maps state → props
- React components **re-render**



Why This Architecture Is Powerful

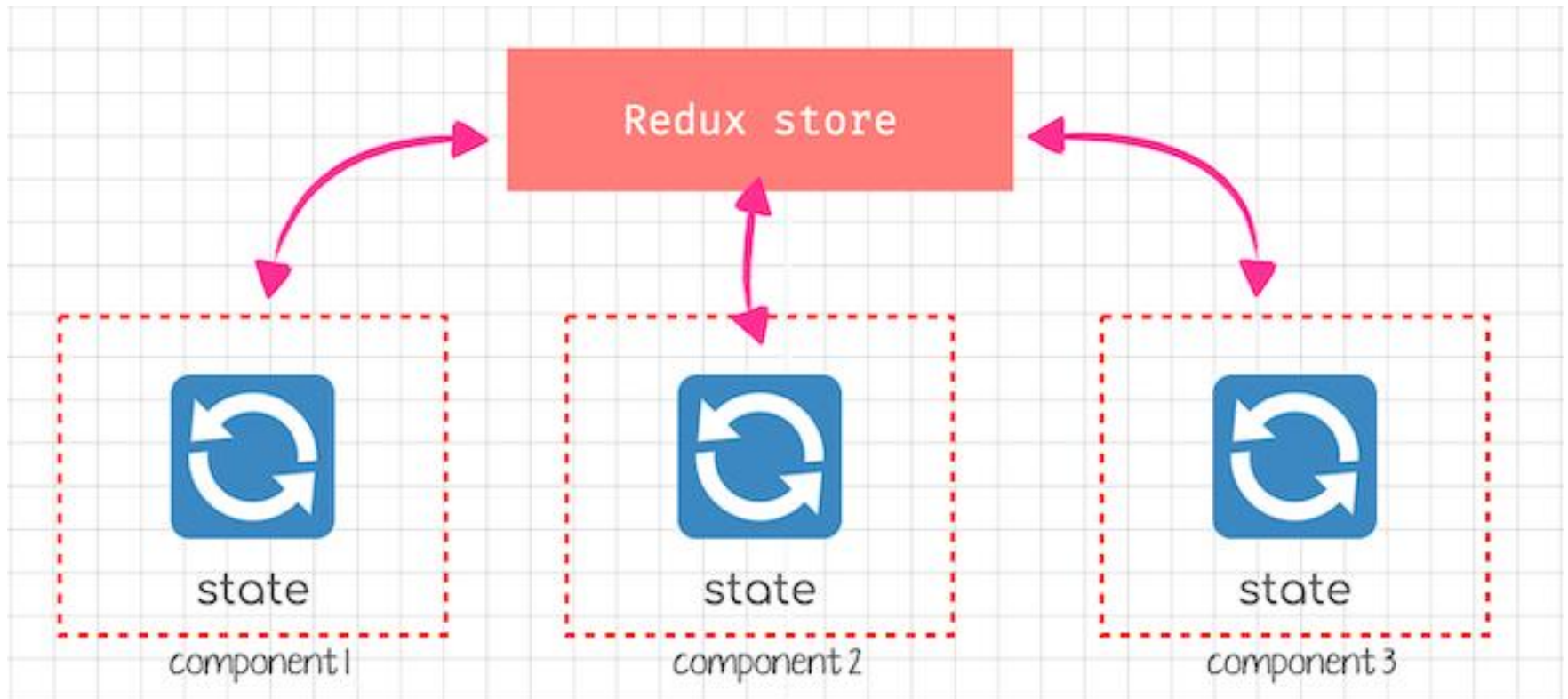
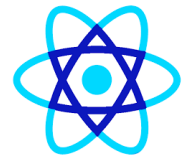
-  Clear separation of concerns
-  Predictable state updates
-  Easy debugging (Redux DevTools)
-  Scales well for large applications
-  Ideal for enterprise apps (like banking, insurance, dashboards)

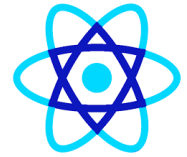
THREE PRINCIPLES OF REDUX



-
- Single source of truth.
 - The state is read-only.
 - Changes are made with pure functions.

Why Redux



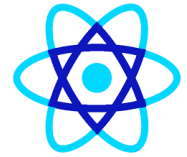


- A store is an object that brings them together. A store has the following responsibilities:
- Holds application state;
- Allows access to state via `getState()`;
- Allows state to be updated via `dispatch(action)`;
- Registers listeners via `subscribe(listener)`;
- It handles unregistering of listeners via the function returned by `subscribe(listener)`

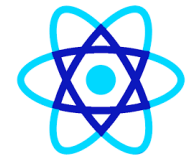


Redux Toolkit (Recommended)

- Today we use **Redux Toolkit (RTK)** because it:
 - reduces boilerplate
 - supports immutable updates easily (Immer)
 - includes dev tools friendly patterns
 - provides async helpers (createAsyncThunk)
 - supports RTK Query (data fetching)
 - Install:
 - `npm i @reduxjs/toolkit react-redux`



- Dumb Components
- Smart Component(Containers)
- Dumb components render the data, and they did not care about any logic. They have nothing to do with a redux store.
- Smart components are concern about logic and directly connected to the store.



Redux with mongodb

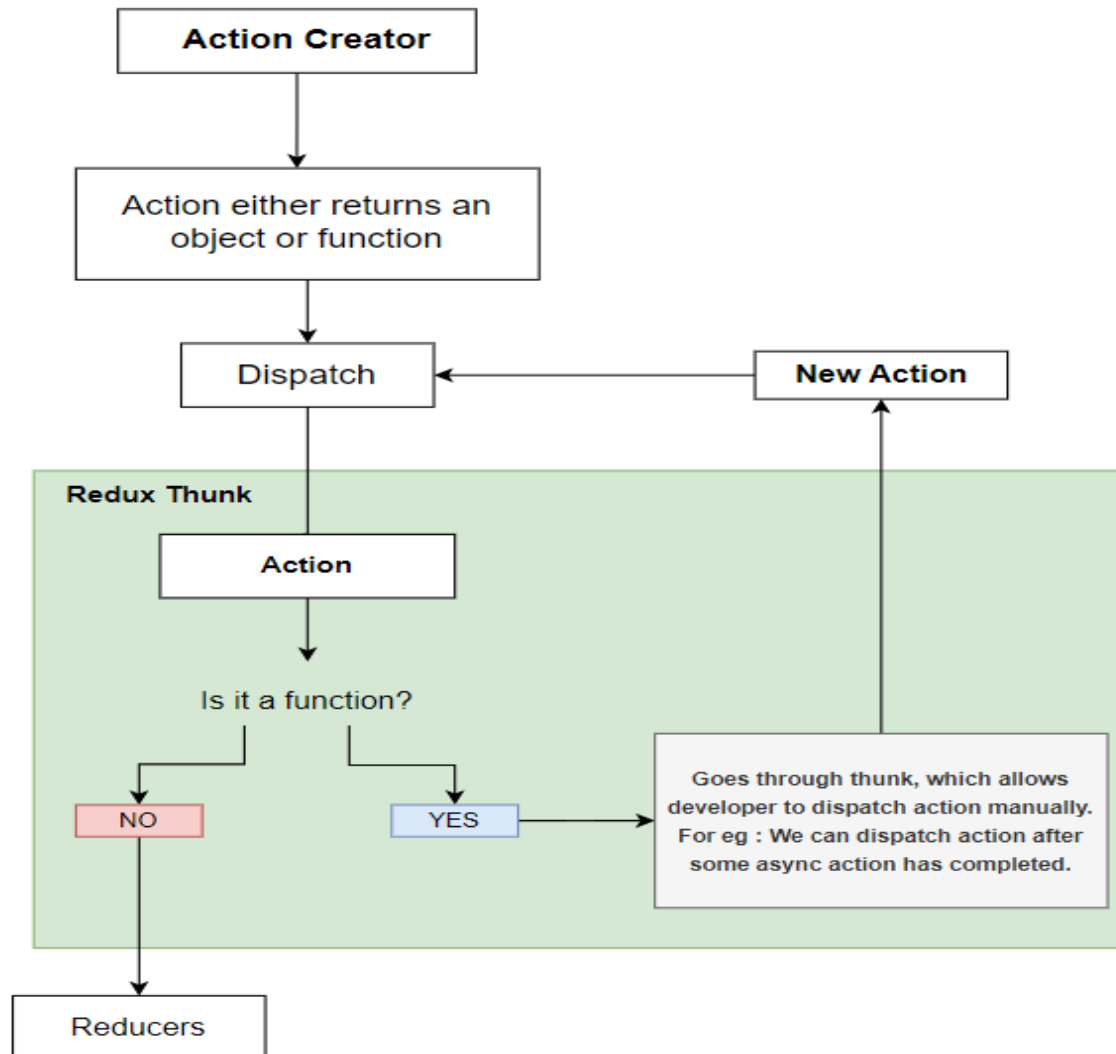
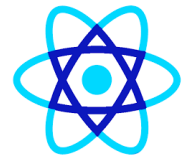
- Go to the project folder and install dependencies using this command: **npm install**
- **yarn add** redux react-redux
- **yarn add** @reduxjs/toolkit
- Start the mongodb server using this command: **mongod**
- Start the Node.js server using this command: **nodemon server/server**
- Start the Reacyt.js dev server using this command: **yarn start**



Redux Async

- `npm install redux-thunk --save`
- *For middleware*
- `npm install express body-parser mongoose --save`
- `npm install nodemon --save-dev`

Redux Async





Redux Steps

- Step 1
 - `npm i redux --save-dev`
- Step 2
 - `mkdir -p src/js/store`
 - Create `index.js`
 - `createStore` takes a reducer as the first argument
- Step 3
 - A Redux reducer is just a JavaScript function. It takes two parameters: the current state and action



Redux Steps

- Step 3
 - In a typical React component the local state might be mutated in place.
 - In Redux we're not allowed to do that.
 - The third principle of Redux prescribes that the state is immutable and cannot change in place.
 - The reducer must be pure. A pure function is one that returns the exact same output for the given input.



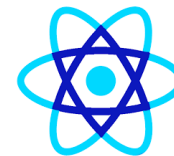
Redux Steps

- Step 4
 - mkdir -p src/js/reducers
 - create a new file, **src/js/reducers/index.js**
- Step 5
 - mkdir -p src/js/actions/index.js
- Step 6
 - Strings are prone to typos and duplicates and for this reason it's better to declare actions as constants.
 - Create a new folder for them
 - mkdir -p src/js/constants



Redux Steps

- `src/js/constants/action-types.js`
- `src/js/actions/index.js`
- Refactoring the reducer
 - how does a reducer know when to generate the next state? The key here is the Redux store.
 - When an action is dispatched, the store forwards a message (the action object) to the reducer.
 - At this point the reducer says "oh, let's look at the type property of this action".



Redux Steps

- Then depending on the action type, the reducer produces the next state, eventually merging the action payload into the new state.
- Open up `src/js/reducers/index.js` and update the reducer with an if statement for checking the action type



Redux store methods

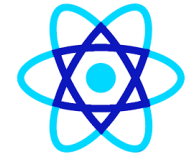
- Redux itself is a small library (2KB) and the most important methods are just three:
 - getState for reading the current state of the application
 - dispatch for dispatching an action
 - subscribe for listening to state changes
- Step 7
 - *connecting a React component with Redux*
 - *Create a new file named src/Book/BookList.js.*



what is a Redux middleware?

- The building blocks of Redux:
- **store**, the traffic policeman. **Reducers**, which makes the state in Redux.
- Then there are actions, plain JavaScript objects, acting as messengers in your application.
- Finally, we have action creators, functions for creating those messages.
- A **Redux middleware** is a function that is able to **intercept, and act accordingly, our actions, before they reach the reducer.**

Json-server



```
Administrator: C:\WINDOWS\system32\cmd.exe - "node" "C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\json-server\lib\cli\bin.js" --watch products.json --port 3004
GET /insuranceProducts 404 5.015 ms - 2
GET /insuranceProducts 404 12.859 ms - 2
^C
F:\sapients2022\globalinsurancereactjs\globalinsuranceapp>cd db
F:\sapients2022\globalinsurancereactjs\globalinsuranceapp\db>json-server --watch products.json --port 3004

\{^_^}/ hi!

Loading products.json
Done

Resources
http://localhost:3004/insuranceProducts

Home
http://localhost:3004

Type s + enter at any time to create a snapshot of the database
Watching...

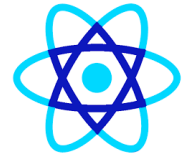
GET /insuranceProducts 200 13.649 ms - -
GET /insuranceProducts 304 30.340 ms - -
GET /insuranceproducts 200 31.847 ms - -
GET /insuranceproducts 304 30.107 ms - -
GET /insuranceproducts 304 28.985 ms - -
GET /insuranceproducts 304 31.366 ms - -
GET /insuranceproducts 304 31.850 ms - -
GET /insuranceproducts 304 14.128 ms - -
```



Asynchronous actions in Redux with Redux Thunk

- `npm i redux-thunk --save-dev`

Docker Reactjs



```
Administrator: Command Prompt
=> => transferring dockerfile: 398B 0.2s
=> [internal] load .dockerignore 0.2s
=> => transferring context: 2B 0.1s
=> [internal] load metadata for docker.io/library/node:alpine 4.8s
=> [1/5] FROM docker.io/library/node:alpine@sha256:c59fb39150e4a7ae14dfd42d3f9874398c7941784b73049c2d274115f00d36c8 10.6s
=> => resolve docker.io/library/node:alpine@sha256:c59fb39150e4a7ae14dfd42d3f9874398c7941784b73049c2d274115f00d36c8 0.0s
=> => sha256:c59fb39150e4a7ae14dfd42d3f9874398c7941784b73049c2d274115f00d36c8 1.43kB / 1.43kB 0.0s
=> => sha256:3a718461938f351292be2cb77f7299cd4e3d8f28f29d548f0cf3c8551dc80e08 1.16kB / 1.16kB 0.0s
=> => sha256:41b5b4b810c1b5c99e59c6e97ab6e405081dce4438ec72015626c5b1c7d335c1 6.44kB / 6.44kB 0.0s
=> => sha256:ca7dd9ec2225f2385955c43b2379305acd51543c28cf1d4e94522b3d94cce3ce 2.81MB / 2.81MB 1.4s
=> => sha256:4487691952c066cb3964b94606825bc96c698377909c7d74c889fd12e24e36a7 47.41MB / 47.41MB 5.4s
=> => sha256:206c50ffab466a0ed68db742d6d2015abcedd0a0b2500babb1938ce2a272b425 2.35MB / 2.35MB 1.6s
=> => extracting sha256:ca7dd9ec2225f2385955c43b2379305acd51543c28cf1d4e94522b3d94cce3ce 1.3s
=> => sha256:f6d4361cf153f2e83958f504356eef6e3d041eb3c4d23da466480ee2dfe656ae 448B / 448B 1.8s
=> => extracting sha256:4487691952c066cb3964b94606825bc96c698377909c7d74c889fd12e24e36a7 4.3s
=> => extracting sha256:206c50ffab466a0ed68db742d6d2015abcedd0a0b2500babb1938ce2a272b425 0.3s
=> => extracting sha256:f6d4361cf153f2e83958f504356eef6e3d041eb3c4d23da466480ee2dfe656ae 0.0s
=> [internal] load build context 549.5s
=> => transferring context: 454.39MB 549.2s
=> [2/5] WORKDIR /react-app 0.9s
=> [3/5] COPY ./package.json /react-app 0.5s
=> [4/5] RUN npm install 93.2s
=> [5/5] COPY . . 11.5s
=> exporting to image 11.9s
=> => exporting layers 11.8s
=> => writing image sha256:8b0f636cfbe216471fdee6a24dec3d53191b49092abb10745b5efcc01d81561 0.0s
=> => naming to docker.io/library/amexapp 0.0s

F:\amexecma\amexstream>
F:\amexecma\amexstream>docker run --name amex-c1 -p 3001:3001 -d amexapp
0e6aa4daecf760a5ba0ee97ccf0e5d8358ba47b729c77b71e8e85d9954755512

F:\amexecma\amexstream>
```




Docker Expressjs

```
F:\amexecma\amexapi>docker build -f dockerfile -t amexapi .
[+] Building 10.7s (10/10) FINISHED
=> [internal] load build definition from dockerfile                                0.1s
=> => transferring dockerfile: 32B                                              0.0s
=> [internal] load .dockerignore                                                0.1s
=> => transferring context: 2B                                                  0.0s
=> [internal] load metadata for docker.io/library/node:16                     2.6s
=> [internal] load build context                                               0.8s
=> => transferring context: 406.95kB                                           0.8s
=> [1/5] FROM docker.io/library/node:16@sha256:68fc9f749931453d5c8545521b021dd97267e0692471ce15bdec0814ed1f8fc3 0.0s
=> CACHED [2/5] WORKDIR /usr/src/app                                           0.0s
=> CACHED [3/5] COPY package*.json ./                                          0.0s
=> CACHED [4/5] RUN npm install                                                 0.0s
=> [5/5] COPY . .                                                              4.3s
=> exporting to image                                                         2.7s
=> => exporting layers                                                         2.7s
=> => writing image sha256:38382b7395add999e0f33ae464152758496e6b8d2198d60b721b26cfaa448891 0.0s
=> => naming to docker.io/library/amexapi                                     0.0s

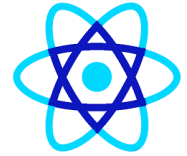
F:\amexecma\amexapi>docker run --name amexapi-c1 -p 4000:4000 --link amex-mongo:mongo -d amexapi
fa396f299f1225e1755c563d91d1c8626f5145ef9e28a2e77d1ee3f2c88ba04b

F:\amexecma\amexapi>
```



React Native

- React Native is like React, but it uses native components instead of web components as building blocks.



- **React** – This is a Framework for building web and mobile apps using JavaScript.
- **Native** – You can use native components controlled by JavaScript.
- **Platforms** – React Native supports IOS and Android platform.



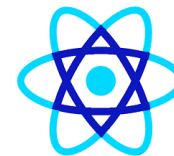
Advantages

- JavaScript – You can use the existing JavaScript knowledge to build native mobile apps.
- Code sharing – You can share most of your code on different platforms.
- Community – The community around React and React Native is large, and you will be able to find any answer you need.



Limitations

- Native Components – If you want to create native functionality which is not created yet, you will need to write some platform specific code.



Installation

- `npm install -g create-react-native-app`
- `create-react-native-app MyReactNative`
- `npm install -g react-native-cli`
- `react-native init rncreate`



Emulators

- Go to project folder
- Open in ios
- `react-native run-ios`
- `react-native run-android`
- `react-native start` (by mistake if you close window)



Bundling

- (in project directory) `mkdir android/app/src/main/assets`
- `react-native bundle --platform android --dev false --entry-file index.js --bundle-output android/app/src/main/assets/index.android.bundle --assets-dest android/app/src/main/res`
- `react-native run-android`



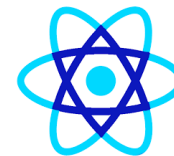
Enzymes

- Enzyme is a JavaScript Testing utility for React that makes it easier to assert, manipulate, and traverse your React Components' output.



Installation

- `npm i --save-dev enzyme enzyme-adapter-react-16`
- `npm install --save enzyme enzyme-adapter-react-16 react-test-renderer`
- `Npm install --save-dev chai`
- `npm install --save jest-enzyme`
- `Npm install --save sinon`
- `Npm test`
- **`"jest": "^27.4.5",`**
- **`"@wojtekmaj/enzyme-adapter-react-17": "^0.6.6"`**



Jest Enzyme Configuration

- Go to `setuptest.js`
- `import '@testing-library/jest-dom';`
- `import Enzyme from 'enzyme';`
- `import Adapter from '@wojtekmaj/enzyme-adapter-react-17';`
- `Enzyme.configure({ adapter: new Adapter() });`



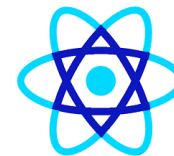
Jest Unit Testing

- **Jest** is a **JavaScript testing framework** used for:
 - Unit testing
 - Integration testing
 - Snapshot testing
- It's **fast**, **zero-config**, and works very well with **React**, **Node.js**, and **TypeScript**.



★ Core Features of Jest

- **❶ Zero-Configuration (Out of the Box)**
- Works immediately for most projects
- Auto-detects:
 - `__tests__` folders
 - `*.test.js`, `*.spec.js`
- Built-in assertions, mocks, spies
- ✓ Ideal for quick setup



★ Core Features of Jest

- **② Fast Test Execution**
- Runs tests **in parallel**
- Uses worker processes
- Smart test scheduling
- ✓ Faster feedback during development



★ Core Features of Jest

- **3 Built-in Assertion Library**
- No extra library needed.
- `expect(5 + 5).toBe(10);`
- `expect(user).toEqual({ name: "John" });`
- Common matchers:
 - `toBe`
 - `toEqual`
 - `toContain`
 - `toHaveLength`
 - `toBeTruthy` / `toBeFalsy`



★ Core Features of Jest

- **4** Snapshot Testing
- Detects unexpected UI changes.
- `expect(tree).toMatchSnapshot();`
- ✓ Very popular for React components



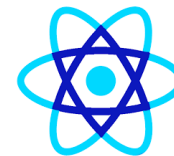
★ Core Features of Jest

- **4** Snapshot Testing
- Detects unexpected UI changes.
- `expect(tree).toMatchSnapshot();`
- ✓ Very popular for React components



★ Core Features of Jest

- **5** Powerful Mocking System
- Mock:
 - Functions
 - Modules
 - APIs
 - Timers
- `const fn = jest.fn();`
- `fn("hello");`
- `expect(fn).toHaveBeenCalled();`
- ✓ No external mocking library required



★ Core Features of Jest

- **6 Fake Timers (Time Control)**
- Test `setTimeout`, `setInterval`, animations.
- `jest.useFakeTimers();`
- `jest.advanceTimersByTime(1000);`
- ✓ Essential for async logic



★ Core Features of Jest

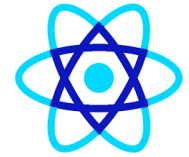
- **7 Code Coverage Built-In** 
- Generate coverage reports easily.
- `jest --coverage`
- Metrics:
 - Statements
 - Branches
 - Functions
 - Lines
 - ✓ HTML + text reports



★ Core Features of Jest

- **8 Test Lifecycle Hooks**

- Run setup & cleanup logic.
- `beforeAll(() => {});`
- `beforeEach(() => {});`
- `afterEach(() => {});`
- `afterAll(() => {});`
- ✓ Helps isolate tests



- **📌 Async Testing Support**
 - Supports:
 - Promises
 - `async/await`
 - Callbacks
 - `test("fetch data", async () => {`
 - `const data = await fetchData();`
 - `expect(data).toBeDefined();`
 - `});`



★ Core Features of Jest

- **10 Watch Mode** 🧐
 - Automatically reruns tests on file changes.
 - `jest --watch`
 - ✓ Improves developer productivity



★ Core Features of Jest

- **1 1 Module Mocking**
- Mock entire files or libraries.
- `jest.mock("./api");`
- ✓ Useful for isolating business logic



★ Core Features of Jest

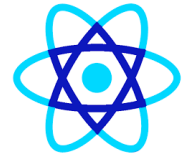
- **1 2 Browser-Like Environment (jsdom)**
- DOM APIs available:
 - document
 - window
 - localStorage
- ✓ Perfect for React testing



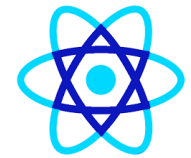
★ Core Features of Jest

- **1 3 Custom Reporters**
- Integrates with CI/CD tools.
- JUnit
- HTML reports
- Coverage dashboards
- ✓ Enterprise-friendly

Summary Table



Feature	Available in Jest
Unit Testing	✓
Snapshot Testing	✓
Mocking	✓
Code Coverage	✓
Async Testing	✓
Watch Mode	✓
React Support	✓
CI/CD Integration	✓



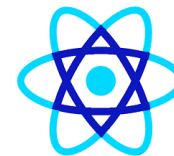
Unit Test Libraries Installation

- **1** Install Testing Libraries (Vite + React)
 - `npm i -D jest @testing-library/react @testing-library/jest-dom @testing-library/user-event`
 - `npm i -D vitest jsdom`
- ✓ Installs Jest test runner
- ✓ Installs jsdom (browser environment for React tests)
- ✓ Installs React Testing Library
- ✓ Installs extra DOM matchers
- ✓ Installs user interaction helpers (click, type, etc.)



Mount, Shallow, Render

- Mounting
 - Full DOM rendering including child components
 - Ideal for use cases where you have components that may interact with DOM API, or use React lifecycle methods in order to fully test the component
 - As it actually mounts the component in the DOM `.unmount()` should be called after each tests to stop tests affecting each other
 - Allows access to both props directly passed into the root component (including default props) and props passed into child components



Mount, Shallow, Render

- Shallow
 - Renders only the single component, not including its children.
 - This is useful to isolate the component for pure unit testing.
 - It protects against changes or bugs in a child component altering the behaviour or output of the component under test
 - As of Enzyme 3 shallow components do have access to lifecycle methods by default
 - Cannot access props passed into the root component (therefore also not default props), but can those passed into child components, and can test the effect of props passed into the root component.



Mount, Shallow, Render

- Render
 - Renders to static HTML, including children
 - Does not have access to React lifecycle methods
 - Less costly than mount but provides less functionality



Selecting Elements

- `getBy\*`
- `getByAll\*`
- `queryBy\*`
- `queryAllBy\*`
- `findBy\*`
- `findAllBy\*`



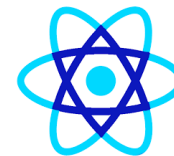
Selecting Elements

- `getByText`
- `getByRole`
- `getByLabelText`
- `getByPlaceholderText`
- `getByAltText`
- `getByDisplayValue`



Selecting Elements

- `queryByText`
- `queryByRole`
- `queryByLabelText`
- `queryByPlaceholderText`
- `queryByAltText`
- `queryByDisplayValue`



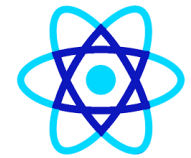
Assertive Elements

- `toBeDisabled`
- `toBeEnabled`
- `toBeEmpty`
- `toBeEmptyDOMElement`
- `toBeInTheDocument`
- `toBeInvalid`
- `toBeRequired`
- `toBeValid`
- `toBeVisible`
- `toContainElement`
- `toContainHTML`
- `toHaveAttribute`



Assertive Elements

- `toHaveClass`
- `toHaveFocus`
- `toHaveFormValues`
- `toHaveStyle`
- `toHaveTextContent`
- `toHaveValue`
- `toHaveDisplayValue`
- `toBeChecked`
- `toBePartiallyChecked`
- `toHaveDescription`

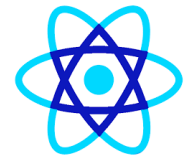


What is Code Coverage?

- **Code Coverage** shows how much of your source code is executed by tests.
- It answers:
 - Which lines ran?
 - Which branches were tested?
 - Which functions are unused?



Coverage Metrics in Jest



Metric	Meaning
Statements	Lines of code executed
Branches	if / else / switch paths
Functions	Functions called
Lines	Executed source lines

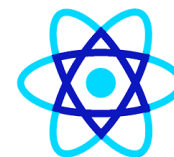
1 Generate Test Coverage Report (HTML – most common)



-
-  **Install coverage plugin**
 - `npm i -D @vitest/coverage-v8`



Update vite.config.js



```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

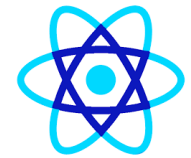
export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: "./src/setupTests.js",

    coverage: {
      provider: "v8",
      reporter: ["text", "html", "lcov"], // ✅ reports
      reportsDirectory: "./coverage",
    },
  },
});
```



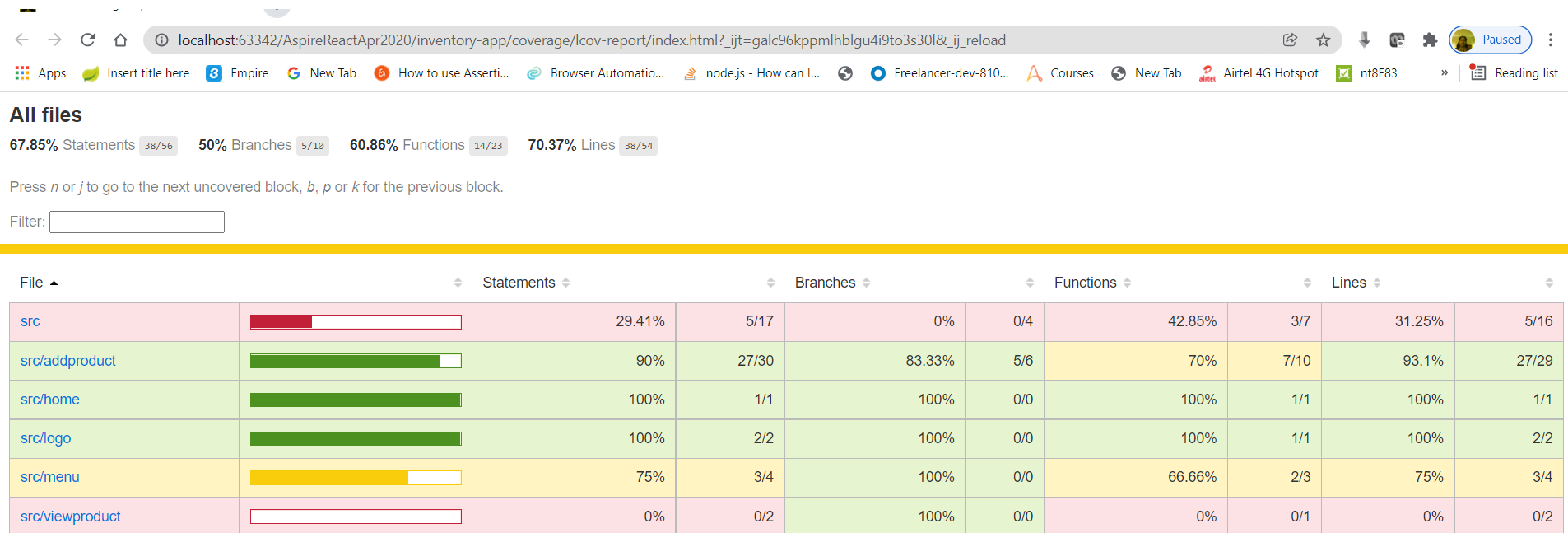

Run tests with coverage

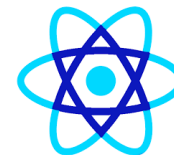
- `npx vitest run --coverage`



React Coverage Report

- `npm run test -- --coverage --watchAll=false`





Parcel Bundler

Benchmarks

Based on a reasonably sized app, containing 1726 modules, 6.5M uncompressed. Built on a 2016 MacBook Pro with 4 physical CPUs.

Bundler	Time
browserify	22.98s
webpack	20.71s
parcel	9.98s
parcel - with cache	2.64s



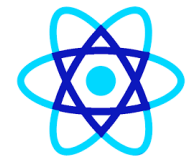
Parcel Bundler

- Assets Bundling - Parcel has out of the box support for JS, CSS, HTML, file assets.
- Automatic transforms - All your code are automatically transformed using Babel, PostCSS, and PostHTML.
- Code Splitting - Parcel allows you to split your output bundle by using the dynamic `import()` syntax.
- Hot module replacement (HMR) - Parcel automatically updates modules in the browser as you make changes during development, no configuration needed.
- Error Logging - Parcel prints syntax highlighted code frames when it encounters errors to help you pinpoint the problem.

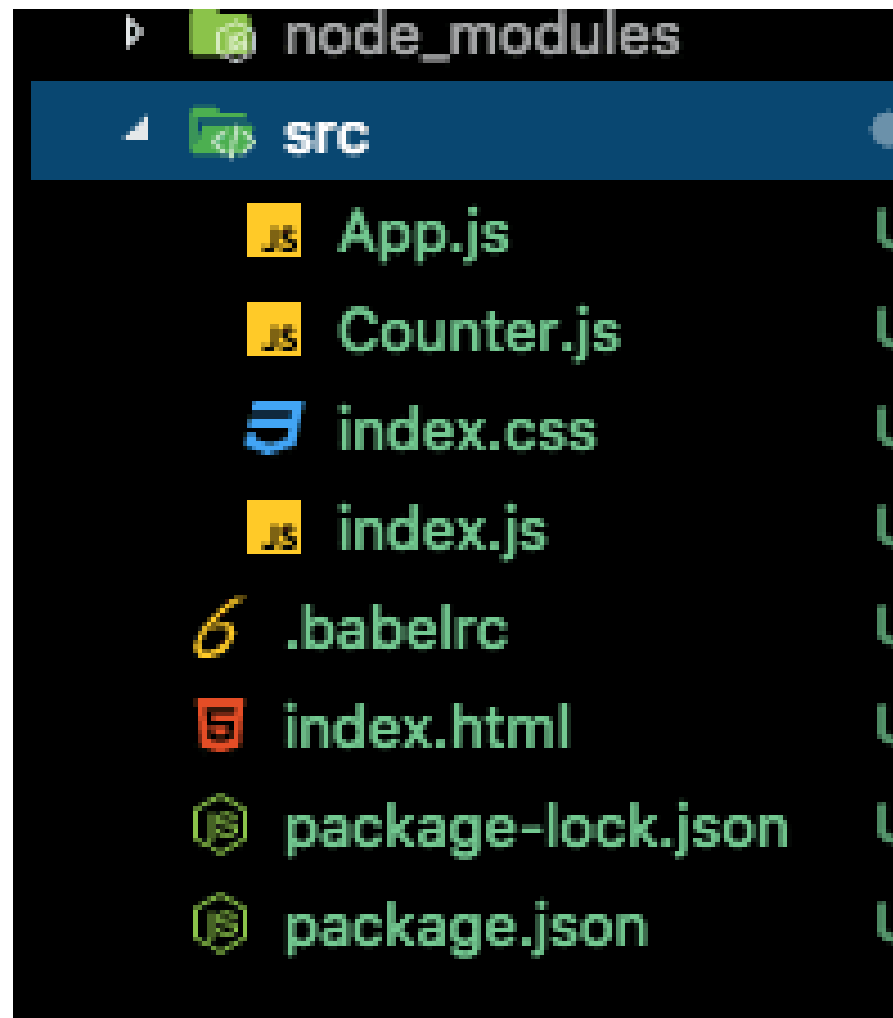


Parcel Bundler

- `mkdir react-parcel`
- `cd react-parcel`
- `npm init -y`
- `npm i --save-dev parcel-bundler`
- `npm i --save react react-dom`



Parcel Bundler

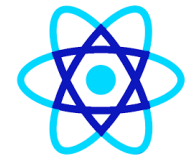




E2e testing

- `npm install nightwatch`
- `npm install chromedriver --save-dev`
- `npm install geckodriver --save-dev`
- Keep web driver in current folder

React Production Deployment



```
File Explorer: fiservpune2019 [F:\fiservpune2019] - WebStorm
Administrator: Node.js command prompt - serve -s build

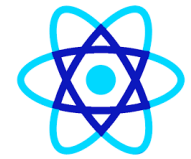
F:\fiservpune2019\bankingapp>npm install netlify-cli -g
npm WARN deprecated flatten@1.0.2: I wrote this module a very long time ago; you should use something else.
C:\Users\Balasubramaniam\AppData\Roaming\npm\ntl -> C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\netlify-cli\bin\run
C:\Users\Balasubramaniam\AppData\Roaming\npm\netlify -> C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\netlify-cli\bin\run

> core-js@2.6.10 postinstall C:\Users\Balasubramaniam\AppData\Roaming\npm\node_modules\netlify-cli\node_modules\core-js
> node postinstall || echo "ignore"

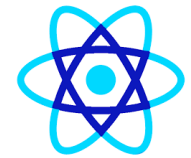
Thank you for using core-js ( https://github.com/zloirock/core-js ) for polyfilling JavaScript standard library!

The project needs your help! Please consider supporting of core-js on Open Collective or Patreon:
> https://opencollective.com/core-js
> https://www.patreon.com/zloirock
```


React Production Deployment



```
Reactjs_v1.pptx - PowerPoint
Administrator: Node.js command prompt - serve -s build
es/netlify-cli/node_modules/node-fetch/lib/index.js:1455:11)
F:\fiservpune2019\bankingapp>netlify deploy
Please provide a publish directory (e.g. "public" or "dist" or "."):
F:\fiservpune2019\bankingapp
? Publish directory F:\fiservpune2019\bankingapp\build
Deploy path: F:\fiservpune2019\bankingapp\build
Deploying to draft URL...
✓ Finished hashing 17 files
✓ CDN requesting 0 files
✓ Finished uploading 0 assets
✓ Draft deploy is live!
Logs: https://app.netlify.com/sites/banking-app-veb/deployments/5da743c6f4ee609bdb2b7deb
Live Draft URL: https://5da743c6f4ee609bdb2b7deb--banking-app-veb.netlify.com
If everything looks good on your draft URL, take it live with the --prod flag.
netlify deploy --prod
```



React Production Deployment

Administrator: Node.js command prompt - netlify deploy -d .

Site ID: 7f57284a-725d-487d-98b1-7c26e68c6248

Deploy path: G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare\netlify\build

Deploying to draft URL...

✓ Finished hashing 0 files

✓ CDN requesting 0 files

✓ Finished uploading 0 assets

/ Waiting for deploy to go live...Terminate batch job (Y/N)? y

G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare>netlify unlink

Folder is not linked to a Netlify site. Run 'netlify link' to link it

G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare>netlify deploy -d .

This folder isn't linked to a site yet

? What would you like to do? + Create & configure a new site

? Team: eswaribala's team

Choose a unique site name (e.g. isnt-eswaribala-awesome.netlify.com) or leave it blank for a random name. You can update the site name later.

? Site name (optional): vebdentalcare

» Warning: vebdentalcare.netlify.com already exists. Please try a different slug.

Choose a unique site name (e.g. super-cool-site-by-eswaribala.netlify.com) or leave it blank for a random name. You can update the site name later.

? Site name (optional): vebdentalcare

Site Created

Admin URL: https://app.netlify.com/sites/vebdentalcare

URL: https://vebdentalcare.netlify.app

Site ID: c4ced4d0-96ce-43b5-a821-c4f09c11973b

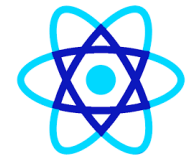
Deploy path: G:\Local disk\vigneshpics\vebapplication\vebdentalcare\dist\vebdentalcare

Deploying to draft URL...

✓ Finished hashing 482 files

✓ CDN requesting 369 files

\ (466/467) Uploading 3rdpartylicenses.txt...



React Production Deployment

Yarn global add serve

```
Administrator: Node.js command prompt - serve -s build
F:\fiservpune2019\react-voting-app-master>serve -s build

Serving!

- Local: http://localhost:5000
- On Your Network: http://172.29.53.177:5000

Copied local address to clipboard!
```



React Production Layout

```
UI External: http://localhost:3003
-----
[Browsersync] Serving files from: ./
[Browsersync] Watching files...
19.11.12 15:05:22 404 GET /index.html
^CTerminate batch job (Y/N)? y

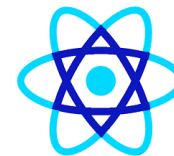
F:\daimlerreactoct2019\blog2019>lite-server --baseDir=build/
Did not detect a `bs-config.json` or `bs-config.js` override file. Using lite-server defaults...
** browser-sync config **
{
  injectChanges: false,
  files: [ './**/*.html,css,js' ],
  watchOptions: { ignored: 'node_modules' },
  server: { baseDir: 'build/', middleware: [ [Function], [Function] ] }
}

[Browsersync] Access URLs:
-----
    Local: http://localhost:3002
    External: http://172.29.53.177:3002
-----
    UI: http://localhost:3003
UI External: http://localhost:3003
-----
[Browsersync] Serving files from: build/
[Browsersync] Watching files...
19.11.12 15:05:53 200 GET /index.html
```



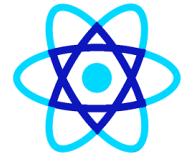
React Optimization

- **Using Immutable Data Structures**
- **Function/Stateless Components and `React.PureComponent`**
- **Multiple Chunk Files**
- **Using Production Mode Flag in Webpack**
- **Dependency optimization**
- **Use `React.Fragments` to Avoid Additional HTML Element Wrappers**

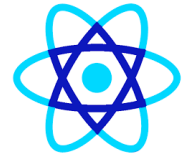


React Optimization

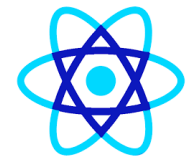
- **Avoid Inline Function Definition in the Render Function.**
- Throttling and Debouncing Event Action in JavaScript
- **Spreading props on DOM elements**
- **CSS Animations Instead of JS Animations**



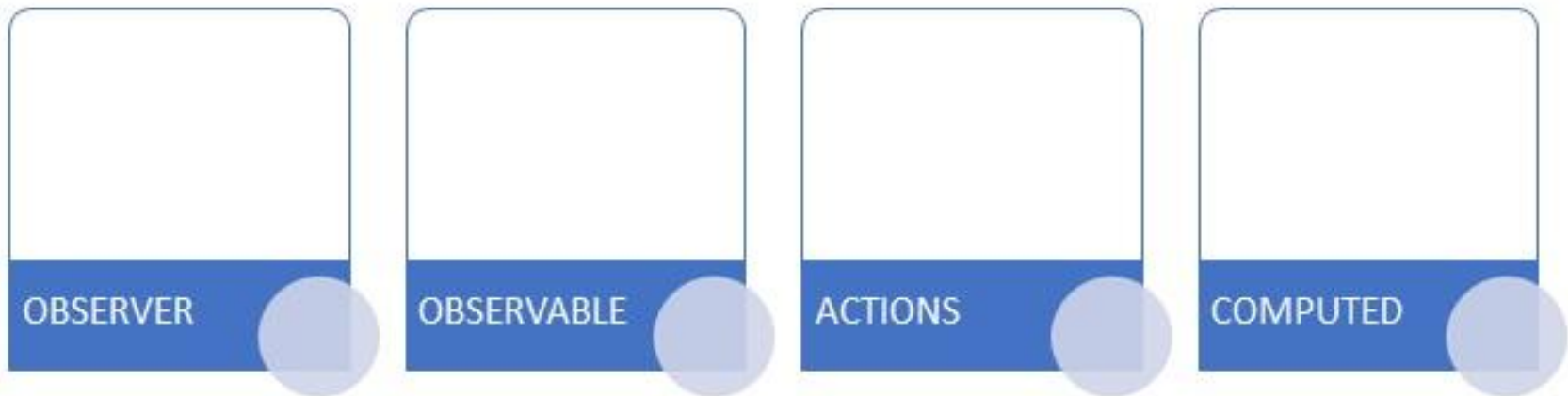
- MobX is a library that aims to make state management simple and scalable by transparently applying functional reactive programming (TFRP).
- According to the readme, the approach of MobX is thus: “Anything that can be derived from the application state, should be derived Automatically.”
- This includes the UI, data serialization, server communication, etc.



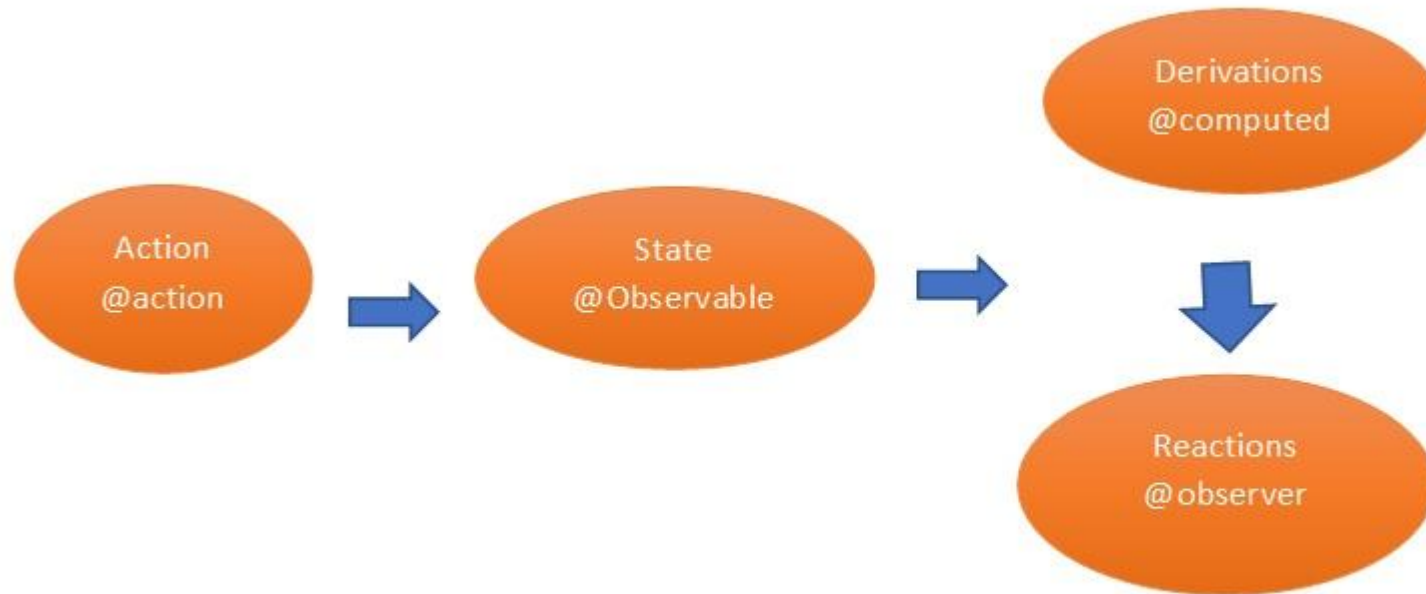
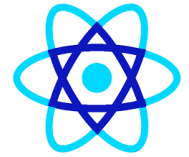
- In addition, some of the core principles of MobX include:
 - May have multiple stores to store the state of the application
 - The state of the entire application is stored in a single object tree
 - Action in any piece of code can change the state of the application
 - All derivations from the state are automatically and atomically updated when the state is changed

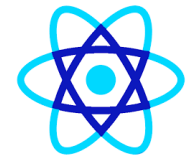


MOB X Core Concepts



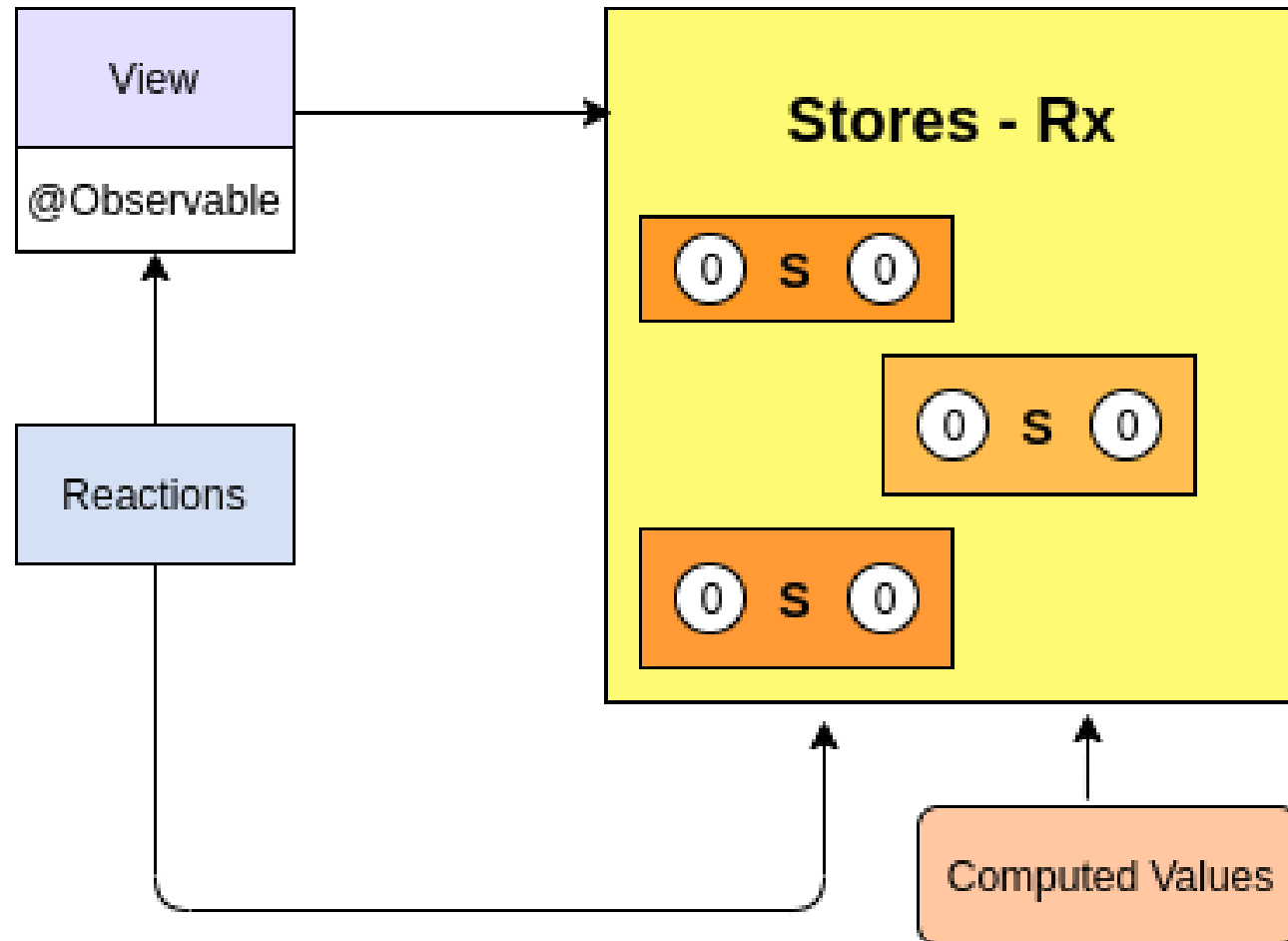
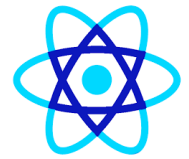
MOB X Core Concepts

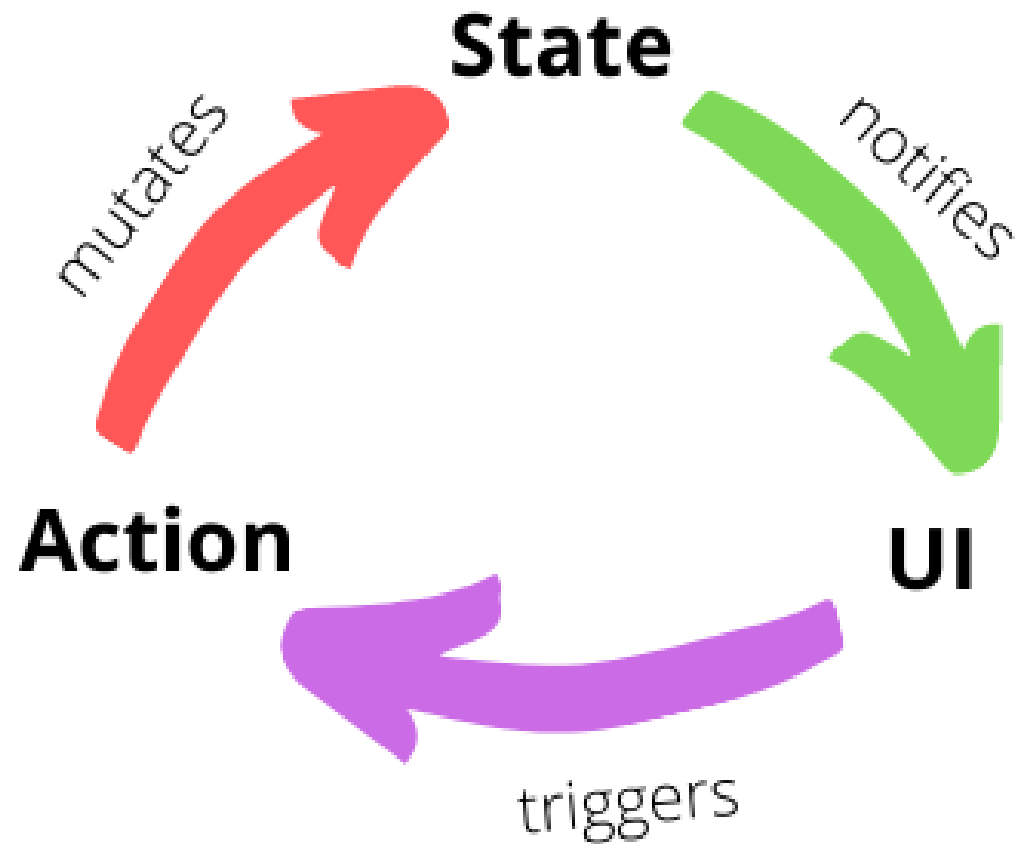
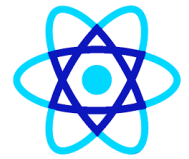


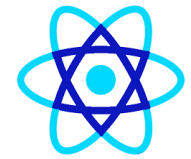


MOB X Core Concepts

- **@Observable**
 - Mobx listens through observables to identify when the values is changed. It can array objects and with @Observable key words we can decorate the object.
- **@Action**
 - Action is a method or function that is used to modify the state of a data. We can decorate with @Action keycard to declare as it is a Mob X Action method.
- **@Inject**
 - To use any store value we have to inject the store using @inject.
- **@Observer**
 - Any component that needs to update with state will need to be decorated with @observer.



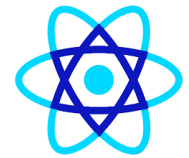




React MobX steps

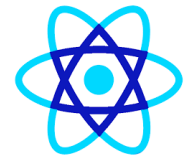
- 1. `npx create-react-app bankingapp`
- 2. `npm install mobx --save`
- 3. `npm install mobx-react --save`
- 4. `npm install react-router-dom --save`
- To install semantic UI(similar to bootstrap) please use the below command,
- `npm install semantic-ui-react --save`
- `npm install semantic-ui-css --save`

Redux vs MobX for React state management



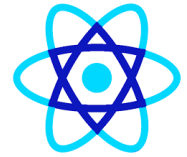
- Data store – single store vs. multiple stores
 - Redux – Single store
 - In Redux, there is only one store.
 - It serves as the single source of truth containing normalized data.
 - Redux state is immutable and for each new state, an ancestor state is cloned
 - MobX – Multiple stores
 - Unlike Redux, MobX usually maintains at least two stores – one for the UI state and one (or more) for the domain state, and they contain denormalized data.
 - The advantage of multiple stores is in the ability to reuse and query the domain state universally, including other applications.
 - All the while, the UI store would remain specific to the current application.

Redux vs MobX for React state management



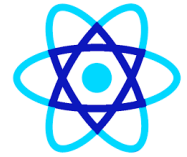
- Store data structure
 - Redux – JavaScript objects
 - Redux uses simple JavaScript objects as the data structure to store the state.
 - This requires that updates be tracked manually.
 - Which can add quite a bit of overhead when it comes to applications with complex states to manage and maintain.
 - Redux vs MobX in React
 - MobX uses observable (or noticeable) data to automatically track changes through subscriptions.
 - Quite clearly, automation makes for an easier life for a developer.
 - So it's no wonder many find MobX to be an obvious winner in this category. It's simply more comfortable to use.

Redux vs MobX for React state management



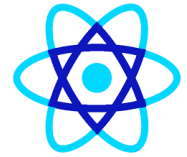
- Purity
- Redux – Pure
 - We've already established that Redux uses a single, immutable source of truth for the states stored.
 - This means that states are all read-only, and reducers can overwrite a state invoked by an action.
 - Reducers are pure functions, as they receive a state and action and return a new state.

Redux vs MobX for React state management



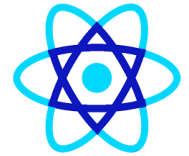
- Purity
 - MobX – Impure
 - On the flip side there's MobX that allows for states to be easily updated and overwritten with new values.
 - While it may be easy to implement, testing and maintaining can become a nightmare of unpredictable outputs.

Redux vs MobX for React state management



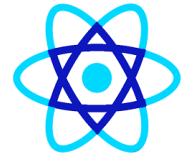
- Boilerplate code
 - Redux – Explicit
 - Perhaps one of the main shortcomings of Redux is the sheer volume of boilerplate code it brings.
 - This is especially true when it comes to React applications.
 - MobX – Implicit
 - Since MobX is a lot more implicit in nature, it packs a lot less boilerplate.
 - In this category, MobX is a clear winner.

Redux vs MobX for React state management



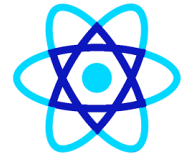
- Scalability
 - The purity and somewhat rigid approach of Redux is an advantage when it comes to scalability and debugging of applications.
 - The predictability of pure functions makes Redux much easier to test, maintain and scale than MobX.

Redux vs MobX for React state management

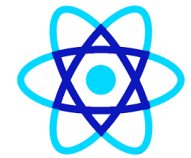


- Debugging
 - Redux – Predictable
 - As mentioned above, Redux is a lot more predictable than MobX as it comes with a lot less abstraction.
 - Add to that a superior set of developer tools (including time traveling) and you got yourself code that is a breeze to debug.
- MobX – Abstract
 - Unlike Redux, MobX relies a lot more on abstraction, which can produce unpredictable results and generally make debugging difficult.
 - Moreover, the lack of efficient tools for MobX debugging and testing add another hurdle for developers considering MobX for their state management needs.

Redux vs MobX for React state management

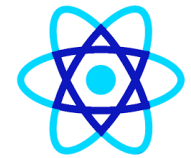


- Learning curve
 - Redux – Steep
 - MobX – Moderate



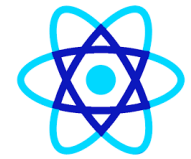
Micro Front End

- Micro Frontend is an architectural style for frontend web development that extends the concepts of microservices to the frontend world.
- It involves breaking up a large, monolithic frontend application into smaller, more manageable pieces that can be developed, deployed, and maintained independently by different teams.
- Each team is responsible for a specific feature or section of the UI, and these parts are integrated together to form a cohesive application.



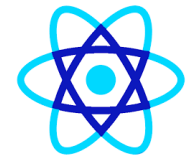
Micro Front End

- Here are key characteristics of Micro Frontends:
- Independence:
 - Each micro frontend is built and deployed independently.
 - Teams can work autonomously without worrying about interfering with others.
- Technology Agnostic:
 - Micro frontends can be built using different frameworks or technologies, allowing each team to choose the tools that best suit their needs (e.g., React, Angular, Vue.js, etc.).



Micro Front End

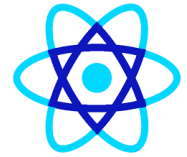
- Here are key characteristics of Micro Frontends:
- Isolation:
 - Each micro frontend operates in isolation from others.
 - They are designed to prevent one part of the application from breaking the entire system.
- Ownership:
 - Teams have full ownership of their micro frontends, including development, testing, and deployment.



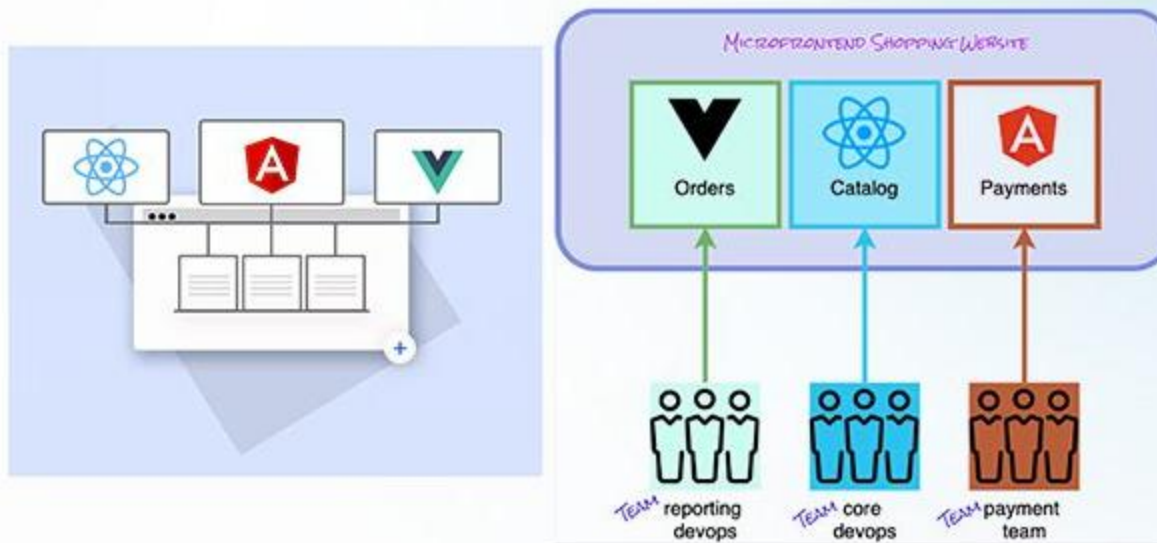
Micro Front End

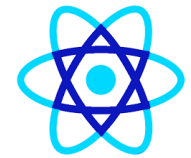
- Here are key characteristics of Micro Frontends:
- Composition:
 - In the final application, micro frontends are composed into a single interface.
 - The composition can happen at different stages, such as server-side or client-side integration.
- Independent Deployment:
 - Micro frontends can be updated, deployed, or rolled back independently without affecting the entire application.

Micro Front Ends



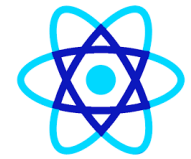
Microfrontend





Micro Front Ends

- Basically, a microservice that runs inside a browser is known as a micro frontend.
- The frontend of the program is divided into separate, loosely linked “micro apps” that can be developed, tested, and deployed independently.
- This is an architectural and organizational style like how the backend is divided into several services in the world of microservices.



Micro Front Ends

What are microfrontends?

Divide a *monolithic* app into multiple, smaller apps

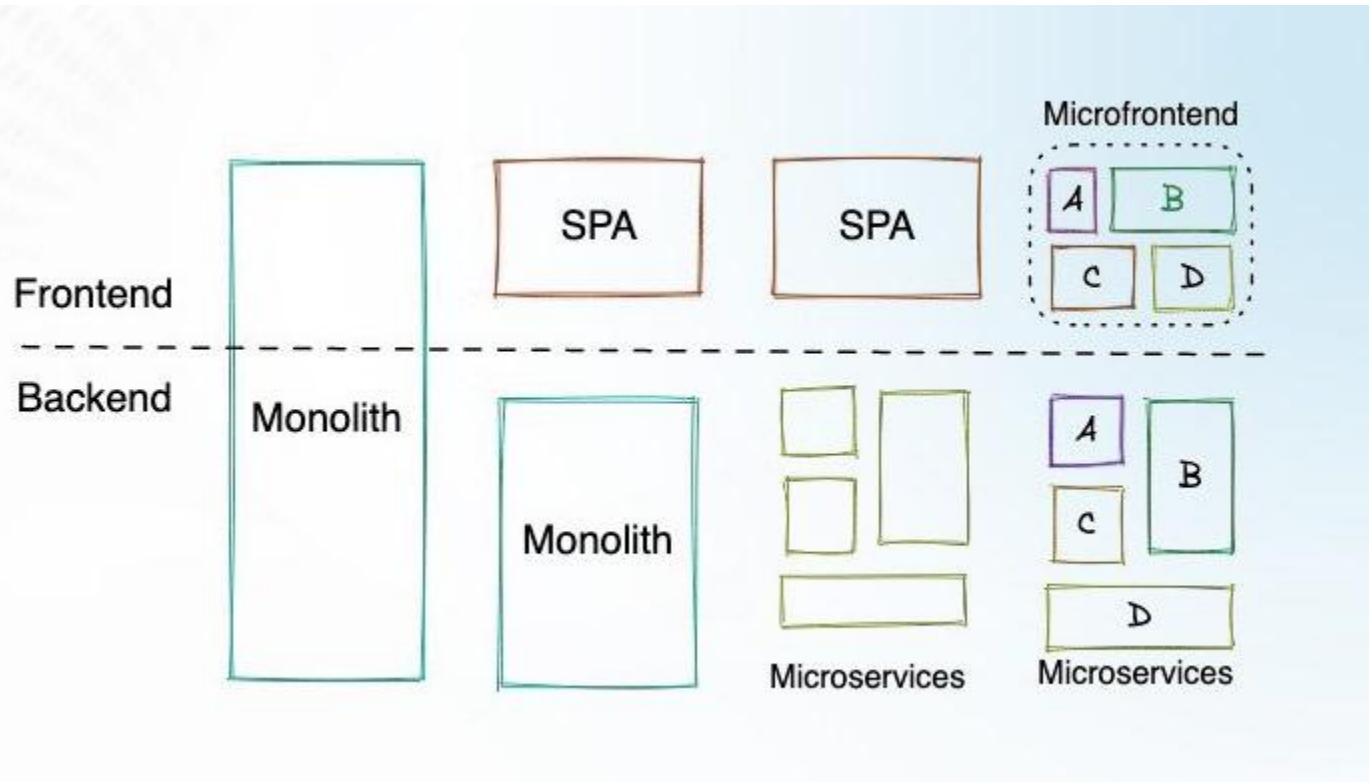
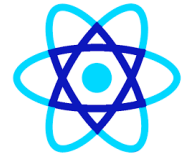
Each smaller app is responsible for a distinct feature of the product

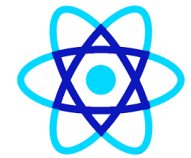
Why use them?

Multiple engineering teams can work in isolation

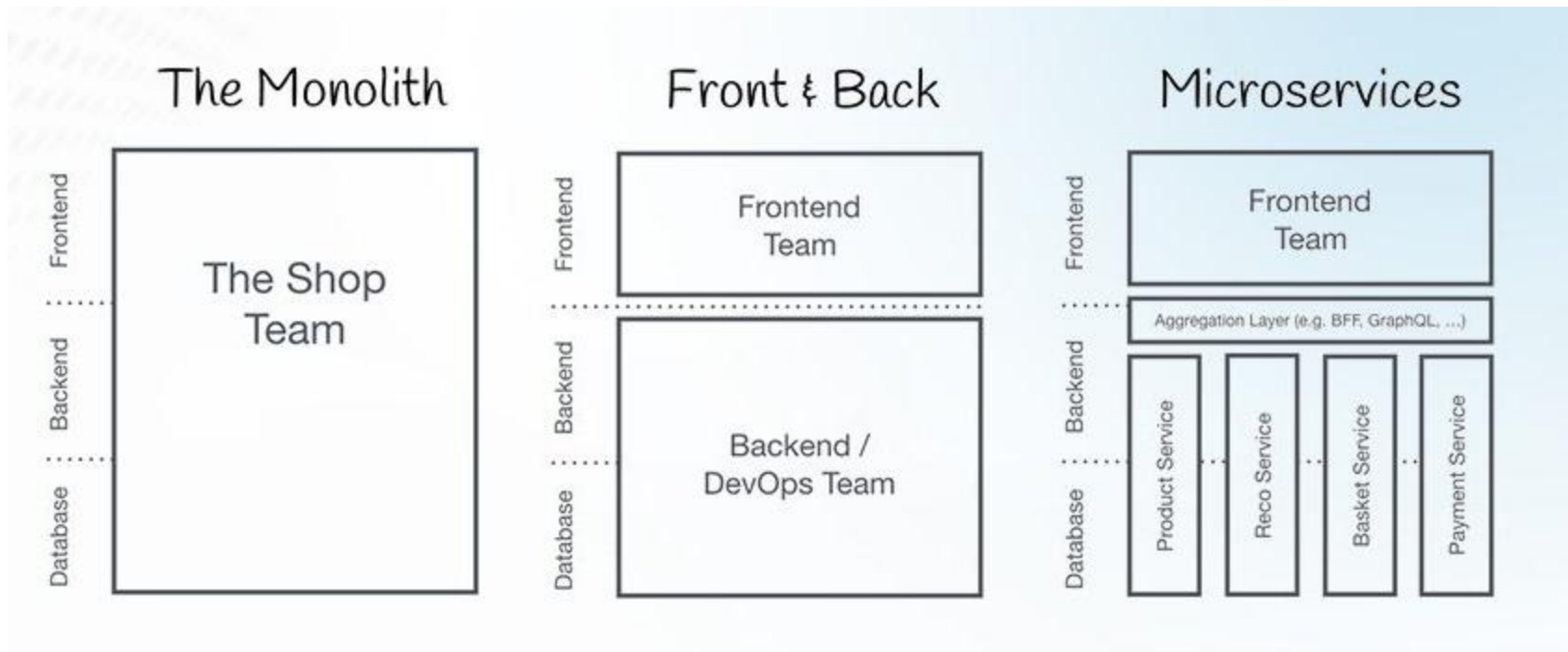
Each smaller app is easier to understand and make changes to

Micro Front Ends

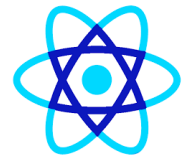




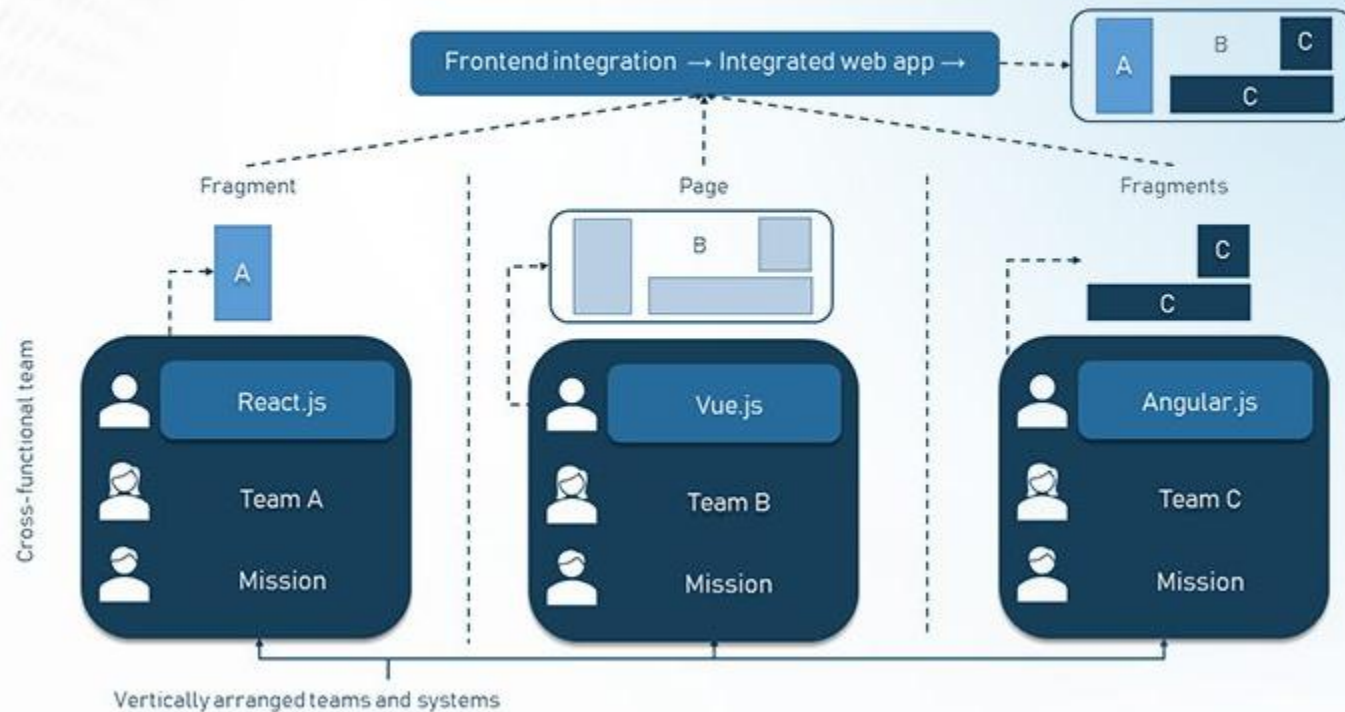
Different Architectural Approaches

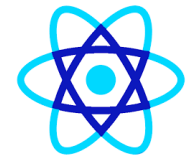


Different Architectural Approaches



Micro-frontend Architecture





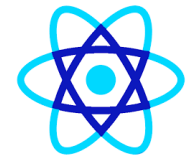
Common Implementation Approaches

- iFrame-based:
 - Embedding each micro frontend in an iFrame, which isolates it from others, though this method can lead to poor user experience due to performance and styling issues.
- JavaScript-based Integration:
 - Loading different JavaScript bundles at runtime and dynamically injecting them into the application.
 - This provides more flexibility but requires careful management of shared resources.



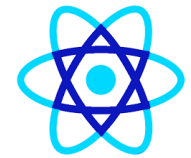
Common Implementation Approaches

- Web Components:
 - Using native browser APIs like Custom Elements and Shadow DOM to encapsulate each micro frontend, which allows better isolation and integration with minimal overhead.
- Module Federation (Webpack 5):
 - A newer technique that allows micro frontends to share code (like libraries) and dynamically load pieces of an application, reducing redundancy and improving performance.



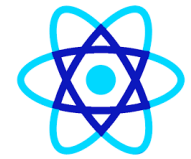
MFE Advantages

- Scalability:
 - Each micro frontend can scale independently.
- Improved Collaboration:
 - Teams can work in parallel without dependency bottlenecks.
- Flexibility:
 - Different technologies and versions of libraries can coexist within the same application.
- Easier Maintenance:
 - Since the codebase is smaller and isolated, it's easier to maintain and refactor.



MFE Challenge

- Consistency:
 - Maintaining a consistent design and user experience across independently developed micro frontends can be difficult.
- Performance:
 - The overhead of integrating and loading multiple independent frontend pieces can affect performance.
- Complexity:
 - Managing multiple repositories, deployments, and interactions between micro frontends adds complexity.

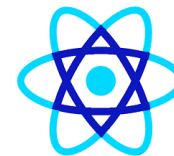


Types of Micro front ends

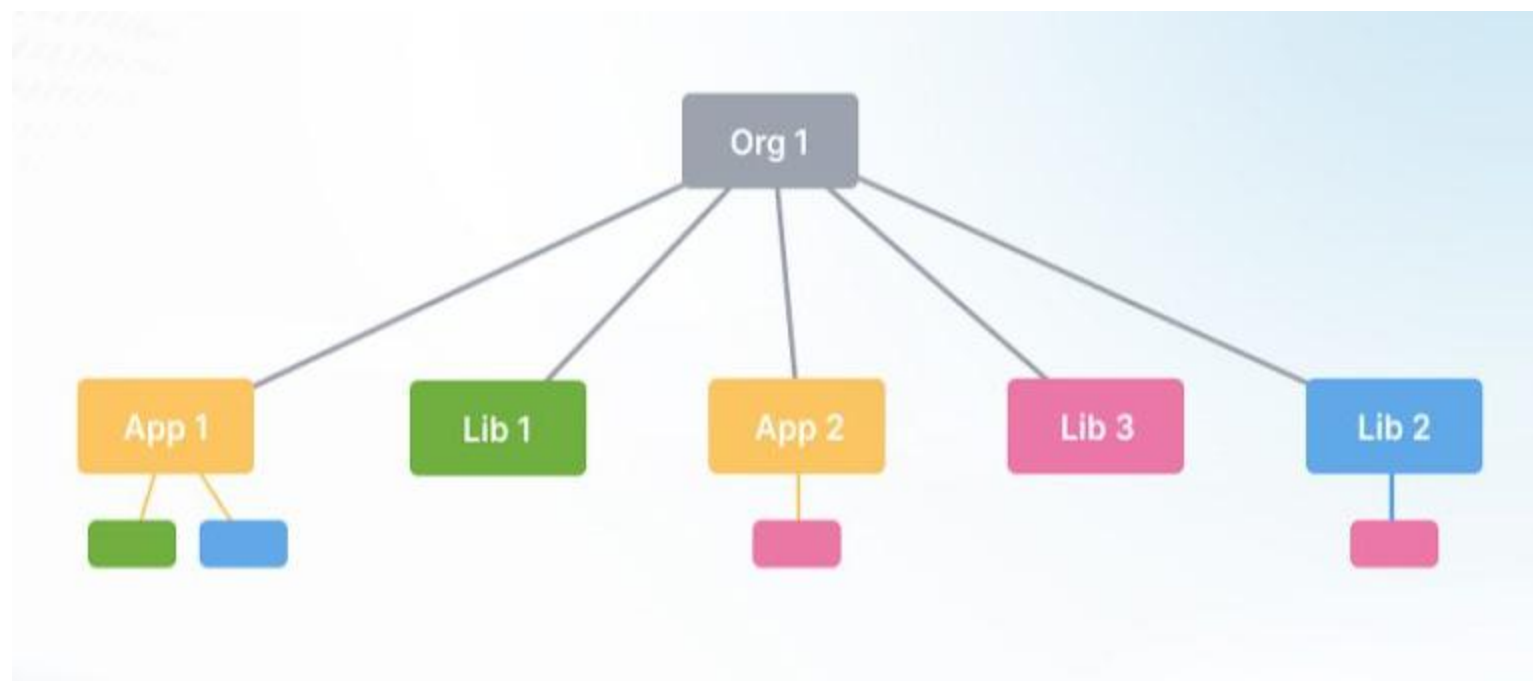
3 Types of Microfrontends

- ✓ Monorepository
- ✓ Multirepository
- ✓ Metarepository

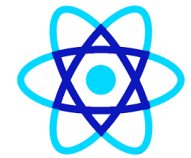




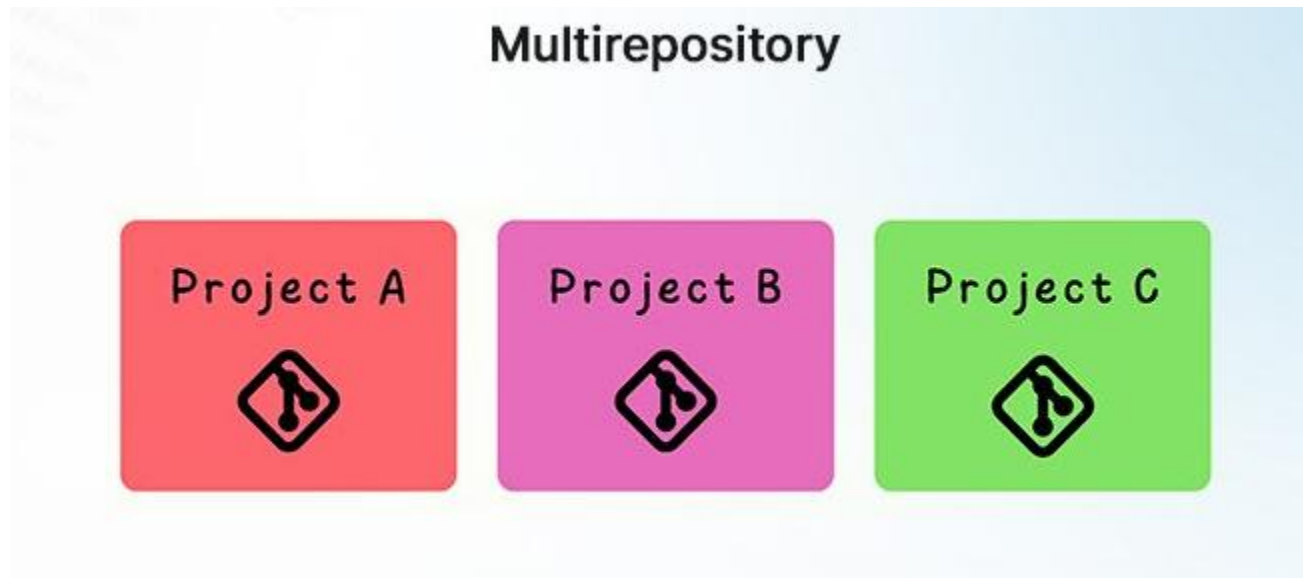
Mono Repository



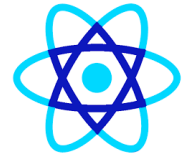
- **Example:** Imagine building an e-commerce website with multiple micro front ends, including product listings, a shopping cart, and a user account.
- **Instead of maintaining separate repositories for each micro frontend, you keep them all in a single mono repository.**



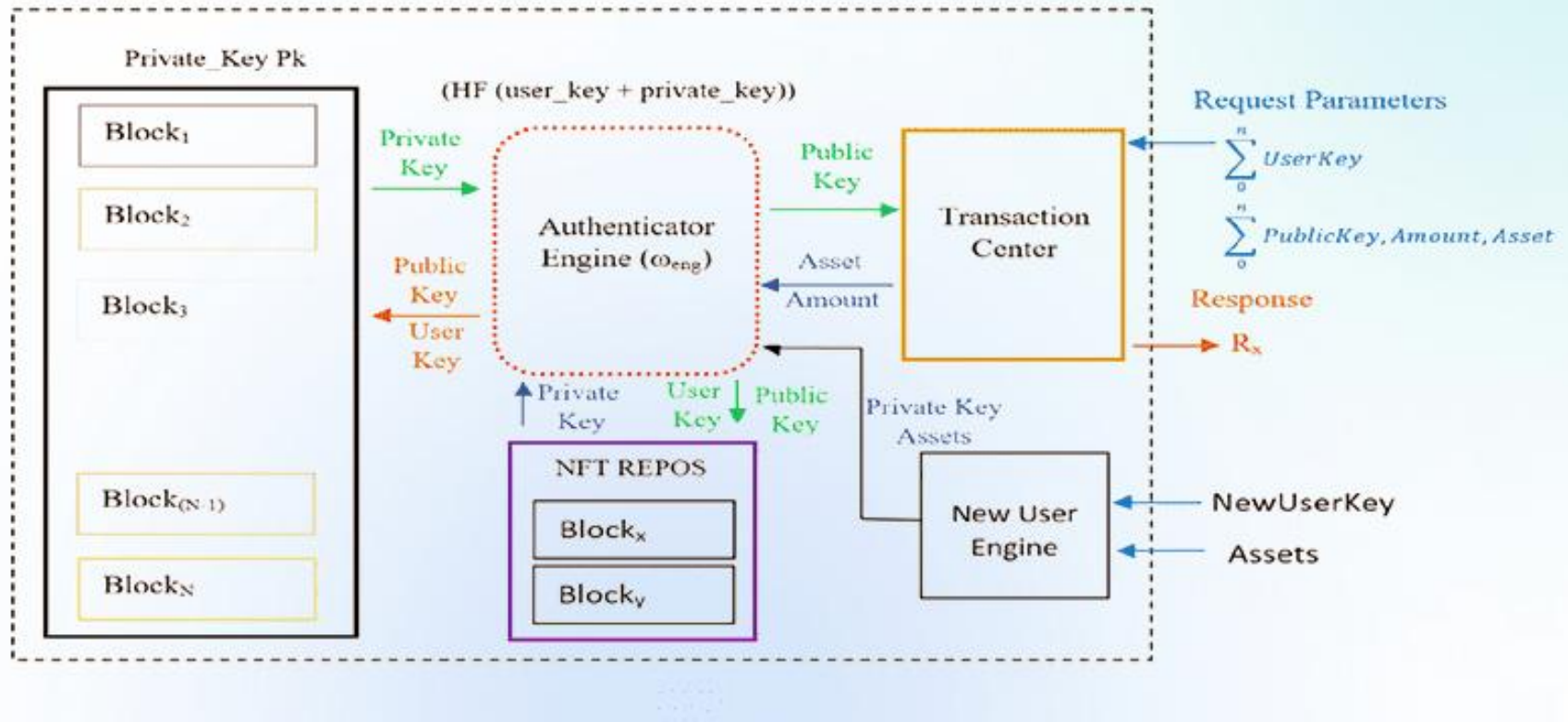
Multi Repository



- **Example: Consider an e-commerce website with micro frontends for product listings, shopping carts, and user accounts.**



The working architecture of the MetaRepo system



Example: Imagine building a content management system (CMS) with various microfrontends, including a blog editor, user dashboard, and analytics dashboard.



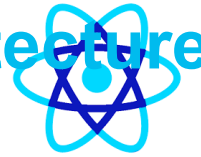
Why Should We Use Micro frontend Architecture?

Top Benefits of Microfrontends

- ✓ Independent Development & Deployment
- ✓ Team Autonomy
- ✓ Continuous Delivery
- ✓ Tech Agnostic
- ✓ Future-Proofing
- ✓ Individual Testing
- ✓ Faster Implementation



Different Approaches of Micro frontend Architecture Integration

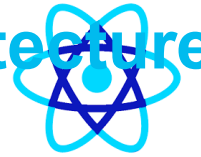


Micro Frontends Architecture Integration Approaches

- ✓ Server-Side Composition
- ✓ Run Time Integration
- ✓ Build Time Integration



Different Approaches of Micro frontend Architecture Integration



Major Categories of Integration

Build-Time Integration *Compile-Time Integration*

Before Container gets loaded in the browser, it gets access to ProductsList source code

Run-Time Integration *Client-Side Integration*

After Container gets loaded in the browser, it gets access to ProductsList source code

Server Integration

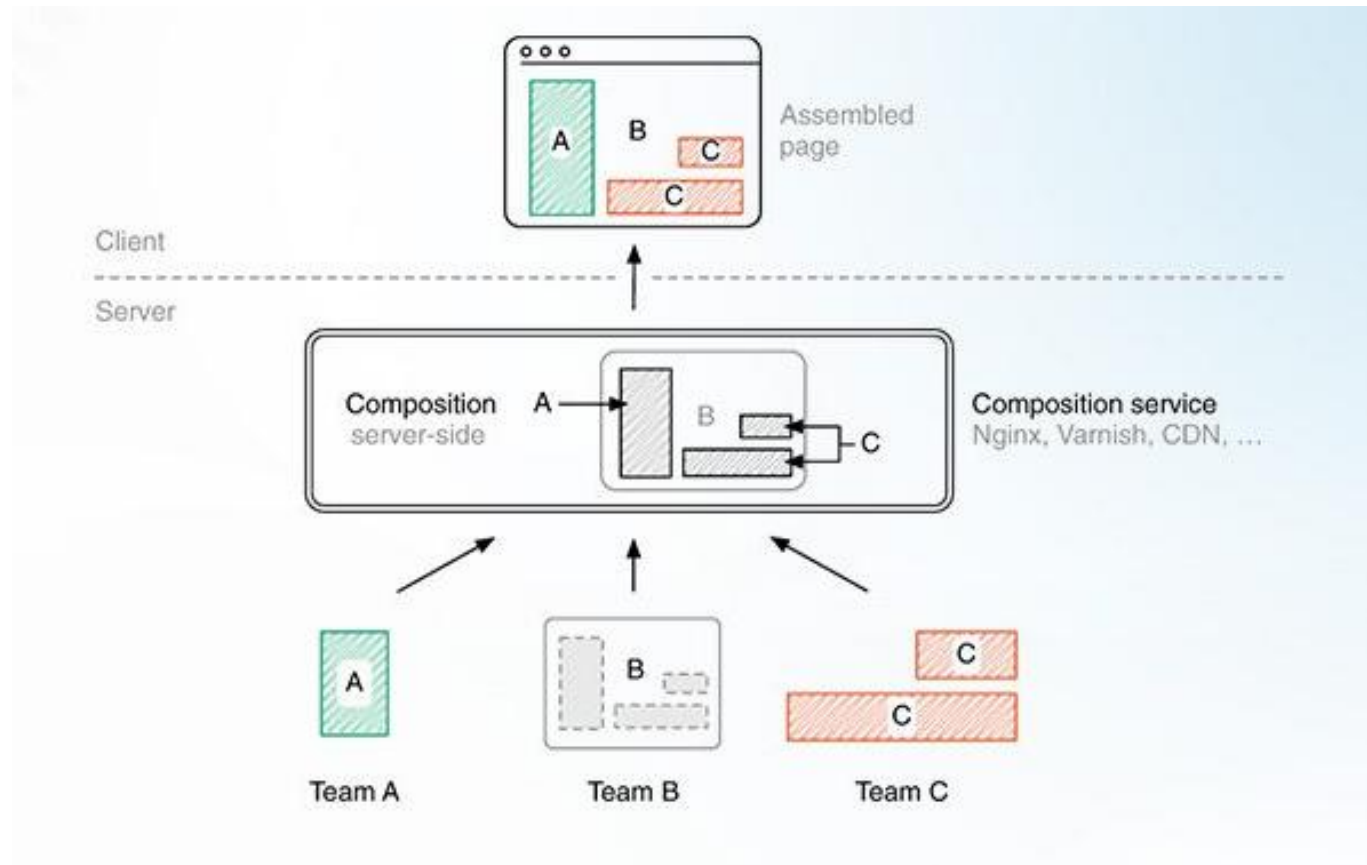
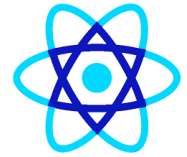
While sending down JS to load up Container, a server decides on whether or not to include ProductsList source

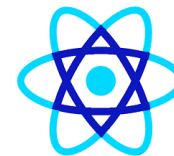


Server-Side Composition

- Micro frontends are cached at Content Delivery Networks (CDNs) by server-side composition, which then composes them into a view that is later served at build time or client runtime.
- It gathers the micro frontend, arranges the view, and composes the finished page.
- It is crucial to ensure the server can manage and adapt scalability because several clients will make different requests if the page is highly customized.

Server-Side Composition

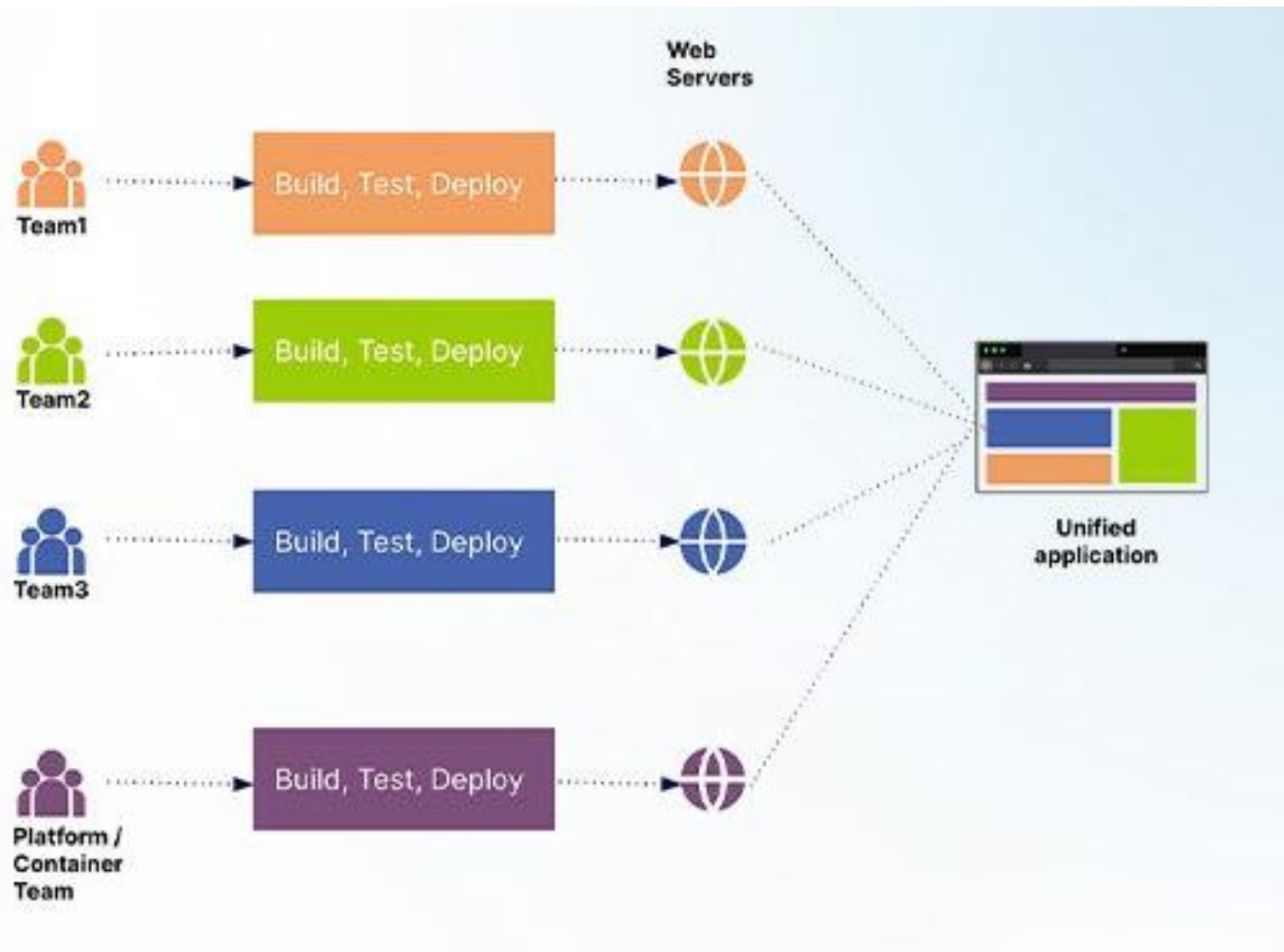
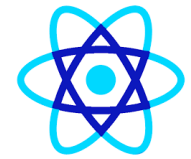




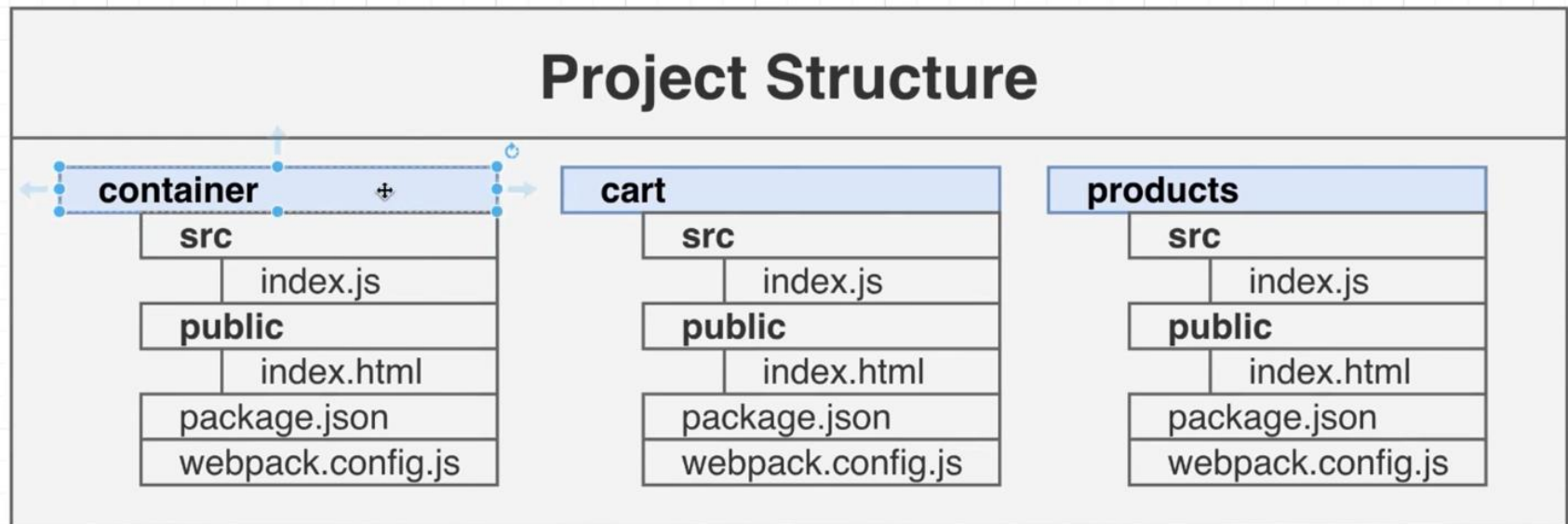
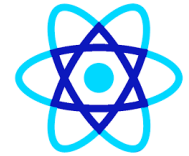
Server-Side Composition

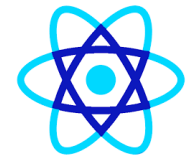
- This approach reduces the need for extensive client-side rendering, improves initial page load times, and ensures a seamless and cohesive user experience.
- Server-side composition in micro frontends allows for efficient content assembly and enhances performance, making it easier to manage and scale micro frontend-based applications.

Run Time Integration



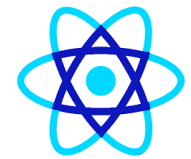
Run Time Integration





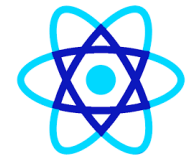
Run Time Integration

- Micro frontend runtime integration provides frontend developers a modular and scalable approach to web application development.
- Teams can work independently on their specific micro frontends while delivering a cohesive user experience through seamless integration within the shell application.
- The runtime Integration technique bundles and configures the micro frontends at runtime.
- We may accomplish runtime integration using JavaScript, iFrames, and web components.



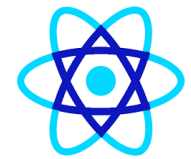
Run Time Integration

Runtime via JavaScript	Microfrontends are loaded and composed in the user's browser using JavaScript. This approach provides flexibility but can increase the initial page load time.
Runtime via iFrames	Each microfrontend is encapsulated in an iFrame, which isolates their JavaScript and styles. Communication between microfrontends can be challenging but offers strong isolation.
Runtime via Web Components	Microfrontends are built as web components, which can be dynamically loaded and rendered within a container application. Web components allow for encapsulation and reusability.

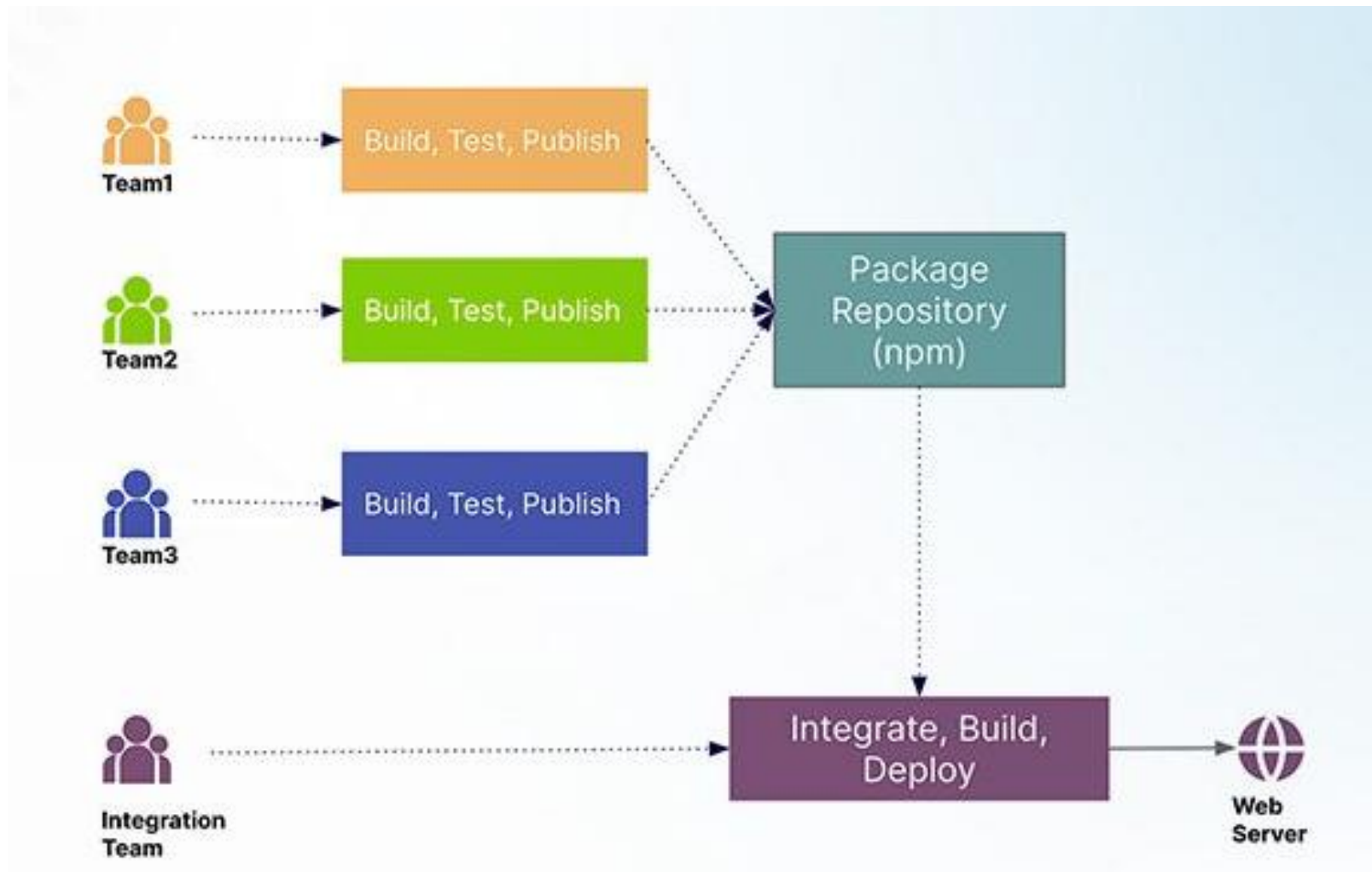


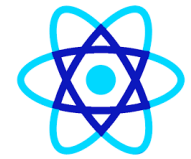
Build Time Integration

- Build-time integration, a tried-and-true technique for developing micro frontends, entails installing components as libraries inside the container.
- Dedicated frontend developers can dedupe common dependencies from apps using the generated deployable JavaScript bundle.
- In this method, each micro frontend must be released and recompiled each time a new product component is released.



Build Time Integration





Build Time Integration

Decides when/where to show each Microfrontend

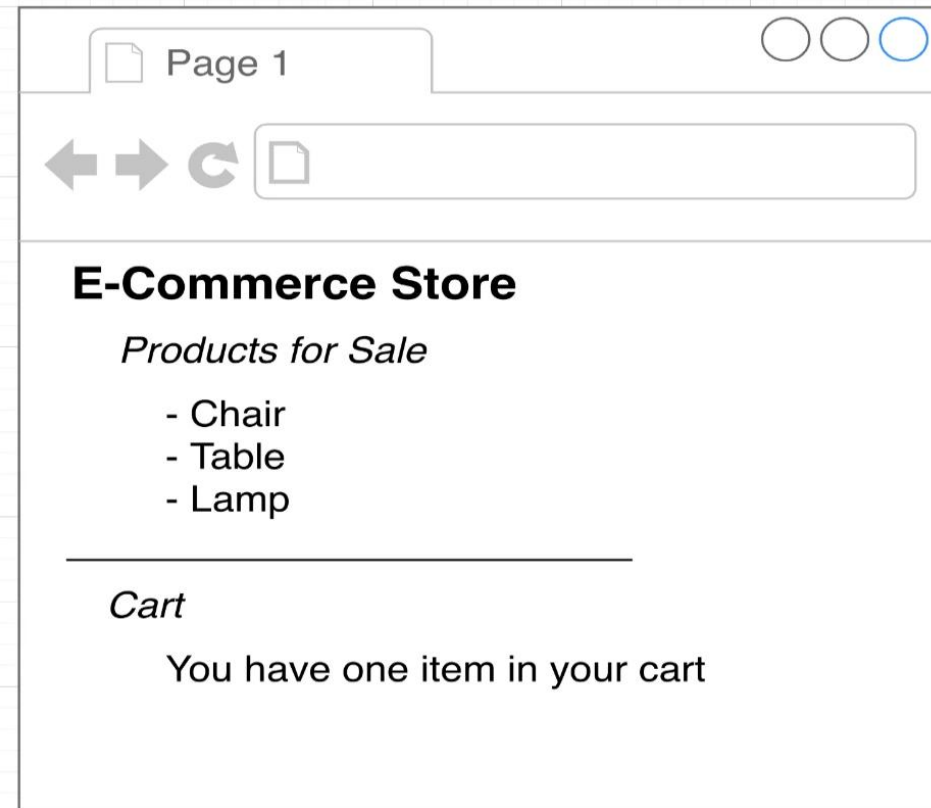
Container

MFE #1

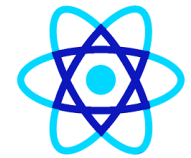
ProductsList

MFE #2

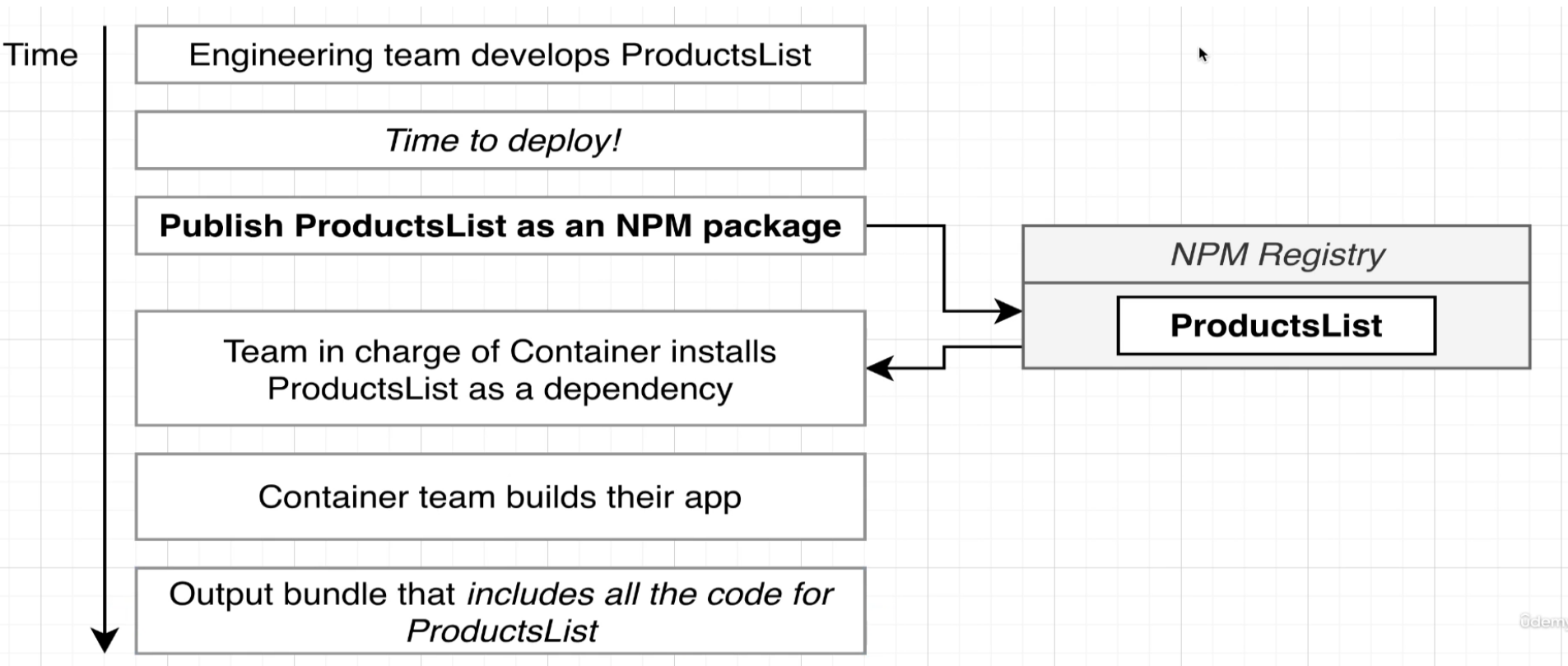
Cart



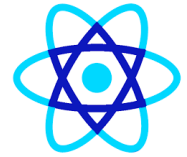
©clenny



Build Time Integration



When to Use Micro frontend Architecture?

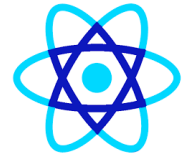


When Microfrontend is a Good Idea?

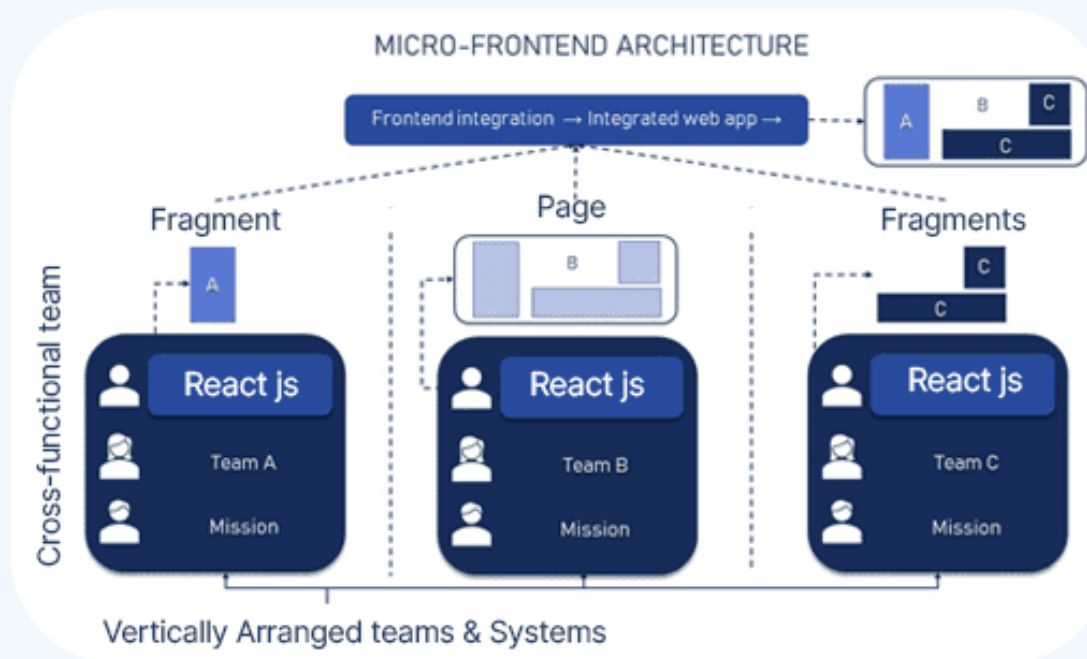
- ✓ Large and Complex Applications:
- ✓ Multiple Autonomous Teams
- ✓ Technological Diversity
- ✓ Independent Deployment
- ✓ Dynamic Content Loading
- ✓ Parallel Development



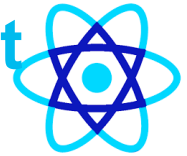
Overview of React Micro Frontend



Micro Frontend Architecture

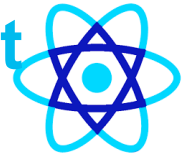


Different Approaches for Micro frontend React Development



Approach	Explanation
Single SPA (Single-Page Application)	This approach involves using a single HTML file that serves as a container for multiple micro frontends, each managed by its own JavaScript framework (such as React).
Module Federation	Introduced by Webpack 5, this approach enables separate React applications to share components and dependencies at runtime. Each micro frontend React is built and deployed independently, and modules can be dynamically loaded into the host application as needed.

Different Approaches for Micro frontend React Development



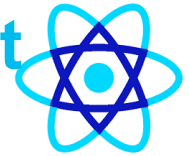
Server-Side Includes (SSI)

With this approach, the server dynamically includes HTML fragments from separate micro frontends to compose the final page. While offering simplicity and server-side rendering capabilities, SSI can lead to tight coupling between micro frontends and potential performance bottlenecks.

IFrames

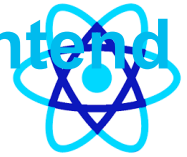
Although less common, IFrames can be used to isolate micro frontends within separate browser contexts. This approach ensures strict separation between micro frontends but may introduce performance overhead and communication challenges.

Different Approaches for Micro frontend React Development

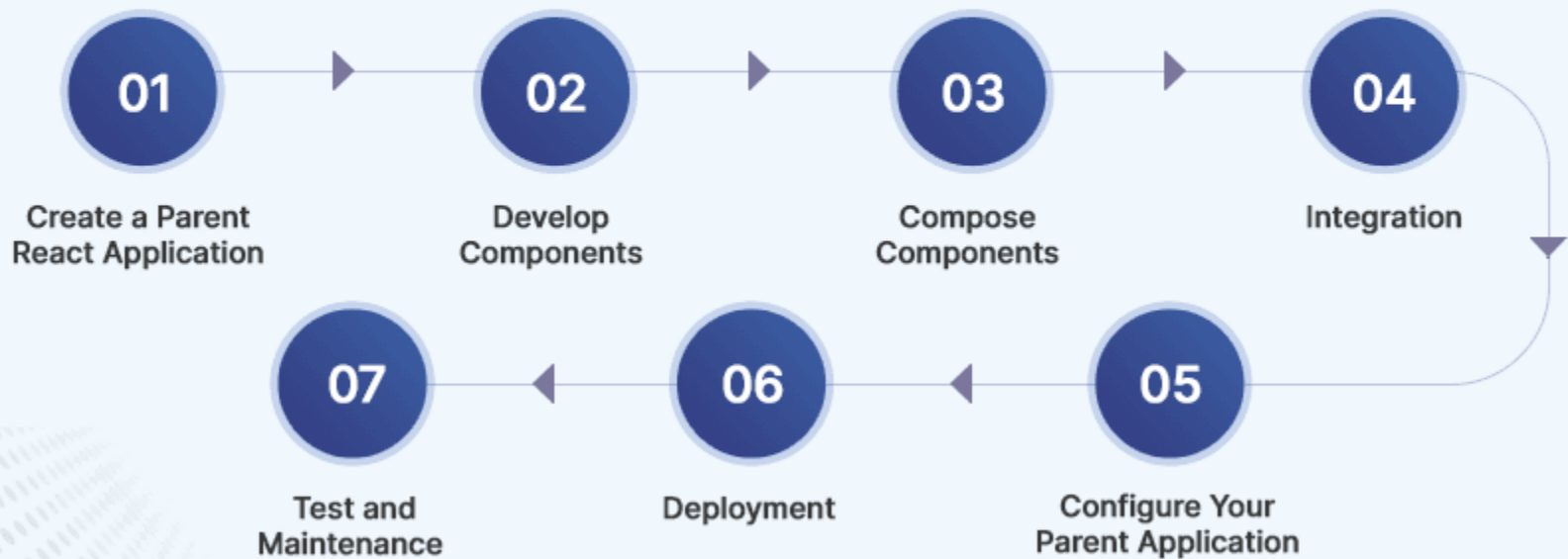


Web Components	
	Leveraging React to create reusable web components allows for encapsulation and reuse across micro frontends and frameworks. This approach promotes component-driven development and interoperability but may require additional tooling and polyfills for browser support.

Step-by-step guide: Building React Micro Frontend Architecture



7 Step Process of Micro Frontend Development with React

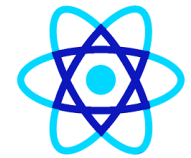




MFE Runtime Integration

Important! We must install slightly newer versions of these libraries to avoid errors. The updated command that should be run is:

```
npm install webpack@5.88.0 webpack-cli@4.10.0 webpack-dev-server@4.7.4  
faker@5.1.0 html-webpack-plugin@5.5.0 --save-exact
```



MFE Runtime Integration

```
{  
  "name": "products",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "start": "webpack"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "dependencies": {  
    "faker": "^5.1.0",  
    "html-webpack-plugin": "5.5.0",  
    "nodemon": "^2.0.15",  
    "webpack": "^5.88.0",  
    "webpack-cli": "4.10.0",  
    "webpack-dev-server": "4.7.4"  
  }  
}
```



MFE Runtime Integration

```
Run `npm audit` for details.
```

```
E:\mfejul2024\ecomm\products>npm run start
```

```
> products@1.0.0 start
```

```
> webpack serve
```

```
<i> [webpack-dev-server] Project is running at:
```

```
<i> [webpack-dev-server] Loopback: http://localhost:8081/
```

```
<i> [webpack-dev-server] On Your Network (IPv4): http://192.168.29.50:8081/
```

```
<i> [webpack-dev-server] On Your Network (IPv6): http://[fe80::3346:77d3:beb6:ac89]:8081/
```

```
<i> [webpack-dev-server] Content not from webpack is served from 'E:\mfejul2024\ecomm\products\public' directory
```

```
asset main.js 3.37 MiB [emitted] (name: main)
```

```
runtime modules 27.4 KiB 12 modules
```

```
modules by path ./node_modules/faker/lib/locales/ 1.93 MiB 1468 modules
```

```
modules by path ./node_modules/faker/lib/*.js 133 KiB 27 modules
```

```
modules by path ./node_modules/webpack-dev-server/client/ 56.8 KiB 12 modules
```

```
modules by path ./node_modules/webpack/hot/*.js 5.18 KiB 4 modules
```

```
modules by path ./node_modules/html-entities/lib/*.js 78.9 KiB
```

```
./node_modules/html-entities/lib/index.js 4.84 KiB [built] [code generated]
```

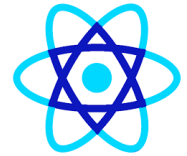
```
+ 3 modules
```



MFE Runtime Integration

```
ucts\public' directory
asset main.js 3.37 MiB [emitted] (name: main)
runtime modules 27.4 KiB 12 modules
modules by path ./node_modules/faker/lib/locales/ 1.93 MiB 1468 modules
modules by path ./node_modules/faker/lib/*.js 133 KiB 27 modules
modules by path ./node_modules/webpack-dev-server/client/ 56.8 KiB 12 modules
modules by path ./node_modules/webpack/hot/*.js 5.18 KiB 4 modules
modules by path ./node_modules/html-entities/lib/*.js 78.9 KiB
  ./node_modules/html-entities/lib/index.js 4.84 KiB [built] [code generated]
+ 3 modules
modules by path ./node_modules/faker/vendor/*.js 21.1 KiB
  ./node_modules/faker/vendor/unique.js 2.79 KiB [built] [code generated]
+ 2 modules
modules by path ./node_modules/faker/lib/image_providers/*.js 10.9 KiB
  ./node_modules/faker/lib/image_providers/lorempixel.js 5.11 KiB [built] [code generated]
+ 2 modules
+ 4 modules
webpack 5.93.0 compiled successfully in 3174 ms
```


MFE Runtime Integration



products E:\mfejul2024\ecomm\products

dist

main.js

node_modules library root

src

package.json

package-lock.json

webpack.config.js

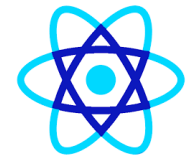
External Libraries

Scratches and Consoles

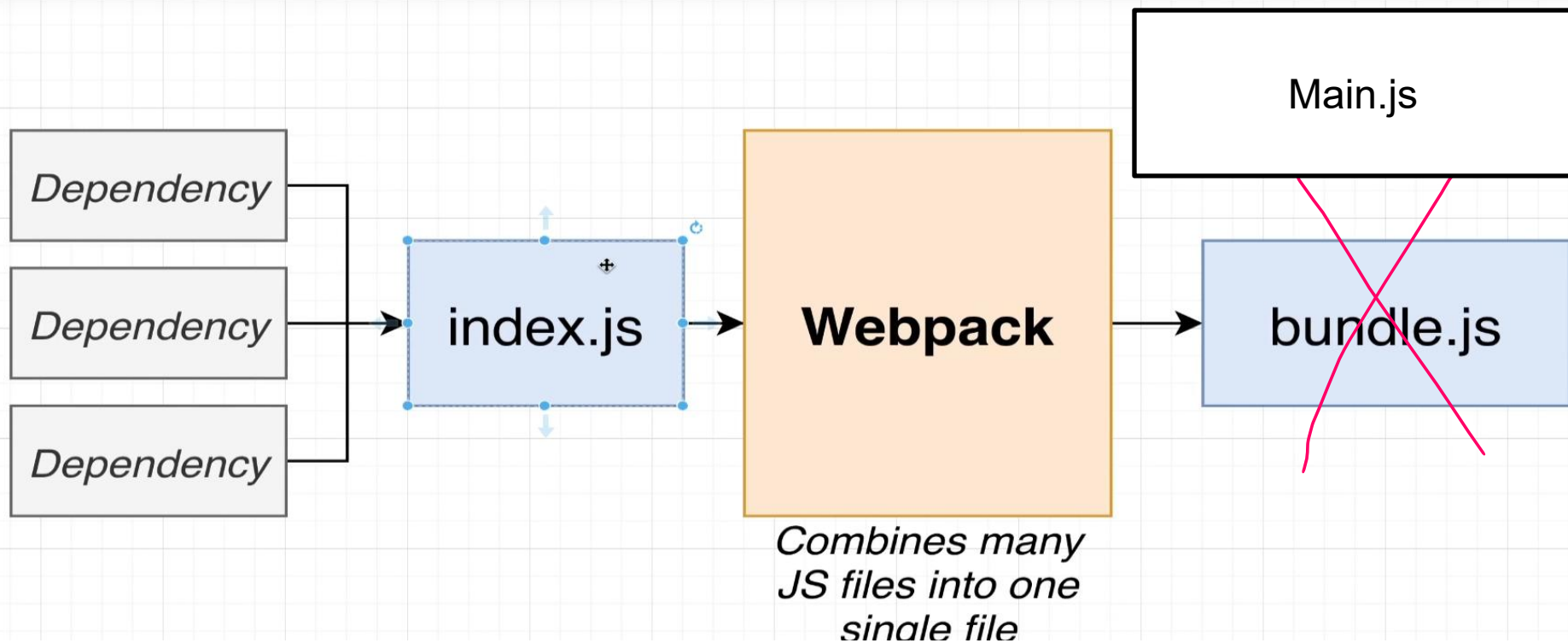
Visual layout of bidirectional text can depend on the base direction (View | Bidi Text Base Direction)

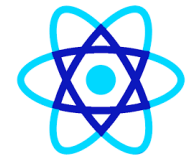
This document contains very long lines. Soft wraps were enabled to improve editor performance.

```
1  /*
2  ⚠ ATTENTION: The "eval" devtool has been used (maybe by default in mode: "development")
3  * This devtool is neither made for production nor for readable output files.
4  * It uses "eval()" calls to create a separate source file in the browser devtools.
5  * If you are trying to read the output file, select a different devtool (https://webpack.js.org/configuration/devtool/#devtool)
6  * or disable the default devtool with "devtool: false".
7  * If you are looking for production-ready output files, see mode: "production" (https://webpack.js.org/configuration/mode/).
8  */
9  /*****/ (() => { // webpackBootstrap
10 /*****/   var __webpack_modules__ = ({
11
12   /***/ "./node_modules/faker/index.js":
13   /*!*****!*\
14     !*** ./node_modules/faker/index.js ***!
15     \*****/
16   /***/ ((module, __unused_webpack_exports, __webpack_require__) => {
17
18     eval("// since we are requiring the top level of faker, load all locales by default\n
19     ./lib */ \"../node_modules/faker/lib/index.js\");\nvar faker = new Faker({ locales: [\n
20     \"../node_modules/faker/lib/locales.js\"] });\nmodule['exports'] = faker;\n\n// source\n
21     ./node_modules/faker/index.js?");
```

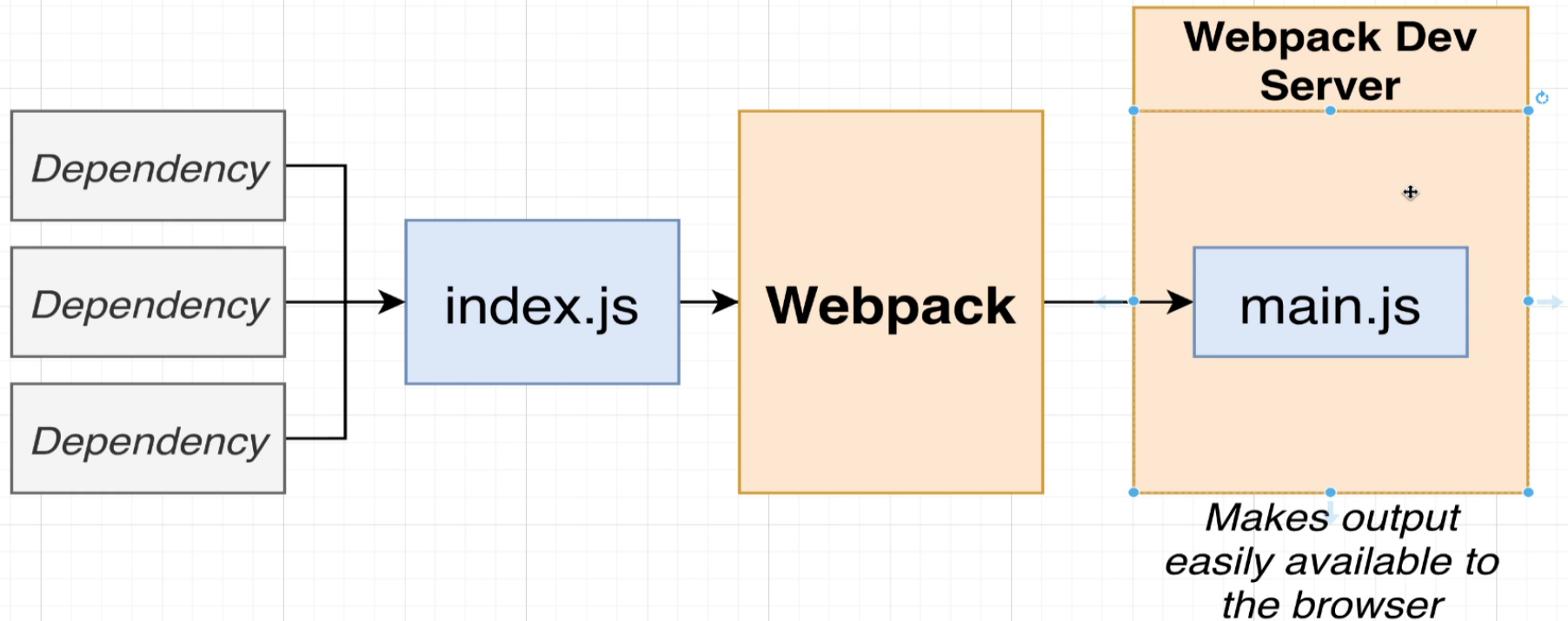


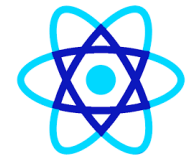
MFE Runtime Integration



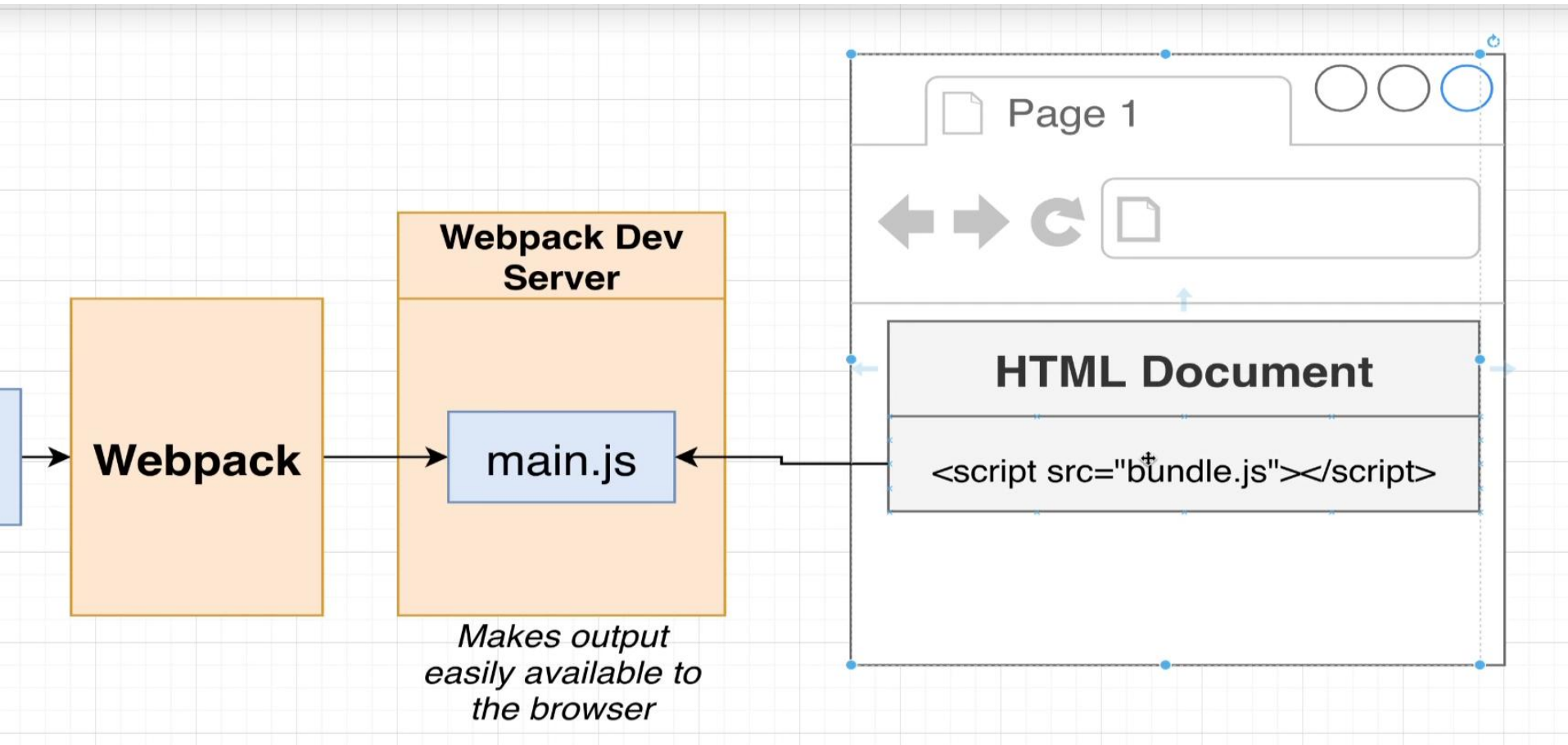


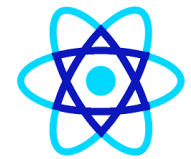
MFE Runtime Integration





MFE Runtime Integration

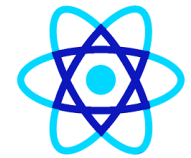




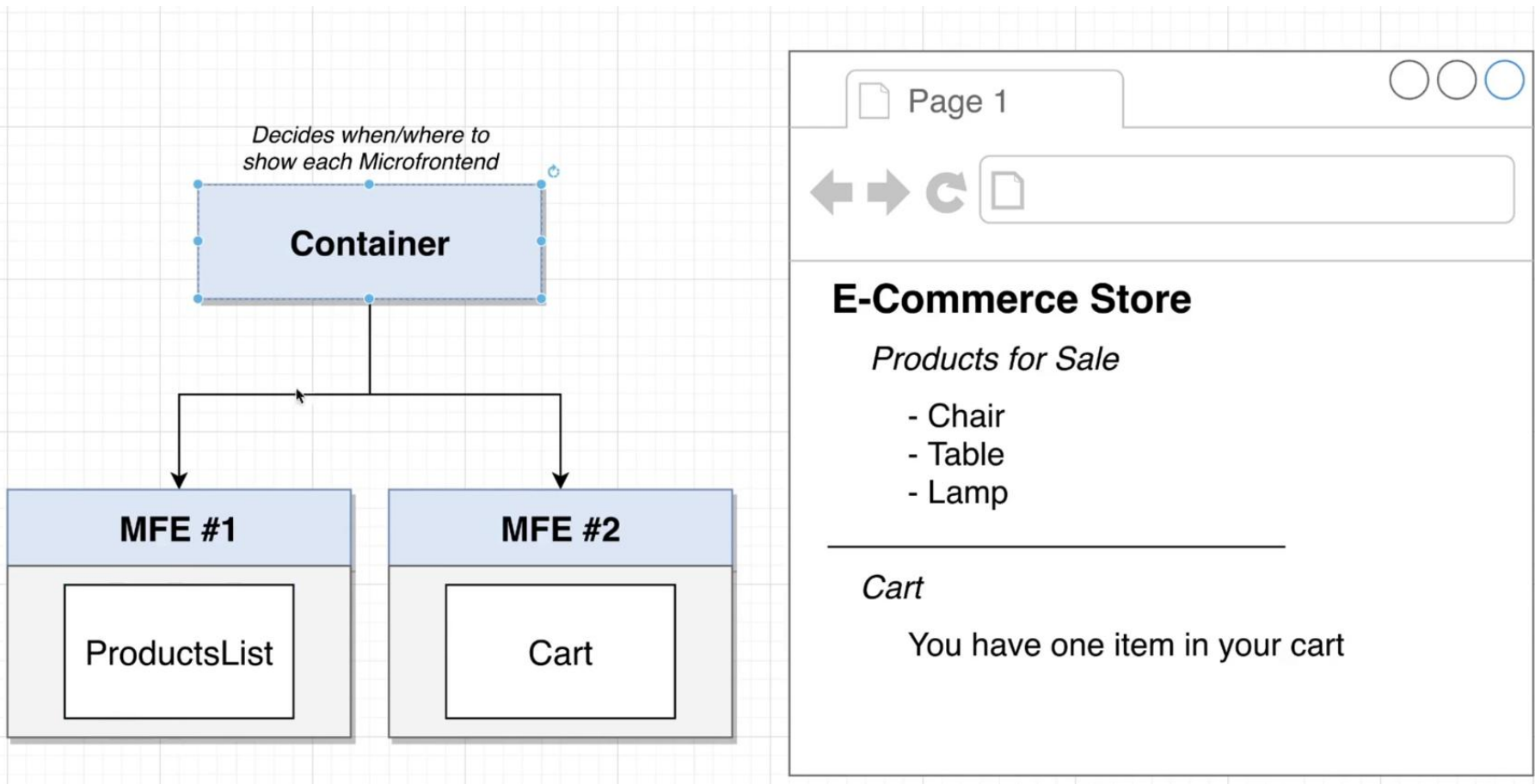
MFE Runtime Integration



Refined Steel Car
Generic Concrete Shoes
Incredible Steel Computer



MFE Runtime Integration

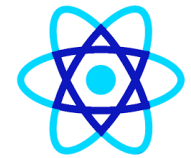




MFE Runtime Integration

Important! We must install slightly newer versions of these libraries to avoid errors. The updated command that should be run is:

```
npm install webpack@5.88.0 webpack-cli@4.10.0 webpack-dev-server@4.7.4  
html-webpack-plugin@5.5.0 nodemon --save-exact
```



MFE Runtime Integration

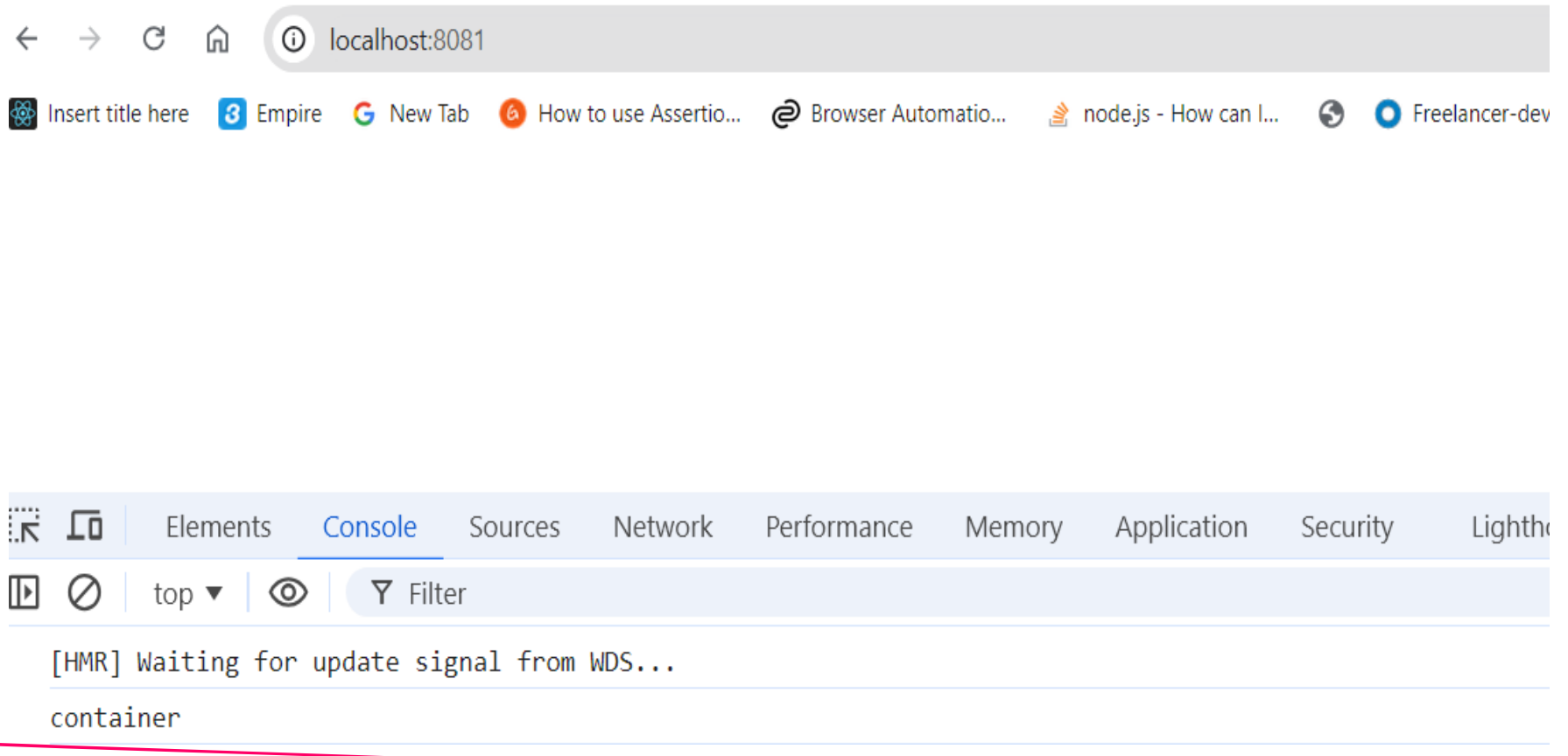
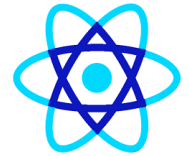
Project ▾

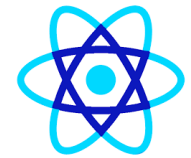
- ▾ **ecom** E:\mfejul2024\ecom
- ▾ container
 - > **node_modules** library root
 - ▾ public
 - <> index.html
 - ▾ src
 - index.js**
 - package.json**
 - package-lock.json
 - webpack.config.js**

index.js × **index.html** **webpack.config.js** **package.json**

```
1 console.log('container')
```


MFE Runtime Integration





Implementing Module Federation

Diagram ▾ Shapes Format Insert Delete ↶ ↷ [] 🔍 🔍 All changes saved

Designate one app as the Host and one as the Remote

In the Remote, decide which modules (files) you want to make available to other projects

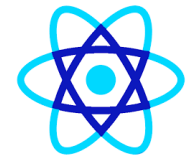
Set up Module Federation plugin to *expose* those files

In the Host, decide which files you want to get from the remote

Set up Module Federation plugin to fetch those files

In the Host, refactor the entry point to load asynchronously

In the Host, import whatever files you need from the remote



Implementing Module Federation

Designate one app as the Host (CONTAINER) and one as the Remote (PRODUCTS)

In the Remote, decide which modules (files) you want to make available to other projects

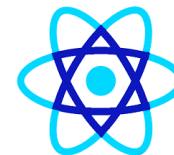
Set up Module Federation plugin to *expose* those files

In the Host, decide which files you want to get from the remote

Set up Module Federation plugin to fetch those files

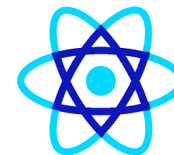
In the Host, refactor the entry point to load asynchronously

In the Host, import whatever files you need from the remote

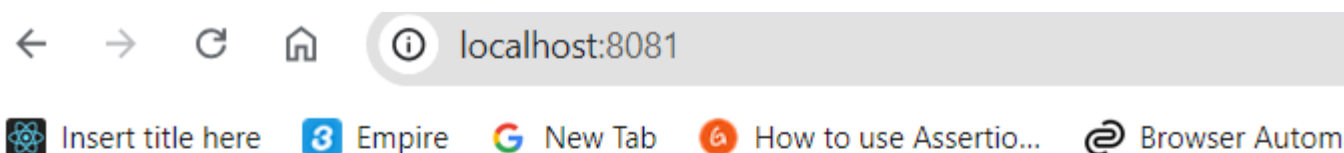


Implementing Module Federation

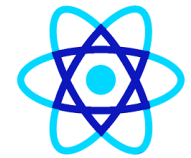
```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const ModuleFederationPlugin=require('webpack/lib/container/ModuleFederationPlugin')
module.exports={
  mode:'development',
  devServer: {
    port:8082
  },
  plugins: [
    new ModuleFederationPlugin({
      name:'products',
      filename:'remoteEntry.js',
      exposes:{
        './ProductsIndex': './src/index',
      },
    }),
    new HtmlWebpackPlugin( options: {
      template: "./public/index.html"
    })
  ]
};
```



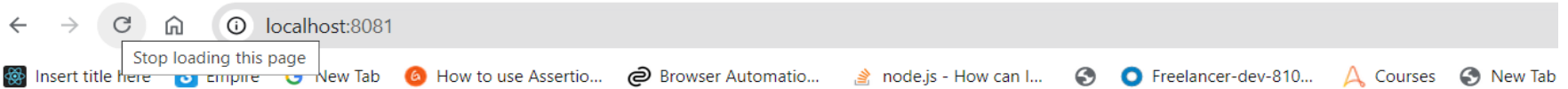
Implementing Module Federation



Awesome Soft Soap
Awesome Concrete Fish
Refined Granite Keyboard

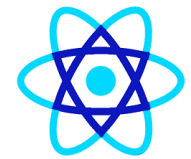


Implementing Module Federation



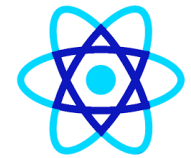
Small Steel Sausages
Handcrafted Soft Keyboard
Small Concrete Cheese

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Perform			
⏏ ⌵ 🔍 ☐ Preserve log ☐ Disable cache No throttling ⌵ 📶 ⬆ ⬇			
🔍 Filter ☐ Invert ☐ Hide data URLs ☐ Hide extension URLs All Fetch/XHR Doc CSS JS Font Img Media M			
☐ Blocked requests ☐ 3rd-party requests			
100 ms 200 ms 300 ms 400 ms 500 ms 600 ms 700 ms			
Name	Status	Type	Initiator
main.js	304	script	(index):5
remoteEntry.js	200	script	main.js:445
src_bootstrap.js.js	304	script	main.js:445
vendors-node_modules_faker_index.js.js	200	script	remoteEntry.js:423
src_index.js.js	200	script	remoteEntry.js:423
injected-script.js	200	script	content_script.js:5
content_script_vite-148e3621.js	200	script	content_script_vite.js:1
prompt.js	200	script	playback.js:1

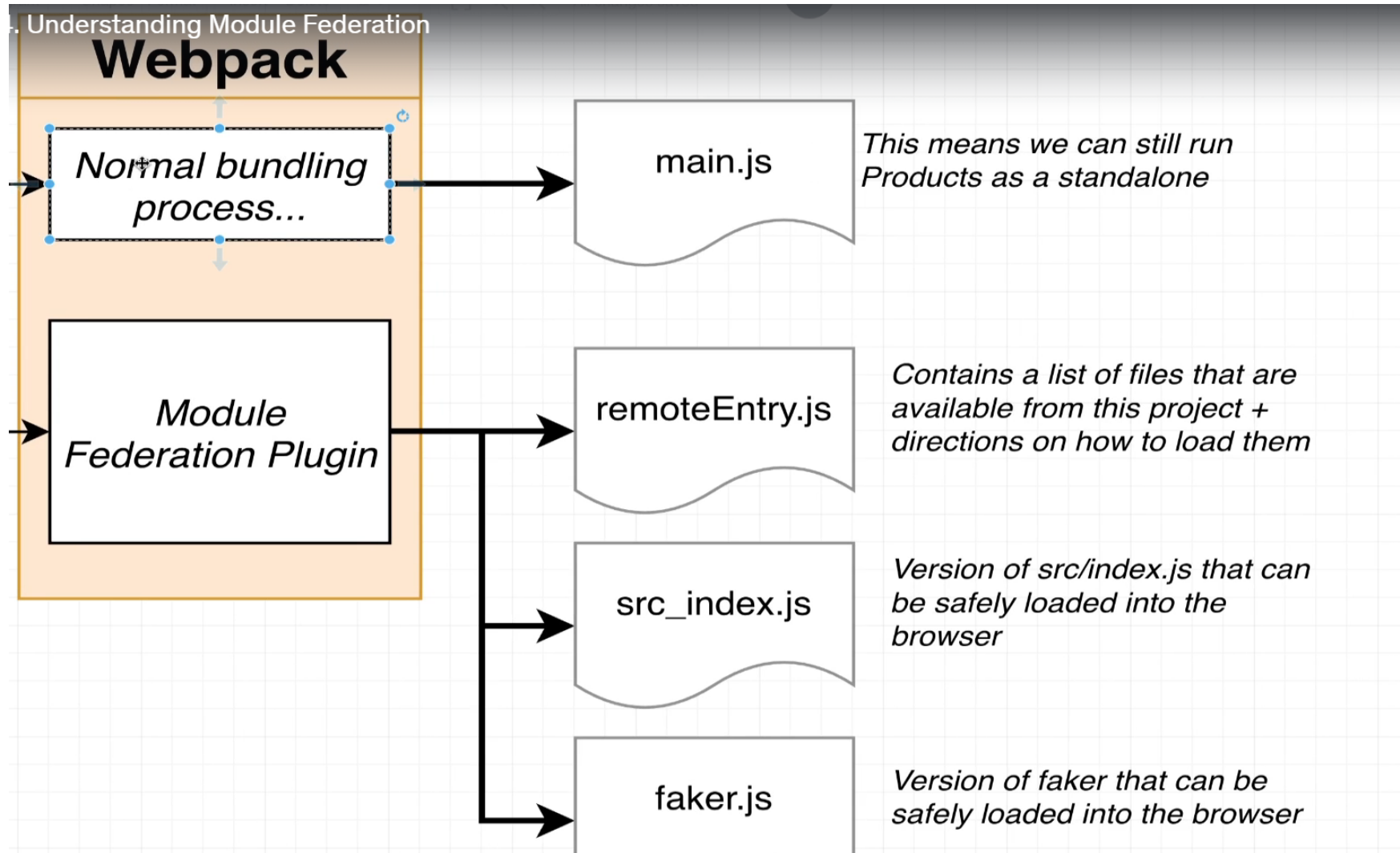


Implementing Module Federation

Name	Url	Status	Type
 main.js	http://localhost:8081/main.js	304	script
 remoteEntry.js	http://localhost:8082/remoteEntry.js	200	script
 src_bootstrap.js.js	http://localhost:8081/src_bootstrap_j...	304	script
 vendors-node_modules_faker_index_js.js	http://localhost:8082/vendors-node_...	200	script
 src_index_js.js	http://localhost:8082/src_index_js.js	200	script
 injected-script.js	chrome-extension://hmmohkncibpkl...	200	script
 content_script_vite-148e3621.js	chrome-extension://idgpnmonknjno...	200	script
 prompt.js	chrome-extension://mooikfkahbdckl...	200	script



Implementing Module Federation





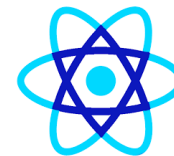
Webpack – Bootstrap.js

- Using bootstrap.js with Webpack involves bundling Bootstrap's JavaScript code with your project, allowing you to modularize and optimize the loading of Bootstrap's components.
- Webpack is a module bundler that helps manage dependencies and assets in modern front-end applications.
- When working with Bootstrap and Webpack, you can either load only the Bootstrap components you need or include the full bootstrap.js library.



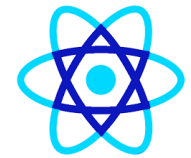
Webpack – Bootstrap.js

- Using bootstrap.js with Webpack involves bundling Bootstrap's JavaScript code with your project, allowing you to modularize and optimize the loading of Bootstrap's components.
- Webpack is a module bundler that helps manage dependencies and assets in modern front-end applications.
- When working with Bootstrap and Webpack, you can either load only the Bootstrap components you need or include the full bootstrap.js library.

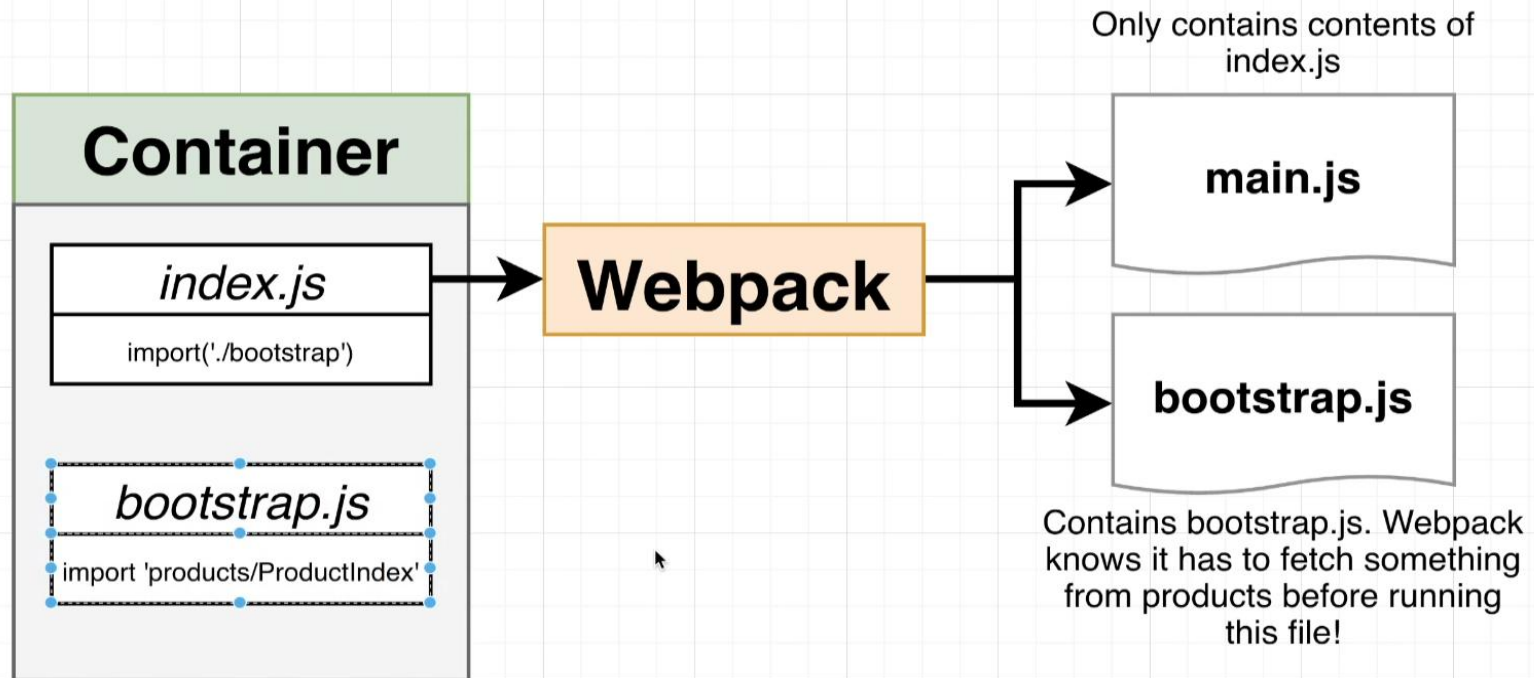


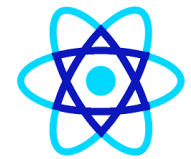
Webpack – Bootstrap.js

- Webpack Module Federation is a feature introduced in Webpack 5 that allows multiple, independently built and deployed JavaScript applications (or "modules") to share code and dependencies between them.
- When working with Bootstrap.js in a micro frontend architecture using Webpack Module Federation, you can share Bootstrap's JavaScript functionality across different applications (or micro frontends) without duplicating it in every build, thus optimizing load time and reducing redundancy.

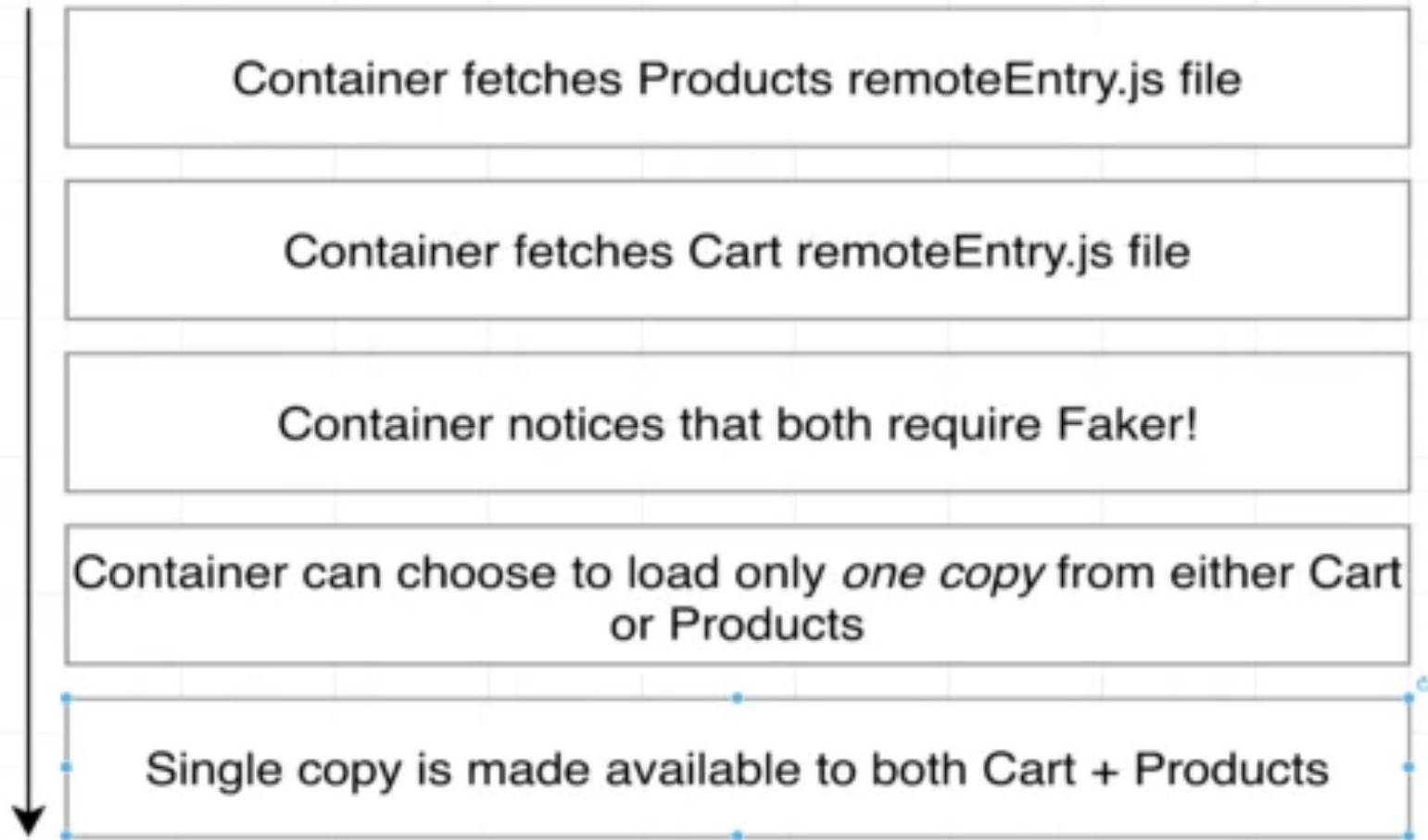


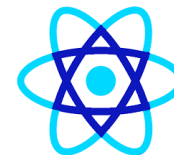
Implementing Module Federation





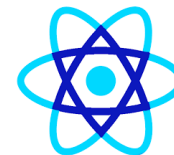
Shared Modules





Shared Modules

```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const ModuleFederationPlugin=require('webpack/lib/container/ModuleFederationPlugin')
module.exports={
  mode:'development',
  devServer: {
    port:8082
  },
  plugins: [
    new ModuleFederationPlugin({
      name:'products',
      filename:'remoteEntry.js',
      exposes:{
        './ProductsIndex': './src/index',
        shared:["faker"]
      },
      new HtmlWebpackPlugin( options: {
        template: "./public/index.html"
      })
    ]
  }
};
```



Shared Modules

```
const HtmlWebpackPlugin = require("html-webpack-plugin");
const ModuleFederationPlugin=require('webpack/lib/container/ModuleFederationPlugin')
module.exports={
  mode:'development',
  devServer: {
    port:8083
  },
  plugins: [
    new ModuleFederationPlugin({
      name:'cart',
      filename:'remoteEntry.js',
      exposes:{
        './CartShow': './src/index',
      },
      shared:["faker"]
    }),
    new HtmlWebpackPlugin( options: {
      template: "./public/index.html"
    })
  ]
};
```



Shared Modules

[HMR] Waiting for update signal from WDS...

✖ ▶ Uncaught

```
Error: Shared module is not available for eager consumption: webpack/sharing/consume/default/faker/faker
  at __webpack_require__.m.<computed> (main.js:1036:54)
  at __webpack_require__ (main.js:282:32)
  at fn (main.js:549:21)
  at eval (index.js:2:63)
  at ./src/index.js (main.js:255:1)
  at __webpack_require__ (main.js:282:32)
  at main.js:1660:37
  at main.js:1662:12
```




Shared Modules

```
import('./bootstrap')
```



```
import faker from 'faker'
```

```
const cartText : string = `<div>You have ${faker.random.number()} items in the cart</div>`;
```



```
document.querySelector(selectors: "#dev-cart").innerHTML=cartText;
```



Shared Modules

🔍 | 🔇 | top ▼ | 👁 | Filter

[webpack-dev-server] Live Reloading enabled.

[webpack-dev-server] Hot Module Replacement enabled.

[webpack-dev-server] Live Reloading enabled.

chrome content script has been inserted

Start ajax tracing with delay of '0'.

start with href <http://localhost:8082/>

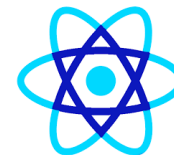
indicate that add on has been installed

check if opened window is for submitting to JIRA

window name = null

opened window is not for submitting to JIRA

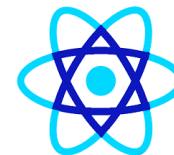
> |



Shared Modules with different versions

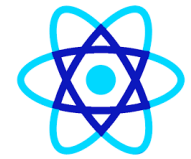
```
{
  "name": "carts",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "webpack serve"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "faker": "^4.1.0",
    "html-webpack-plugin": "5.5.0",
    "nodemon": "^2.0.15",
    "webpack": "^5.88.0",
    "webpack-cli": "4.10.0",
    "webpack-dev-server": "4.7.4"
  }
}
```

```
{
  "name": "products",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "webpack serve"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "faker": "^5.1.0",
    "html-webpack-plugin": "5.5.0",
    "nodemon": "^2.0.15",
    "webpack": "^5.88.0",
    "webpack-cli": "4.10.0",
    "webpack-dev-server": "4.7.4"
  }
}
```

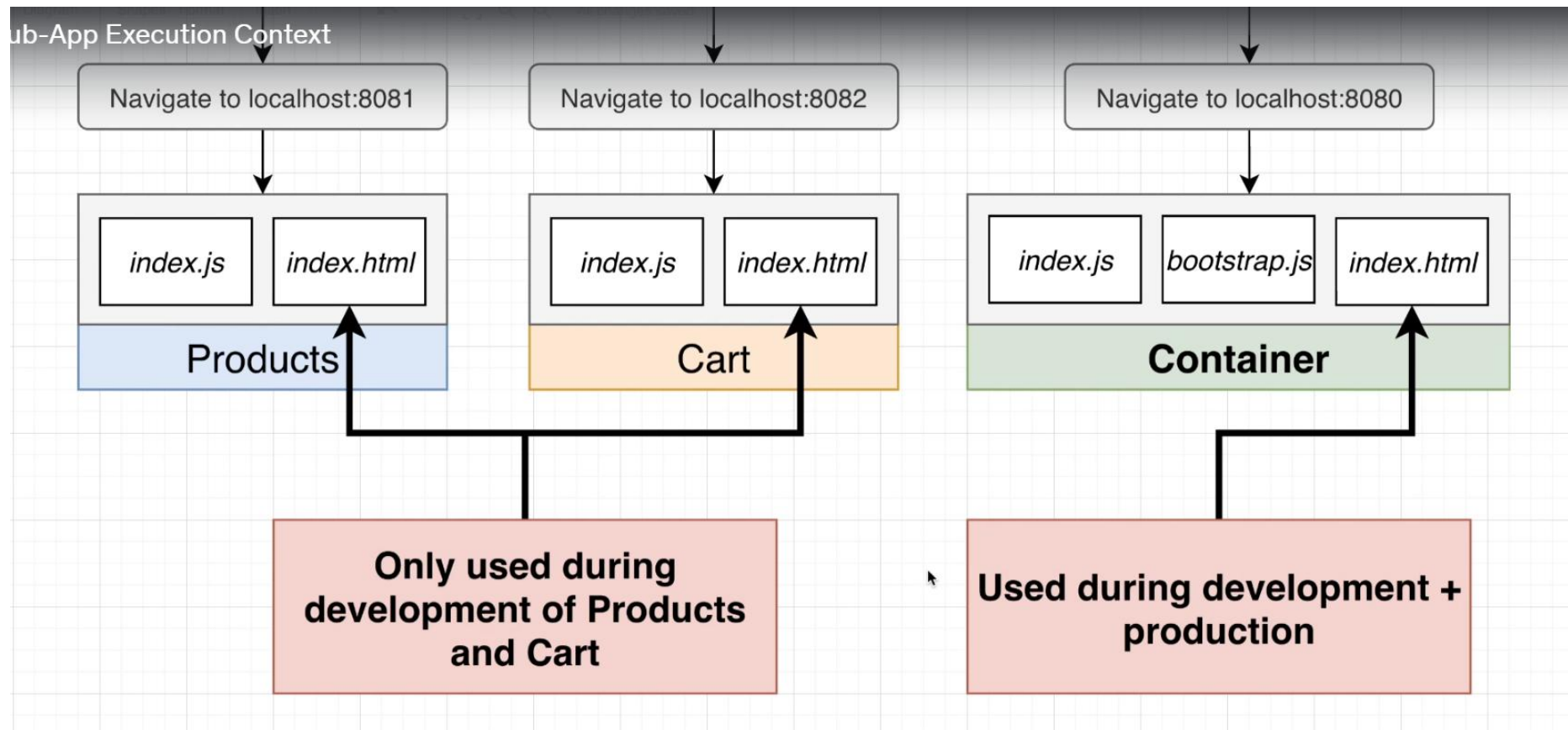


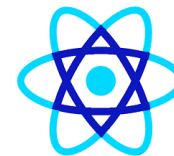
Singleton Loading

```
mode: 'development',
devServer: {
  port: 8083
},
plugins: [
  new ModuleFederationPlugin({
    name: 'cart',
    filename: 'remoteEntry.js',
    exposes: {
      './CartShow': './src/index',
    },
    // shared: ["faker"]
    shared: {
      faker: {
        singleton: true
      }
    }
  })
],
```



Sub App Execution





Remote Module

```
import faker from 'faker'
```

1+ usages

```
const mount=(el) :void =>{  
  let products :string = '';  
  for(let i :number =0;i<3;i++){  
    const name= faker.commerce.productName();  
    products+=`<div>${name}</div>`  
  }  
  el.innerHTML=products;  
}  
  
if(process.env.NODE_ENV === 'development'){  
  const el :Element =document.querySelector(selectors: '#dev-products');  
  if(el){  
    mount(el);  
  }  
}  
export {mount};
```

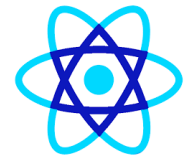


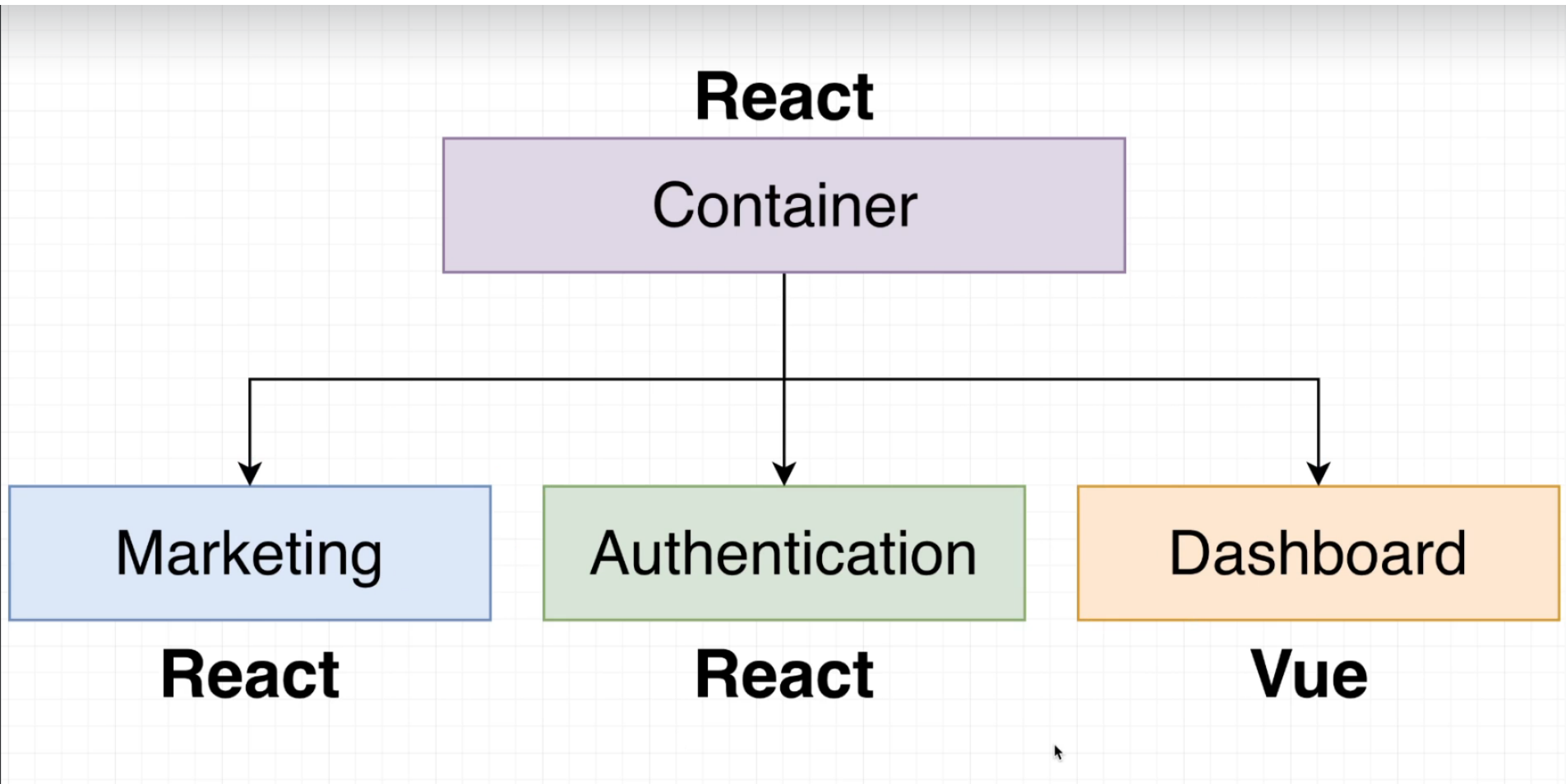
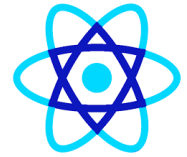
Remote Module

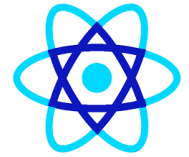
```
import {mount as productsMount} from 'products/ProductsIndex',  
import {mount as cartMount} from 'cart/CartShow';  
console.log('Container!');
```

```
productsMount(document.querySelector(selectors: '#my-products'));  
cartMount(document.querySelector(selectors: '#my-cart'))
```

Application Overview



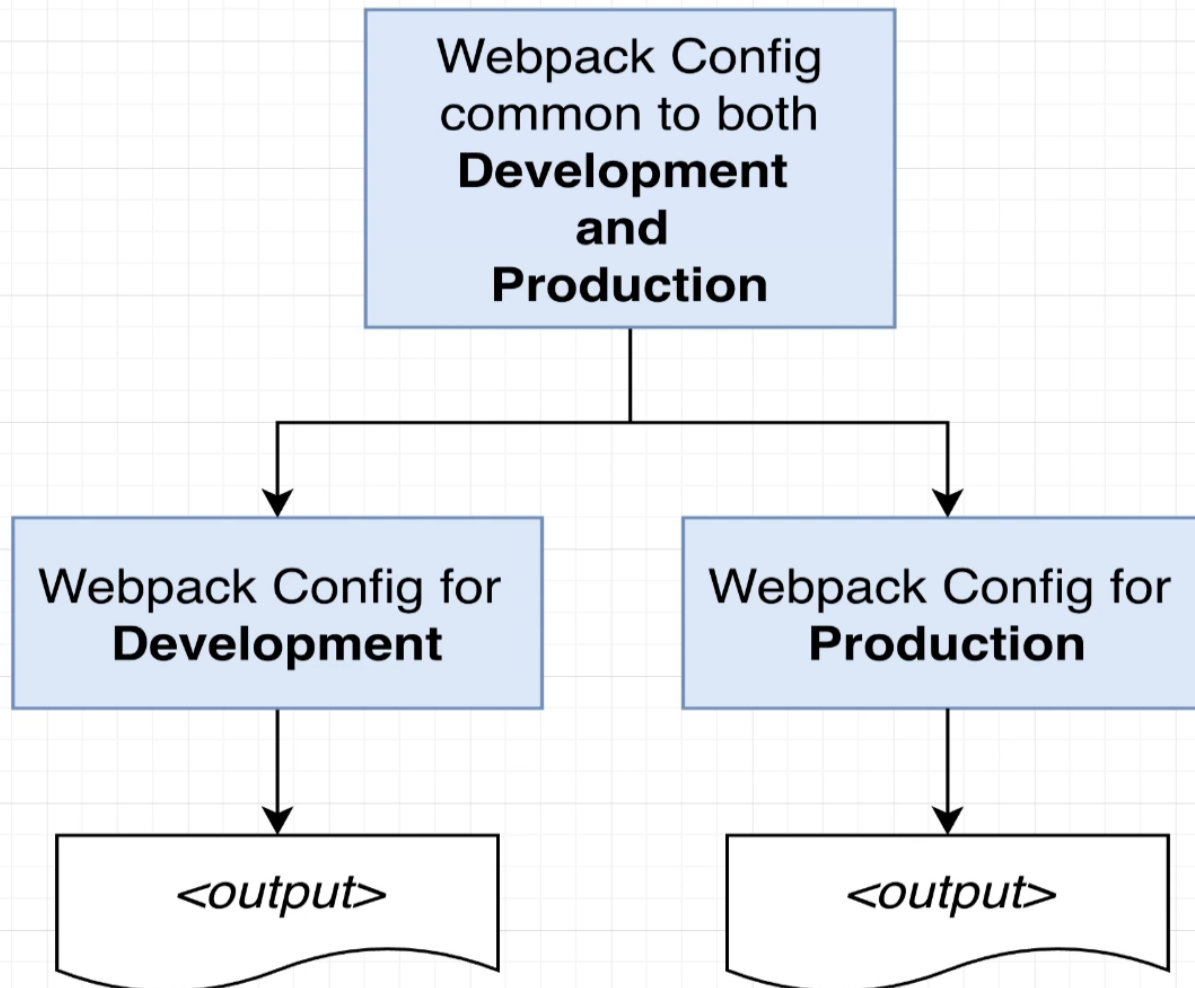
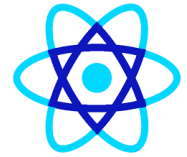




We are setting up all projects from scratch



Why no Create-React-App or Vue CLI?



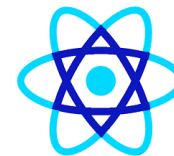


Reactjs Typescript

- `npx create-react-app customer-app --template typescript`

Questions





Module Summary

- Reactjs
- JSX, Props and State
- Events
- Component Interaction

